

**VIETNAM NATIONAL UNIVERSITY, HANOI
UNIVERSITY OF ENGINEERING AND TECHNOLOGY**



KHONG NGOC ANH

**GENERALIZED INVERSES
AND THEIR APPLICATIONS IN
ARTIFICIAL INTELLIGENCE AND OPTIMIZATION**

BACHELOR'S THESIS

Major: Artificial Intelligence

HA NOI - 2026

**VIETNAM NATIONAL UNIVERSITY, HANOI
UNIVERSITY OF ENGINEERING AND TECHNOLOGY**

KHONG NGOC ANH

**GENERALIZED INVERSES
AND THEIR APPLICATIONS IN
ARTIFICIAL INTELLIGENCE AND OPTIMIZATION**

BACHELOR'S THESIS

Major: Artificial Intelligence

Supervisor: Dr. Nguyen Bich Van

HA NOI - 2026

Abstract

The concept of matrix inversion is fundamental in linear algebra, but the classical inverse is restricted to square and nonsingular matrices. In many practical problems arising in applied mathematics, optimization, and machine learning, one instead encounters rectangular, singular, or ill-conditioned matrices for which the ordinary inverse does not exist. This thesis studies the generalized inverse, with particular emphasis on the Moore–Penrose pseudoinverse, as a rigorous and computationally useful extension of the classical inverse.

The thesis first develops the mathematical foundation of generalized inverses, including their definitions, existence properties, projection interpretations, and their role in least-squares and minimum-norm solutions. It then reviews the main computational approaches for pseudoinverse evaluation, focusing on Singular Value Decomposition (SVD)-based methods and hyperpower iterative methods. The numerical behavior of these approaches is analyzed from the perspectives of accuracy, efficiency, and stability, especially in the presence of rank deficiency and ill-conditioning.

A central application considered in this thesis is the Extreme Learning Machine (ELM), a single-hidden-layer feedforward neural network in which the hidden-layer parameters are randomly assigned and the output weights are determined analytically. In this framework, the learning problem reduces to solving the linear system $\mathbf{H}\beta = \mathbf{T}$, where the output weights are computed through the pseudoinverse of the hidden-layer output matrix. This makes the quality of pseudoinverse computation a decisive factor in the performance and stability of ELM.

To address this issue, the thesis compares SVD-based and hyperpower-based pseudoinverse solvers both theoretically and experimentally. The results show that SVD-based methods provide the most robust and numerically stable solutions, particularly for ill-conditioned matrices, while hyperpower methods may be competitive in favorable settings but are more sensitive to conditioning, initialization, and stopping criteria. The thesis also discusses regularization as a possible stabilization mechanism for pseudoinverse-based ELM.

Experimental studies are carried out on synthetic matrix benchmarks and on capped sub-

sets of MNIST, Fashion-MNIST, and ISOLET for ELM classification. The findings confirm that generalized inverses provide an effective bridge between linear algebra theory and machine learning practice, and that the choice of pseudoinverse algorithm has a direct influence on the speed, stability, and predictive performance of the resulting model. The thesis further develops a spectral interpretation of ELM stability through the singular-value structure of the hidden-layer matrix and investigates width-dependent conditioning behavior using tools from random matrix theory. Overall, the thesis demonstrates that the Moore–Penrose inverse is not only a theoretical tool for generalized matrix inversion, but also a practical computational framework for modern data-driven learning systems.

Keywords: *Generalized Inverse, Moore-Penrose Inverse, Pseudoinverse, Extreme Learning Machine.*

Declaration of Authorship

I hereby declare that this thesis entitled "**Generalized Inverses and their applications in Artificial Intelligence and Optimization**" is my own work under the supervision of Dr. Nguyen Bich Van. The research and conclusions presented in this thesis are truthful and have not been copied from any source in any form. All references to previous works have been properly cited and acknowledged according to academic standards.

I take full responsibility for this declaration.

Hanoi, June 15, 2026

Author

Khong Ngoc Anh

Acknowledgments

I would like to express my sincere gratitude to my supervisor, **Dr. Nguyen Bich Van**, for her invaluable guidance, insightful feedback, and continuous support throughout this research. Her expertise and encouragement have been instrumental in shaping this work.

I am deeply honored to be a member of the first cohort of the Artificial Intelligence program at the **Institute for Artificial Intelligence, University of Engineering and Technology – Vietnam National University, Hanoi**. I am grateful to the faculty members for their dedication in building this pioneering program and providing an excellent academic foundation in AI and machine learning.

I would like to thank my colleagues and fellow students for creating a vibrant learning community. The collaborative spirit, stimulating discussions, and mutual support have greatly enriched my academic experience and understanding of the research area.

Finally, I owe my deepest gratitude to my family for their unconditional love, patience, and unwavering support throughout my academic journey. Their encouragement has been the foundation of my success.

Hanoi, 2026

Khong Ngoc Anh

Contents

Abstract	i
Declaration of Authorship	iii
Acknowledgments	iv
List of Abbreviations and Notations	xiii
1 Introduction	1
1.1 Motivation and Background	1
1.2 Research Objectives and Contributions	2
1.3 Thesis Structure	4
2 Background	6
2.1 Invertible Matrices	6
2.1.1 Invertible Matrices	7
2.1.2 Properties	7
2.2 Generalized inverse	7
2.2.1 Definition	7
2.3 Moore-Penrose pseudoinverse	9
2.3.1 Pseudoinverse	9
2.3.2 Computation of the pseudoinverse	10
2.4 Projection and Orthogonal Projection Matrices	11
2.4.1 Projection matrices	11
2.4.2 Orthogonal projection matrix	12
2.5 Least-Squares and Minimum-Norm Solutions	13
2.5.1 The linear least square problem	13
2.5.2 Minimum-norm solution to a consistent linear system	14
2.5.3 Singular Value Decomposition	15

3	Theory of Conditioning and Numerical Stability	19
3.1	Introduction	19
3.2	Matrix Norms and Induced Norms	20
3.3	Singular Values and the Spectral Norm	21
3.4	Condition Number	22
3.4.1	Interpretation	23
3.5	Conditioning of Linear Systems	23
3.6	Conditioning of Least-Squares and Pseudoinverse Problems	24
3.7	Forward Error, Backward Error, and Residual	24
3.8	Algorithmic Stability	25
3.9	Orthogonal Transformations and Numerical Stability	26
3.10	Perturbation of Singular Values	26
3.11	Conditioning of the Hidden-Layer Matrix in ELM	27
3.12	Implications for SVD and Hyperpower Methods	27
3.13	Summary	28
4	Literature Review	29
4.1	Overview of Generalized Inverses	29
4.2	Moore–Penrose Inverse and Least-Squares Problems	30
4.3	SVD-Based Methods for Computing the Pseudoinverse	30
4.3.1	Singular Value Decomposition as a Standard Approach	30
4.3.2	Classical SVD Algorithms	31
4.3.3	Advantages and Limitations of SVD-Based Methods	31
4.4	Iterative and Hyperpower Methods for the Pseudoinverse	32
4.4.1	Early Iterative Approaches	32
4.4.2	Hyperpower and Higher-Order Iterations	32
4.4.3	Advantages and Challenges of Iterative Methods	33
4.5	Extreme Learning Machine in the Literature	33
4.5.1	Analytical Training of Single-Hidden-Layer Networks	33
4.5.2	Theoretical and Practical Development of ELM	34
4.6	Numerical Stability in Pseudoinverse-Based Learning	34
4.6.1	Why Stability Matters	34
4.6.2	SVD Versus Hyperpower Methods in the Stability Context	35
4.7	Research Gap and Position of This Thesis	35
5	Extreme Learning Machine and Pseudoinverse Formulation	37
5.1	Single Hidden Layer Feedforward Networks	37

5.2	Gradient-Based Training of SLFNs	38
5.3	Extreme Learning Machine	39
5.4	Output Weight Computation via the Moore–Penrose Inverse	40
5.5	Role and Structure of the Hidden-Layer Output Matrix	41
5.6	Ill-Conditioning and Motivation for Stability Analysis	42
5.7	Summary	43
6	Numerical Methods for Pseudoinverse Computation	44
6.1	Introduction	44
6.2	Pseudoinverse from Singular Value Decomposition	45
6.3	Direct Construction via the Gram Matrix	46
6.4	Golub–Reinsch Algorithm	47
6.4.1	Bidiagonalization	47
6.4.2	Diagonalization of the Bidiagonal Matrix	48
6.5	Divide-and-Conquer SVD	48
6.6	One-Sided Jacobi SVD	49
6.7	Iterative Methods for Inverse and Pseudoinverse Approximation	50
6.8	Newton–Schulz Iteration	50
6.9	Higher-Order Hyperpower Methods	51
6.9.1	Chebyshev-Type Cubic Iteration	52
6.9.2	Other Cubic Schemes	52
6.9.3	Fourth- and Fifth-Order Methods	52
6.10	Stopping Criteria and Numerical Safeguards	53
6.11	Comparison Between SVD-Based and Iterative Approaches	53
6.11.1	Advantages of SVD-Based Methods	54
6.11.2	Advantages of Iterative Methods	54
6.11.3	Limitations	54
6.12	Implications for Extreme Learning Machine	55
6.13	Summary	55
7	Stability Analysis of Pseudoinverse Computation	57
7.1	Introduction	57
7.2	Conditioning of the Pseudoinverse Problem	58
7.2.1	A Random Matrix Justification for the Width–Stability Bound	59
7.3	Stability of the SVD-Based Pseudoinverse	61
7.3.1	Backward Stability of the SVD	61
7.3.2	Sensitivity Through Singular Values	62

7.3.3	Implication for ELM	62
7.4	Stability of Hyperpower Iterative Methods	63
7.4.1	Basic Error Propagation	63
7.4.2	Dependence on Initialization and Spectrum	63
7.4.3	Finite-Precision Effects	64
7.5	Effect of Singular Values and Ill-Conditioning	65
7.5.1	Spectral Interpretation	65
7.5.2	Worst-Case Reconstruction Error	65
7.6	Stability of ELM Based on Pseudoinverse Solvers	66
7.6.1	Propagation of Solver Error to Output Weights	66
7.6.2	SVD-Based ELM	67
7.6.3	Hyperpower-Based ELM	67
7.7	Regularization as a Stabilization Strategy	68
7.8	Summary	69
8	Experimental Analysis	70
8.1	Overview	70
8.2	Experiment A: Synthetic Matrix Benchmark	71
8.2.1	Design	71
8.2.2	Metrics	72
8.2.3	Results	72
8.2.4	Runtime and Trace Behavior	78
8.3	Experiment B: ELM Dataset Benchmark	80
8.3.1	Design	81
8.3.2	Method-Level Results	81
8.3.3	Dataset Results	83
8.3.4	Conditioning and Hidden-Layer Spectra	83
8.4	Discussion	87
8.5	Chapter Summary	89
9	Conclusion and Future Work	90
9.1	Conclusion	90
9.2	Experimental Findings	91
9.3	Limitations	92
9.4	Future Work	92
9.5	Final Remarks	93

A	Algorithms	94
A.1	Golub–Reinsch SVD	94
A.2	Divide-and-Conquer SVD	96
A.3	One-Sided Jacobi SVD	97
A.4	Thresholded SVD Pseudoinverse	97
A.5	Hyperpower Pseudoinverse Iteration	97
B	Experiment A Residual Traces	101
B.1	Convergence Threshold $1e-10$	101
B.2	Convergence Threshold $1e-7$	106

List of Figures

8.1	Runtime comparison for Experiment A with threshold $1e-10$	79
8.2	Runtime comparison for Experiment A with threshold $1e-7$	80
8.3	Experiment B conditioning, weight norm, accuracy, and runtime	84
8.4	Experiment B method comparison across hidden widths	85
8.5	Experiment B smallest hidden singular value by width	85
8.6	Experiment B smallest hidden singular value by sample size and width	86
8.7	Experiment B sample-width relationship for smallest hidden singular value . .	86
8.8	Representative hidden-layer singular-value spectrum for MNIST.	87
8.9	Representative hidden-layer singular-value spectrum for Fashion-MNIST. . . .	88
8.10	Representative hidden-layer singular-value spectrum for ISOLET.	88
B.1	Residual traces for the well-conditioned spectrum under $\epsilon_{\text{conv}} = 10^{-10}$	102
B.2	Residual traces for the moderately ill-conditioned spectrum under $\epsilon_{\text{conv}} = 10^{-10}$. .	103
B.3	Residual traces for the severely ill-conditioned spectrum under $\epsilon_{\text{conv}} = 10^{-10}$. .	104
B.4	Residual traces for the rank-deficient spectrum under $\epsilon_{\text{conv}} = 10^{-10}$	105
B.5	Residual traces for clustered small singular values under $\epsilon_{\text{conv}} = 10^{-10}$	106
B.6	Residual traces for the well-conditioned spectrum under $\epsilon_{\text{conv}} = 10^{-7}$	107
B.7	Residual traces for the moderately ill-conditioned spectrum under $\epsilon_{\text{conv}} = 10^{-7}$. .	108
B.8	Residual traces for the severely ill-conditioned spectrum under $\epsilon_{\text{conv}} = 10^{-7}$. .	109
B.9	Residual traces for the rank-deficient spectrum under $\epsilon_{\text{conv}} = 10^{-7}$	110
B.10	Residual traces for clustered small singular values under $\epsilon_{\text{conv}} = 10^{-7}$	111

List of Tables

8.1	Experiment A median method summary for the well-conditioned spectrum. .	73
8.2	Experiment A median method summary for the moderately ill-conditioned spectrum.	74
8.3	Experiment A median method summary for the severely ill-conditioned spectrum.	75
8.4	Experiment A median method summary for the rank deficient spectrum. . . .	76
8.5	Experiment A median method summary for the clustered small spectrum. . .	77
8.6	Iterative-method convergence by spectrum. Each entry counts successful iterative runs.	77
8.7	Experiment B aggregate results for SVD and Adam methods. Accuracy and runtime are reported as medians across data sets, hidden widths, initializations, and activations.	82
8.8	Experiment B aggregate results for iterative hyperpower methods. Accuracy and runtime are reported as medians; all methods are unsuccessful under the implemented stopping policy.	82
8.9	Experiment B mean test accuracy by data set. Values are averaged over hidden widths, initializations, and activation functions.	83
A.1	Hyperpower update formulas used in the implementation.	100

List of Algorithms

A.1	Golub–Reinsch SVD	95
A.2	Divide-and-Conquer SVD	96
A.3	Bidiagonal Divide-and-Conquer SVD	96
A.4	One-Sided Jacobi SVD	98
A.5	Thresholded SVD Pseudoinverse	99
A.6	Safeguarded Hyperpower Pseudoinverse Iteration	99

List of Abbreviations and Notations

Abbreviations

Abbreviation	Meaning
AI	Artificial Intelligence
ELM	Extreme Learning Machine
F1	F1 score
GPU	Graphics Processing Unit
ISOLET	Isolated Letter Speech Recognition data set
MNIST	Modified National Institute of Standards and Technology data set
QR	QR factorization or QR iteration
SLFN	Single Hidden Layer Feedforward Network
SVD	Singular Value Decomposition

Mathematical Notations

Notation	Meaning
$\mathbf{A}$	Matrix (bold uppercase)
$\mathbf{a}$	Vector (bold lowercase)
a	Scalar (normal font)
$\mathbf{I}$	Identity matrix
$\mathbf{O}, \mathbf{0}$	Zero matrix
$\mathbf{A}^T$	Transpose of matrix $\mathbf{A}$
$\mathbf{A}^{-1}$	Inverse of matrix $\mathbf{A}$
$\mathbf{A}^+$	Pseudo-inverse of matrix $\mathbf{A}$
$\ \cdot\ $	Euclidean norm for vectors or spectral norm for matrices when no subscript is shown
$\ \cdot\ _2$	Euclidean norm for vectors or induced spectral norm for matrices
$\ \cdot\ _F$	Frobenius matrix norm
$\ \cdot\ _1, \ \cdot\ _\infty$	Vector norms or corresponding induced matrix norms
$\ \cdot\ _{\psi_2}$	Sub-Gaussian norm of a random variable
$\text{rank}(\mathbf{A})$	Rank of matrix $\mathbf{A}$
$\text{Col}(\mathbf{A})$	Column space of matrix $\mathbf{A}$
$\text{Row}(\mathbf{A})$	Row space of matrix $\mathbf{A}$
$\text{Null}(\mathbf{A})$	Null space of matrix $\mathbf{A}$
$\text{Range}(\mathbf{A})$	Range of matrix $\mathbf{A}$
$\text{diag}(\mathbf{a})$	Diagonal matrix from vector $\mathbf{a}$
$\mathbb{R}$	Set of real numbers
$\mathbb{R}^n$	n-dimensional Euclidean space
$\mathbb{R}^{m \times n}$	Space of $m \times n$ real matrices
∇E	Gradient of the training error/objective
$\mathbb{E}[\cdot]$	Expectation
$\mathbb{P}(\cdot)$	Probability of an event

Notation	Meaning
m, n	Matrix dimensions
N	Number of training samples
L	Number of hidden nodes or hidden width
r	Rank of a matrix
p	Minimum matrix dimension, $p = \min(m, n)$
H	Hidden-layer output matrix in ELM
T	Target matrix in ELM training
Y	Target or label matrix in the dataset benchmark
X	Approximate inverse, pseudoinverse, or solution matrix
B	Right-hand-side matrix or bidiagonal matrix, depending on context
E	Perturbation or error matrix
G	Generalized inverse or working matrix, depending on context
P, Q	Projection, reflector, or orthogonal transformation matrices
U, V	Left and right singular-vector matrices in the SVD
W	Collection of trainable parameters
b	Bias matrix or bias vector broadcast across samples
W_{out}	ELM output-weight matrix in the dataset benchmark
x_j	Input vector of the j -th sample
t_j	Target vector of the j -th sample
w_i	Input-weight vector of the i -th hidden neuron
b_i	Bias of the i -th hidden neuron
β	ELM output-weight matrix or vector
$g(\cdot)$	Activation function
η	Learning rate

Notation	Meaning
λ_i	Eigenvalue of a matrix
σ_i	Singular value of a matrix
$\sigma_{\min}(\mathbf{A})$	Smallest singular value of matrix $\mathbf{A}$
Σ	Diagonal matrix of singular values
Σ^+	Pseudo-inverse of the singular-value matrix
Σ_r	Compact diagonal matrix of positive singular values
$\mathbf{U}_r, \mathbf{V}_r$	Compact SVD singular-vector matrices
κ	Condition number
λ	Regularization parameter
$\hat{\cdot}$	Computed or approximate quantity
Δ	Perturbation or error term
$\varepsilon_{\text{conv}}$	Convergence threshold
τ	Numerical tolerance
$\rho_{\max}$	Failure threshold for residual monitoring
r_{ls}	Relative least-squares residual
r_1, r_2	Relative Penrose residual metrics

Introduction

Chapter Overview:

This chapter introduces the motivation for extending classical matrix inversion to singular and rectangular matrices through the theory of generalized inverses. It explains the importance of the Moore–Penrose inverse in least-squares problems and highlights its applications in optimization and machine learning. The chapter then formulates the main research problem of the thesis, namely the study of pseudoinverse computation and its impact on Extreme Learning Machine. Finally, it states the research objectives, contributions, and overall structure of the thesis.

1.1 Motivation and Background

Matrix inversion is one of the most fundamental operations in linear algebra and plays a central role in solving linear systems, optimization problems, and many computational models in science and engineering. However, the classical inverse is only defined for square and non-singular matrices. In practical applications, especially in modern data analysis and machine learning, one frequently encounters rectangular matrices or singular systems for which the standard inverse does not exist. Such situations naturally arise in overdetermined systems, underdetermined systems, least-squares problems, and high-dimensional learning tasks.

To overcome this limitation, the concept of the generalized inverse was introduced as an extension of the classical matrix inverse. Among the different types of generalized inverses, the Moore–Penrose inverse is the most widely used because of its existence for every matrix

and its strong connection to least-squares and minimum-norm solutions. The Moore–Penrose inverse provides a rigorous and unified framework for solving linear systems even when exact inversion is impossible, thereby serving as an important bridge between theoretical linear algebra and practical numerical computation.

Beyond its theoretical significance, the generalized inverse has broad applications in statistics, signal processing, control theory, optimization, and machine learning. In particular, it plays a crucial role in models where the training problem can be reduced to solving a linear system. One representative example is the Extreme Learning Machine (ELM), a single-hidden-layer feedforward neural network in which the hidden-layer parameters are randomly assigned and the output weights are computed analytically. In this framework, the training phase reduces to solving a least-squares problem of the form

$$\mathbf{H}\beta = \mathbf{T},$$

where $\mathbf{H}$ is the hidden-layer output matrix, $\mathbf{T}$ is the target matrix, and β is the output weight matrix. Since $\mathbf{H}$ is generally rectangular or rank-deficient, the Moore–Penrose inverse provides a natural tool for computing the minimum-norm least-squares solution.

From a computational perspective, the pseudoinverse can be obtained using different numerical methods. Among them, Singular Value Decomposition (SVD) is the classical and most reliable approach due to its robustness and numerical stability. On the other hand, iterative hyperpower methods, such as Newton–Schulz and higher-order variants, offer an alternative that may reduce computational cost in some settings. This creates an important trade-off between numerical stability, convergence behavior, and computational efficiency. The issue becomes particularly significant when the matrix involved is ill-conditioned, since small singular values may strongly amplify numerical errors and affect the stability of the final learning model.

Motivated by these theoretical and practical considerations, this thesis studies generalized inverses from both mathematical and computational viewpoints, and investigates their application to Extreme Learning Machine. Special attention is given to the comparison between SVD-based and hyperpower-based pseudoinverse computation, with emphasis on numerical stability and its impact on ELM performance.

1.2 Research Objectives and Contributions

The main objective of this thesis is to investigate the theory, computation, and application of generalized inverses, with a specific focus on the Moore–Penrose inverse and its role in machine learning. In particular, this thesis aims to answer the following questions:

How can the concept of matrix inversion be extended from nonsingular square matrices to rectangular and singular matrices?

Why is the Moore–Penrose inverse the appropriate tool for least-squares and minimum-norm solutions?

What are the main computational methods for evaluating the pseudoinverse, and what are their advantages and limitations?

How does the choice of pseudoinverse algorithm affect the stability and performance of Extreme Learning Machine?

How do ill-conditioning and singular values of the hidden-layer output matrix influence the numerical behavior of ELM?

To address these questions, this thesis makes the following contributions:

Theoretical formulation of generalized inverses. The thesis presents the definitions, existence properties, projection interpretations, and Moore–Penrose conditions for generalized inverses, and explains why the Moore–Penrose inverse gives the canonical least-squares and minimum-norm solution for rectangular or singular systems.

Pseudoinverse formulation of ELM training. The thesis connects the output-weight computation in Extreme Learning Machine to the linear system $\mathbf{H}\beta = \mathbf{T}$, showing how the hidden-layer matrix $\mathbf{H}$ determines both the learned weights and the numerical stability of the training process.

Comparison of SVD-based and hyperpower pseudoinverse methods. The thesis reviews and implements SVD-based solvers and several iterative hyperpower methods, including Newton–Schulz and higher-order variants, and analyzes their trade-offs in stability, convergence behavior, stopping criteria, and computational cost.

Spectral stability analysis for ELM hidden matrices. While classical random matrix theory provides smallest singular value estimates for sub-Gaussian matrices, their implications for the spectral conditioning and numerical stability of ELM hidden-layer matrices remain largely unexplored. This thesis interprets these spectral estimates in the context of pseudoinverse-based ELM training and establishes a width-dependent stability perspective for random hidden-layer representations.

Controlled synthetic benchmark for pseudoinverse solvers. The thesis evaluates the implemented solvers on square, tall, wide, rank-deficient, and ill-conditioned matrices with prescribed singular-value spectra, separating algorithmic behavior from dataset-specific effects.

Dataset benchmark for pseudoinverse-based ELM. The thesis evaluates the same solver families inside ELM classifiers on capped subsets of MNIST, Fashion-MNIST, and ISO-LET, and compares closed-form SVD-based ELM with Adam-trained output-layer baselines.

Empirical link between hidden-matrix spectra and solver reliability. The thesis reports condition-number, singular-value, runtime, accuracy, and convergence evidence showing that hyperpower methods can be effective on favorable synthetic spectra, but that SVD-based methods remain the most reliable option for the tested unregularized ELM configurations.

Overall, the thesis shows that the generalized inverse is not only a mathematically elegant extension of classical inversion, but also a practical computational tool for modern machine learning. The study further demonstrates that the stability of pseudoinverse-based ELM is fundamentally tied to the spectrum of the hidden-layer matrix and that SVD-based computation remains the most robust reference method when numerical stability is critical.

1.3 Thesis Structure

The remainder of this thesis is organized as follows.

Chapter 2 presents the mathematical background for the thesis. It reviews invertible matrices, generalized inverses, the Moore–Penrose pseudoinverse, projection matrices, orthogonal projections, and the relationship between the pseudoinverse and least-squares or minimum-norm solutions.

Chapter 3 develops the theory of conditioning and numerical stability. It reviews matrix norms, singular values, condition numbers, perturbation effects, forward and backward error, residuals, and the implications of conditioning for least-squares, pseudoinverse computation, and the hidden-layer matrix in ELM.

Chapter 4 reviews the related literature. It covers generalized inverses, Moore–Penrose inverse theory, SVD-based pseudoinverse methods, iterative and hyperpower methods, Extreme Learning Machine, and numerical stability in pseudoinverse-based learning.

Chapter 5 discusses Extreme Learning Machine and its pseudoinverse formulation. It introduces single-hidden-layer feedforward networks, contrasts gradient-based training with ELM, explains output-weight computation through the Moore–Penrose inverse, and examines the role and conditioning of the hidden-layer output matrix $\mathbf{H}$.

Chapter 6 describes numerical methods for pseudoinverse computation. It covers SVD-

based construction, Gram-matrix approaches, classical SVD algorithms, iterative inverse and pseudoinverse approximations, Newton–Schulz iteration, higher-order hyperpower methods, stopping criteria, safeguards, and their implications for ELM.

Chapter 7 analyzes the stability of pseudoinverse computation. It studies the conditioning of the pseudoinverse problem, the stability of SVD-based and hyperpower-based methods, the effect of singular values and ill-conditioning, the propagation of solver error in ELM, regularization as a stabilization strategy, and the width-dependent behavior of the smallest singular value of random hidden-layer matrices.

Chapter 8 reports the experimental analysis. It contains a controlled synthetic matrix benchmark for pseudoinverse solvers and an ELM dataset benchmark on capped subsets of MNIST, Fashion-MNIST, and ISOLET. The chapter compares SVD-based methods, hyperpower methods, and Adam-trained output-layer baselines, and relates the observed accuracy, runtime, convergence, condition numbers, and singular values to the stability analysis.

Chapter 9 concludes the thesis. It summarizes the theoretical and experimental findings, discusses the limitations of the current work, and outlines future directions such as multi-seed validation, regularized ELM experiments, improved stopping rules for hyperpower methods, and larger-scale implementations.

Appendix A provides the algorithmic forms referred to in the thesis, including SVD-based pseudoinverse construction and the iterative hyperpower solvers used in the experiments.

Appendix B provides representative residual-trace plots for Experiment A. These plots complement Chapter 8 by showing how iterative pseudoinverse residuals evolve on selected synthetic spectra and convergence thresholds.

Chapter 2

Background

Chapter Overview:

This chapter presents the mathematical foundations required for the rest of the thesis. It reviews invertible matrices, generalized inverses, projection matrices, orthogonal projections, and the Moore–Penrose pseudoinverse. The chapter also establishes the connection between the pseudoinverse and the solution of least-squares and minimum-norm problems. These results provide the formal linear algebra framework on which the later ELM formulation and numerical analysis are built. The exposition and notations used here are primarily based on [1], which provides a comprehensive treatment of the topic.

2.1 Invertible Matrices

In this section, we present the theoretical background necessary to understand the concept of the generalized inverse. This includes formal definitions, key properties, and fundamental theorems that establish its mathematical foundation and relevance in linear algebra.

2.1.1 Invertible Matrices

Definition 2.1 – Invertible Matrix

[1, p. 1] A square matrix $A \in \mathbb{R}^{n \times n}$ is **invertible** if there exists a matrix B such that:

$$AB = BA = I \quad (2.1)$$

B is called *inverse matrix* of A , denoted A^{-1} ,

2.1.2 Properties

Theorem 2.1 – Invertible matrix theorem

[1, p. 1] Let $A \in \mathbb{R}^{n \times n}$. The following statements are either all true or all false for any given matrix:

- ▶ A is invertible,
- ▶ A^T is invertible,
- ▶ The linear transformation mapping from x to Ax is invertible,
- ▶ A has full rank: $\text{rank}(A) = n$,
- ▶ A has nonzero determinant: $\det(A) \neq 0$,

Remark 2.1

For any invertible matrix $A \in \mathbb{R}^{n \times n}$ and any vector $b \in \mathbb{R}^n$, the linear system

$$Ax = b$$

has an unique solution $x^* = A^{-1}b$,

2.2 Generalized inverse

2.2.1 Definition

Definition 2.2

[1, p. 2] Let $A \in \mathbb{R}^{m \times n}$ be any matrix. We call $G \in \mathbb{R}^{n \times m}$ a generalized inverse if :

$$AGA = A \quad (2.2)$$

Remark 2.2

If $\mathbf{A}$ is square and invertible, then $\mathbf{G} = \mathbf{A}^{-1}$,

Remark 2.3

The generalized inverse always exists, but may not be unique.

Theorem 2.2

[1, p. 41] Let $\mathbf{A} = \begin{bmatrix} \mathbf{A}_{11} & \mathbf{A}_{12} \\ \mathbf{A}_{21} & \mathbf{A}_{22} \end{bmatrix} \in \mathbb{R}^{m \times n}$ be a matrix of rank r and $\mathbf{A}_{11} \in \mathbb{R}^{r \times r}$. If $\mathbf{A}_{11}$ is invertible, then $\mathbf{G} = \begin{bmatrix} \mathbf{A}_{11}^{-1} & \mathbf{O} \\ \mathbf{O} & \mathbf{O} \end{bmatrix} \in \mathbb{R}^{n \times m}$ is a generalized inverse of $\mathbf{A}$,

Proof: Let $\mathbf{A} = \begin{bmatrix} \mathbf{A}_{11} & \mathbf{A}_{12} \\ \mathbf{A}_{21} & \mathbf{A}_{22} \end{bmatrix}$. Because $\mathbf{A}_{11}$ is invertible, therefore its rows are linear independent, and likewise, the first r rows of $\mathbf{A}$ is linear independent. Because the total rank is r , the remaining rows of $\mathbf{A}$ must be a linear combination of the first r rows. Thus, there exists a matrix $\alpha \in \mathbb{R}^{(m-r) \times r}$ such that

$$\begin{bmatrix} \mathbf{A}_{21} & \mathbf{A}_{22} \end{bmatrix} = \alpha \begin{bmatrix} \mathbf{A}_{11} & \mathbf{A}_{12} \end{bmatrix}$$

This implied that $\mathbf{A}_{21} = \alpha \mathbf{A}_{11}$ and $\mathbf{A}_{22} = \alpha \mathbf{A}_{12}$

Now we calculate $\mathbf{AGA}$. We have

$$\begin{aligned} \mathbf{AGA} &= \begin{bmatrix} \mathbf{A}_{11} & \mathbf{A}_{12} \\ \mathbf{A}_{21} & \mathbf{A}_{22} \end{bmatrix} \begin{bmatrix} \mathbf{A}_{11}^{-1} & \mathbf{O} \\ \mathbf{O} & \mathbf{O} \end{bmatrix} \begin{bmatrix} \mathbf{A}_{11} & \mathbf{A}_{12} \\ \mathbf{A}_{21} & \mathbf{A}_{22} \end{bmatrix} \\ &= \begin{bmatrix} \mathbf{I} & \mathbf{O} \\ \mathbf{A}_{21}\mathbf{A}_{11}^{-1} & \mathbf{O} \end{bmatrix} \begin{bmatrix} \mathbf{A}_{11} & \mathbf{A}_{12} \\ \mathbf{A}_{21} & \mathbf{A}_{22} \end{bmatrix} \\ &= \begin{bmatrix} \mathbf{A}_{11} & \mathbf{A}_{12} \\ \mathbf{A}_{21}\mathbf{A}_{11}^{-1}\mathbf{A}_{11} & \mathbf{A}_{21}\mathbf{A}_{11}^{-1}\mathbf{A}_{12} \end{bmatrix} \\ &= \begin{bmatrix} \mathbf{A}_{11} & \mathbf{A}_{12} \\ \mathbf{A}_{21} & \alpha \mathbf{A}_{11}\mathbf{A}_{11}^{-1}\mathbf{A}_{12} \end{bmatrix} \\ &= \begin{bmatrix} \mathbf{A}_{11} & \mathbf{A}_{12} \\ \mathbf{A}_{21} & \alpha \mathbf{A}_{12} \end{bmatrix} \\ &= \begin{bmatrix} \mathbf{A}_{11} & \mathbf{A}_{12} \\ \mathbf{A}_{21} & \mathbf{A}_{22} \end{bmatrix} = \mathbf{A} \end{aligned}$$

This shows that $G = \begin{bmatrix} A_{11}^{-1} & \mathbf{O} \\ \mathbf{O} & \mathbf{O} \end{bmatrix} \in \mathbb{R}^{n \times m}$ is a generalized inverse of A , □

Remark 2.4

Any matrix $A \in \mathbb{R}^{m \times n}$ with rank r can be arranged through row and column permutations to have the above partitioned form with an invertible $r \times r$ submatrix in the top-left. This essentially establishes the existence of a generalized inverse for any matrix.

Theorem 2.3

[1, p. 2] Consider the linear system

$$Ax = b$$

Suppose $b \in \text{Col}(A)$ such that the system is consistent (at least one solution). Let G be a generalized inverse of A . Then $x^* = Gb$ is a particular solution to the system.

Proof: Multiplying both sides by AG gives that

$$AGb = AGAx = Ax = b$$

This shows that $x^* = Gb$ is a particular solution, □

2.3 Moore-Penrose pseudoinverse

2.3.1 Pseudoinverse

Definition 2.3 – Pseudoinverse

[1, p. 40] Let $A \in \mathbb{R}^{m \times n}$ be any matrix. We call $B \in \mathbb{R}^{n \times m}$ a pseudoinverse if:

1. $ABA = A$
2. $BAB = B$
3. $(AB)^T = AB$,
4. $(BA)^T = BA$,

Remark 2.5

From the above definition:

- If B satisfies 1. and 2., it is called a reflexive generalize inverse,
- For any $A \in \mathbb{R}^{m \times n}$, the pseudoinverse always exists and is unique, denoted A^+ ,
- Pseudoinverse is also called Moore-Penrose inverse.

► The symmetric form of the definition implies $\mathbf{B} = \mathbf{A}^+$ and $\mathbf{A} = \mathbf{B}^+$ and thus $\mathbf{A} = (\mathbf{A}^+)^+$,

2.3.2 Computation of the pseudoinverse

Theorem 2.4

[2, p. 82] Let $\mathbf{A} \in \mathbb{R}^{m \times n}$ be any tall matrix with full column rank ($\text{rank}(\mathbf{A}) = n \leq m$). Then the pseudoinverse of $\mathbf{A}$ is

$$\mathbf{A}^+ = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T$$

Proof: We verify this with the pseudoinverse conditions:

1. $\mathbf{A}\mathbf{A}^+\mathbf{A} = \mathbf{A}(\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{A} = \mathbf{A}(\mathbf{A}^T \mathbf{A})^{-1} (\mathbf{A}^T \mathbf{A}) = \mathbf{A}$,
2. $\mathbf{A}^+\mathbf{A}\mathbf{A}^+ = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{A} (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T$,
3. $\mathbf{A}\mathbf{A}^+ = \mathbf{A}(\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T$ is symmetric,
4. $\mathbf{A}^+\mathbf{A} = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{A} = \mathbf{I}$ is symmetric,

Therefore $\mathbf{A}^+ = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T$ is a pseudoinverse of $\mathbf{A}$, □

Remark 2.6

This theorem implies that for any tall matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ with full column rank, the following is an orthogonal projection matrix (onto $\text{Col}(\mathbf{A})$)

$$\mathbf{A}\mathbf{A}^+ = \mathbf{A}(\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T$$

Remark 2.7

Let $\mathbf{U} \in \mathbb{R}^{m \times n}$ be a tall matrix with orthonormal columns. Then it has full column rank, and:

$$\mathbf{U}^T \mathbf{U} = \mathbf{I}_n$$

It follows that $\mathbf{U}^+ = \mathbf{U}^T$ and $\mathbf{U}\mathbf{U}^+ = \mathbf{U}\mathbf{U}^T$,

Theorem 2.5

[1, p. 43] Let $\mathbf{A} \in \mathbb{R}^{m \times n}$ be a rectangular diagonal matrix. Then $\mathbf{A}^+$ is a matrix $\mathbf{B} \in \mathbb{R}^{n \times m}$ such that:

$$b_{ii} = \begin{cases} \frac{1}{a_{ii}} & \text{if } a_{ii} \neq 0 \\ 0 & \text{if } a_{ii} = 0 \end{cases}$$

To find the pseudoinverse of any arbitrary matrix, we use its Singular Value Decomposition and the formula of diagonal pseudoinverse

Theorem 2.6

[1, p. 207] Let $\mathbf{A} \in \mathbb{R}^{m \times n}$ be any matrix. Suppose its full SVD is $\mathbf{A} = \mathbf{U}\Sigma\mathbf{V}^T$. Then:

$$\mathbf{A}^+ = \mathbf{V}\Sigma^+\mathbf{U}^T$$

Proof: We verify this with the pseudoinverse conditions:

1. $\mathbf{A}\mathbf{A}^+\mathbf{A} = \mathbf{U}\Sigma\mathbf{V}^T \cdot \mathbf{V}\Sigma^+\mathbf{U}^T \cdot \mathbf{U}\Sigma\mathbf{V}^T = \mathbf{U}\Sigma\Sigma^+\Sigma\mathbf{V}^T = \mathbf{U}\Sigma\mathbf{V}^T = \mathbf{A}$,
2. $\mathbf{A}^+\mathbf{A}\mathbf{A}^+ = \mathbf{V}\Sigma^+\mathbf{U}^T \cdot \mathbf{U}\Sigma\mathbf{V}^T \cdot \mathbf{V}\Sigma^+\mathbf{U}^T = \mathbf{V}\Sigma^+\Sigma\Sigma^+\mathbf{U}^T = \mathbf{V}\Sigma^+\mathbf{U}^T = \mathbf{A}^+$,
3. $\mathbf{A}\mathbf{A}^+ = \mathbf{U}\Sigma\mathbf{V}^T \cdot \mathbf{V}\Sigma^+\mathbf{U}^T = \mathbf{U}\Sigma\Sigma^+\mathbf{U}^T$ is symmetric,
4. $\mathbf{A}^+\mathbf{A} = \mathbf{V}\Sigma^+\mathbf{U}^T \cdot \mathbf{U}\Sigma\mathbf{V}^T = \mathbf{V}\Sigma^+\Sigma\mathbf{V}^T$ is symmetric,

Therefore $\mathbf{A}^+ = \mathbf{V}\Sigma^+\mathbf{U}^T$ is a pseudoinverse of $\mathbf{A} = \mathbf{U}\Sigma\mathbf{V}^T$, □

2.4 Projection and Orthogonal Projection Matrices

2.4.1 Projection matrices

Definition 2.4

[1, p. 43] A square matrix $\mathbf{P}$ is called a projection matrix if $\mathbf{P}^2 = \mathbf{P}$,

Remark 2.8

Projection matrix must have determinant 0 or 1 because $\det(\mathbf{P}) = \det(\mathbf{P}^2) = \det(\mathbf{P})^2$,

Remark 2.9

Consider a projection matrix $\mathbf{P} \in \mathbb{R}^{n \times n}$

- A projection matrix projects any vector in $\mathbb{R}^n$ into its range (column space). Let $\mathbf{x} \in \mathbb{R}^n$, then:

$$\mathbf{P}\mathbf{x} = \sum x_i \mathbf{p}_i \in \text{Col}(\mathbf{P})$$

- A projection matrix keeps all point from its range (when applied to them) in their original place. Let $\mathbf{v} \in \text{Range}(\mathbf{P})$, then there exists $\mathbf{x} \in \mathbb{R}^n$ such that $\mathbf{P}\mathbf{x} = \mathbf{v}$. Then:

$$\mathbf{P}\mathbf{v} = \mathbf{P}\mathbf{P}\mathbf{x} = \mathbf{P}^2\mathbf{x} = \mathbf{P}\mathbf{x} = \mathbf{v}$$

Theorem 2.7

[1, p. 43] Let $\mathbf{A} \in \mathbb{R}^{m \times n}$ with a generalized matrix $\mathbf{G} \in \mathbb{R}^{n \times m}$. Then $\mathbf{AG} \in \mathbb{R}^{m \times m}$ is a projection matrix,

Proof: We have $(\mathbf{AG})^2 = (\mathbf{AG})(\mathbf{AG}) = (\mathbf{AGA})\mathbf{G} = \mathbf{AG}$, □

Remark 2.10

Similarly, $\mathbf{GA} \in \mathbb{R}^{n \times n}$ is a projection matrix,

Remark 2.11

$\mathbf{A}$ and $\mathbf{AG}$ must have the same column space,

Proof: For any $\mathbf{y} \in \text{Col}(\mathbf{AG})$ there exists some $\mathbf{x} \in \mathbb{R}^m$ such that $\mathbf{y} = (\mathbf{AG})\mathbf{x}$. It follows that $\mathbf{y} = \mathbf{A}(\mathbf{G}\mathbf{x}) \in \text{Col}(\mathbf{A})$. This shows that $\text{Col}(\mathbf{AG}) \subseteq \text{Col}(\mathbf{A})$. Subsequently, for any $\mathbf{y} \in \text{Col}(\mathbf{A})$ there exists some $\mathbf{x} \in \mathbb{R}^m$ such that $\mathbf{y} = \mathbf{A}\mathbf{x}$. We have $\mathbf{y} = \mathbf{A}\mathbf{x} = (\mathbf{AGA})\mathbf{x} = (\mathbf{AG})(\mathbf{A}\mathbf{x}) \in \text{Col}(\mathbf{AG})$. This shows that $\text{Col}(\mathbf{A}) \subseteq \text{Col}(\mathbf{AG})$. Therefore $\mathbf{AG}$ is a projection matrix onto the column space of $\mathbf{A}$. □

2.4.2 Orthogonal projection matrix

Definition 2.5

[2, p. 44], [1, p. 74] A square matrix $\mathbf{P}$ is called a orthogonal projection matrix if $\mathbf{P} = \mathbf{P}^T$ and $\mathbf{P}^2 = \mathbf{P}$,

Theorem 2.8

[1, p. 105], [2, p. 82] For any matrix $A \in \mathbb{R}^{m \times n}$, AA^+ is an orthogonal projection matrix (onto $\text{Col}(A)$),

Proof: We have A^+ is still a generalized inverse. Thus AA^+ is a projection matrix onto the column space of A . Since A^+ is a pseudoinverse of A , AA^+ must be symmetric. Therefore, by definition, AA^+ is an orthogonal projection matrix onto $\text{Col}(A)$, $\square$

Remark 2.12

Similarly, A^+A is an orthogonal projection matrix (onto $\text{Row}(A)$),

Remark 2.13

For any projection matrix P and vector x , we have:

$$x = Px + (I - P)x$$

If P is an orthogonal projection, then two components are orthogonal to each other:

$$Px^T(I - P)x = x^T P(I - P)x = x^T (P - P^2)x = 0$$

This implies that orthogonal projection produces orthogonal decompositions of vectors,

2.5 Least-Squares and Minimum-Norm Solutions

2.5.1 The linear least square problem

Consider a system of linear equations $Ax = b$ where $A \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^m$. In general, a solution may not exist or not be unique. In either case, we seek a least squares solution instead by solving the least square problem [2, p. 78]:

$$\min_{x \in \mathbb{R}^n} \|Ax - b\|^2$$

This problem always has a solution.

Theorem 2.9 – Minimum-norm least-squares solution

[1, p. 109] The minimum-norm solution of the linear least square problem is:

$$x^* = A^+b$$

Proof: Let the compact SVD of A be

$$A = U_r \Sigma_r V_r^T,$$

where $r = \text{rank}(A)$. Every vector $x \in \mathbb{R}^n$ can be decomposed as

$$x = V_r y + z, \quad z \in \text{Null}(A).$$

Then

$$Ax = U_r \Sigma_r y,$$

so the least-squares problem is minimized by choosing $\Sigma_r y$ equal to the orthogonal projection of b onto $\text{Col}(A) = \text{Col}(U_r)$. Hence

$$y = \Sigma_r^{-1} U_r^T b.$$

Therefore every least-squares solution has the form

$$x = V_r \Sigma_r^{-1} U_r^T b + z = A^+ b + z, \quad z \in \text{Null}(A).$$

Since $A^+ b \in \text{Row}(A)$ and $\text{Null}(A)$ is orthogonal to $\text{Row}(A)$,

$$\|x\|_2^2 = \|A^+ b\|_2^2 + \|z\|_2^2 \geq \|A^+ b\|_2^2.$$

Thus the minimum-norm least-squares solution is $x^* = A^+ b$. $\square$

Remark 2.14

If A has full column rank ($\text{rank}(A) = n \leq m$), then the solution is unique $x^* = A^+ b = (A^T A)^{-1} A^T b$,

2.5.2 Minimum-norm solution to a consistent linear system

For linear systems $Ax = b$ with non-unique solutions (such as under-determined systems), the pseudoinverse may be used to construct the solution with minimum Euclidean norm among all solutions.

Theorem 2.10

[1, p. 109] Let $A \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^m$. If the linear system $Ax = b$ has solutions, then $x^* = A^+ b$ is an exact solution and has the smallest possible norm for all solutions,

Proof: Since A^+ is a generalized inverse, $A^+ b$ must be a solution to $Ax = b$. By theorem 2.4.2 and Remark 2.4.2 we know that $A^+ A$ is an orthogonal projection matrix. From Remark 2.4.2 we know that using $A^+ A$, we can decompose a vector x into 2 orthogonal components:

The projection of $\mathbf{x}$ onto $\text{Row}(\mathbf{A})$ and the orthogonal complement of $\text{Row}(\mathbf{A})$. Now, for any solution $\mathbf{x} \in \mathbb{R}^n$, consider

$$\mathbf{x} = (\mathbf{A}^+ \mathbf{A})\mathbf{x} + (\mathbf{I} - \mathbf{A}^+ \mathbf{A})\mathbf{x} = \mathbf{A}^+ \mathbf{b} + (\mathbf{I} - \mathbf{A}^+ \mathbf{A})\mathbf{x}$$

Because the components are orthogonal, we can apply the Pythagorean theorem:

$$\|\mathbf{x}\|^2 = \|\mathbf{A}^+ \mathbf{b}\|^2 + \|(\mathbf{I} - \mathbf{A}^+ \mathbf{A})\mathbf{x}\|^2$$

Since norms are non-negative, we conclude:

$$\|\mathbf{x}\|^2 \geq \|\mathbf{A}^+ \mathbf{b}\|^2$$

This shows that $\|\mathbf{x}\| \geq \|\mathbf{A}^+ \mathbf{b}\|$ with equality if and only if $(\mathbf{I} - \mathbf{A}^+ \mathbf{A})\mathbf{x} = 0$ which implies $\mathbf{x} = \mathbf{A}^+ \mathbf{b} = \mathbf{x}^*$, $\square$

2.5.3 Singular Value Decomposition

As shown in theorem 2.3.2, the Moore–Penrose pseudoinverse can be computed once an SVD of the matrix is known. This subsection distinguishes two forms of the decomposition. The *full SVD* keeps complete orthonormal bases for the domain and codomain. The *compact SVD* keeps only the singular-vector pairs associated with non-zero singular values. Both forms describe the same linear map, but the compact form is usually shorter and is often the most convenient form for hand calculations.

Definition 2.6 – Singular Value

[2, p. 26], [1, p. 202] Let $\mathbf{A} \in \mathbb{R}^{m \times n}$ and let $p = \min(m, n)$. The singular values of $\mathbf{A}$ are the non-negative square roots of the eigenvalues of $\mathbf{A}^T \mathbf{A}$, arranged in non-increasing order:

$$\sigma_i = \sqrt{\lambda_i}.$$

If $r = \text{rank}(\mathbf{A})$, then exactly r singular values are positive:

$$\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0, \quad \sigma_{r+1} = \dots = \sigma_p = 0.$$

Theorem 2.11 – Existence of Singular Value Decomposition

[1, p. 206], [2, p. 29] Let $\mathbf{A} \in \mathbb{R}^{m \times n}$ with rank r , and let $p = \min(m, n)$. Then there exist orthogonal matrices $\mathbf{U} \in \mathbb{R}^{m \times m}$ and $\mathbf{V} \in \mathbb{R}^{n \times n}$, and a rectangular diagonal matrix $\Sigma \in \mathbb{R}^{m \times n}$, such that

$$\mathbf{A} = \mathbf{U} \Sigma \mathbf{V}^T$$

where the diagonal entries of Σ are the singular values $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0$, followed by zeros.

Proof: The matrix $\mathbf{A}^T \mathbf{A}$ is symmetric and positive semidefinite since

$$\mathbf{x}^T (\mathbf{A}^T \mathbf{A}) \mathbf{x} = \mathbf{x}^T \mathbf{A}^T \mathbf{A} \mathbf{x} = (\mathbf{A} \mathbf{x})^T \mathbf{A} \mathbf{x} \geq 0 \quad \forall \mathbf{x} \in \mathbb{R}^n$$

Therefore it has an orthonormal eigendecomposition

$$\mathbf{A}^T \mathbf{A} = \mathbf{V} \mathbf{\Lambda} \mathbf{V}^T = \sum_{i=1}^n \lambda_i \mathbf{v}_i \mathbf{v}_i^T,$$

with $\lambda_i \geq 0$. For every positive eigenvalue, define $\sigma_i = \sqrt{\lambda_i}$ and

$$\mathbf{u}_i = \frac{\mathbf{A} \mathbf{v}_i}{\sigma_i}, \quad i = 1, \dots, r.$$

These vectors are orthonormal because

$$\mathbf{u}_i^T \mathbf{u}_j = \frac{\mathbf{v}_i^T \mathbf{A}^T \mathbf{A} \mathbf{v}_j}{\sigma_i \sigma_j} = \frac{\sigma_j^2 \mathbf{v}_i^T \mathbf{v}_j}{\sigma_i \sigma_j} = \delta_{ij}.$$

Extend $\{\mathbf{u}_1, \dots, \mathbf{u}_r\}$ to an orthonormal basis of $\mathbb{R}^m$, and let these basis vectors be the columns of $\mathbf{U}$. For $i \leq r$, $\mathbf{A} \mathbf{v}_i = \sigma_i \mathbf{u}_i$. For $i > r$, $\lambda_i = 0$, so $\|\mathbf{A} \mathbf{v}_i\|^2 = \mathbf{v}_i^T \mathbf{A}^T \mathbf{A} \mathbf{v}_i = 0$, hence $\mathbf{A} \mathbf{v}_i = \mathbf{0}$. Thus $\mathbf{A} \mathbf{V} = \mathbf{U} \mathbf{\Sigma}$, and multiplying by $\mathbf{V}^T$ gives $\mathbf{A} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^T$. $\square$

Definition 2.7 – Full Singular Value Decomposition

[2, p. 28] The **full Singular Value Decomposition** of $\mathbf{A} \in \mathbb{R}^{m \times n}$ is the factorization

$$\mathbf{A} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^T$$

where

- $\mathbf{U} \in \mathbb{R}^{m \times m}$ is an orthogonal matrix. Its columns are called the left singular vectors of $\mathbf{A}$.
- $\mathbf{\Sigma} \in \mathbb{R}^{m \times n}$ is a rectangular diagonal matrix with non-negative real entries $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_p$ where $p = \min(m, n)$.
- $\mathbf{V} \in \mathbb{R}^{n \times n}$ is an orthogonal matrix. Its columns are called the right singular vectors of $\mathbf{A}$.

According to theorem 2.3.2, the pseudoinverse of $\mathbf{A}$ can be computed from the full SVD as

$$\mathbf{A}^+ = \mathbf{V} \mathbf{\Sigma}^+ \mathbf{U}^T$$

where $\mathbf{\Sigma}^+ \in \mathbb{R}^{n \times m}$ is obtained by replacing each non-zero diagonal entry σ_i of $\mathbf{\Sigma}$ with $1/\sigma_i$, transposing the rectangular diagonal shape, and leaving the zero entries equal to zero.

Definition 2.8 – Compact Singular Value Decomposition

[2, p. 27], [1, p. 207] Let $r = \text{rank}(\mathbf{A})$. The **compact SVD** of $\mathbf{A} \in \mathbb{R}^{m \times n}$ keeps only the r positive singular values:

$$\mathbf{A} = \mathbf{U}_r \Sigma_r \mathbf{V}_r^T.$$

Here $\mathbf{U}_r \in \mathbb{R}^{m \times r}$ has orthonormal columns, $\mathbf{V}_r \in \mathbb{R}^{n \times r}$ has orthonormal columns, and $\Sigma_r \in \mathbb{R}^{r \times r}$ is diagonal with positive diagonal entries $\sigma_1, \dots, \sigma_r$.

The compact SVD gives the same pseudoinverse without carrying the zero singular directions:

$$\mathbf{A}^+ = \mathbf{V}_r \Sigma_r^{-1} \mathbf{U}_r^T.$$

This formula is equivalent to the full-SVD formula because the missing singular directions correspond exactly to zero entries in Σ^+ .

Example: Given the input matrix $\mathbf{A}$:

$$\mathbf{A} = \begin{bmatrix} 1.0000 & 2.0000 \\ 3.0000 & 4.0000 \\ 5.0000 & 6.0000 \end{bmatrix}$$

Since $\mathbf{A} \in \mathbb{R}^{3 \times 2}$ has rank 2, it is convenient to use the compact SVD

$$\mathbf{A} = \mathbf{U}_r \Sigma_r \mathbf{V}_r^T,$$

where $\mathbf{U}_r \in \mathbb{R}^{3 \times 2}$, $\Sigma_r \in \mathbb{R}^{2 \times 2}$, and $\mathbf{V}_r \in \mathbb{R}^{2 \times 2}$. This is equivalent to the full SVD for the purpose of computing the pseudoinverse, because the zero singular directions do not contribute to Σ^+ . Numerically, one compact SVD is:

$$\begin{aligned} \mathbf{U}_r &= \begin{bmatrix} -0.2298 & 0.8835 \\ -0.5247 & 0.2408 \\ -0.8196 & -0.4019 \end{bmatrix} \\ \Sigma_r &= \begin{bmatrix} 9.5255 & 0.0000 \\ 0.0000 & 0.5143 \end{bmatrix} \\ \mathbf{V}_r^T &= \begin{bmatrix} -0.6196 & -0.7849 \\ -0.7849 & 0.6196 \end{bmatrix} \end{aligned}$$

The signs of corresponding columns of $\mathbf{U}_r$ and $\mathbf{V}_r$ are not unique; changing both signs in the same singular-vector pair gives the same matrix $\mathbf{A}$. The signs above are paired consistently.

To calculate Σ_r^{-1} , take the reciprocal of the non-zero diagonal entries:

$$\Sigma_r^{-1} = \begin{bmatrix} \frac{1}{9.5255} & 0 \\ 0 & \frac{1}{0.5143} \end{bmatrix} = \begin{bmatrix} 0.1050 & 0.0000 \\ 0.0000 & 1.9444 \end{bmatrix}$$

For the compact SVD, the pseudoinverse is

$$\mathbf{A}^+ = \mathbf{V}_r \Sigma_r^{-1} \mathbf{U}_r^T.$$

We need $\mathbf{V}_r$ and $\mathbf{U}_r^T$:

$$\mathbf{V}_r = (\mathbf{V}_r^T)^T = \begin{bmatrix} -0.6196 & -0.7849 \\ -0.7849 & 0.6196 \end{bmatrix}$$

$$\mathbf{U}_r^T = \begin{bmatrix} -0.2298 & -0.5247 & -0.8196 \\ 0.8835 & 0.2408 & -0.4019 \end{bmatrix}$$

Then scale the columns of $\mathbf{V}_r$ by the inverted singular values:

$$\mathbf{V}_r \Sigma_r^{-1} = \begin{bmatrix} -0.6196 & -0.7849 \\ -0.7849 & 0.6196 \end{bmatrix} \begin{bmatrix} 0.1050 & 0.0000 \\ 0.0000 & 1.9444 \end{bmatrix} = \begin{bmatrix} -0.0651 & -1.5262 \\ -0.0824 & 1.2048 \end{bmatrix}$$

Finally, multiply the result by $\mathbf{U}_r^T$:

$$\begin{aligned} \mathbf{A}^+ &= \begin{bmatrix} -0.0651 & -1.5262 \\ -0.0824 & 1.2048 \end{bmatrix} \begin{bmatrix} -0.2298 & -0.5247 & -0.8196 \\ 0.8835 & 0.2408 & -0.4019 \end{bmatrix} \\ &= \begin{bmatrix} -1.3333 & -0.3333 & 0.6667 \\ 1.0833 & 0.3333 & -0.4167 \end{bmatrix} \end{aligned}$$

Theory of Conditioning and Numerical Stability

Chapter Overview:

This chapter introduces the numerical analysis concepts required for studying the stability of pseudoinverse computation and its effect on Extreme Learning Machine. It presents matrix norms, singular values, spectral norms, and condition numbers as the main tools for measuring sensitivity to perturbation. The chapter also explains the difference between conditioning of a mathematical problem and stability of a numerical algorithm, and reviews the notions of forward error, backward error, and residual. The exposition and notations used here are primarily based on [2, 3], which provides a comprehensive treatment of the topic.

3.1 Introduction

The previous chapter established the exact linear algebraic framework of generalized inverses, the Moore–Penrose pseudoinverse, least-squares solutions, and the singular value decomposition. In exact arithmetic, these concepts provide a complete and elegant solution to matrix inversion problems involving rectangular, singular, or rank-deficient matrices. However, practical computation is performed in finite precision, and therefore the quality of a computed solution depends not only on the mathematical problem itself, but also on the numerical behavior of the algorithm used to solve it.

This distinction is especially important in the context of pseudoinverse computation. Although the Moore–Penrose inverse is uniquely defined for every matrix, its computation may be highly sensitive to perturbations when the matrix is ill-conditioned. Since the Extreme Learning Machine (ELM) determines its output weights through the formula

$$\beta = \mathbf{H}^+ \mathbf{T},$$

where $\mathbf{H}$ is the hidden-layer output matrix, instability in pseudoinverse computation directly affects the stability of the learned model.

The purpose of this chapter is to introduce the numerical concepts required for the later analysis of pseudoinverse algorithms and ELM stability. In particular, we present the theory of matrix norms, singular values, condition numbers, perturbation sensitivity, forward and backward error, and the basic notion of algorithmic stability. These concepts will form the theoretical basis for comparing SVD-based and hyperpower-based methods in the subsequent chapters.

3.2 Matrix Norms and Induced Norms

To analyze numerical stability, one first needs a quantitative way to measure the size of vectors, matrices, and errors.

Definition 3.1 – Norm

A norm on a real vector space is a function $\|\cdot\|$ that assigns a non-negative real number to each element and satisfies, for all elements x, y and scalars α ,

$$\|x\| \geq 0, \quad \|x\| = 0 \iff x = 0, \quad \|\alpha x\| = |\alpha| \|x\|, \quad \|x + y\| \leq \|x\| + \|y\|.$$

These properties apply whether the elements are vectors or matrices.

Definition 3.2

[2, p. 17] Let $\mathbf{x} = (x_1, \dots, x_n)^T \in \mathbb{R}^n$. The vector 1-norm, Euclidean norm, and infinity norm are respectively defined by

$$\|\mathbf{x}\|_1 = \sum_{i=1}^n |x_i|, \quad \|\mathbf{x}\|_2 = \left(\sum_{i=1}^n x_i^2 \right)^{1/2}, \quad \|\mathbf{x}\|_\infty = \max_{1 \leq i \leq n} |x_i|.$$

Definition 3.3

[2, p. 19] Let $\mathbf{A} \in \mathbb{R}^{m \times n}$. For $p \in \{1, 2, \infty\}$, the matrix norm induced by the corresponding vector p -norm is

$$\|\mathbf{A}\|_p = \max_{\mathbf{x} \neq 0} \frac{\|\mathbf{A}\mathbf{x}\|_p}{\|\mathbf{x}\|_p}.$$

In particular,

$$\|\mathbf{A}\|_1 = \max_{1 \leq j \leq n} \sum_{i=1}^m |a_{ij}|, \quad \|\mathbf{A}\|_\infty = \max_{1 \leq i \leq m} \sum_{j=1}^n |a_{ij}|,$$

and $\|\mathbf{A}\|_2$ is called the spectral norm.

Definition 3.4 – Frobenius norm

Let $\mathbf{A} = (a_{ij}) \in \mathbb{R}^{m \times n}$. The Frobenius norm is defined by

$$\|\mathbf{A}\|_F = \left(\sum_{i=1}^m \sum_{j=1}^n |a_{ij}|^2 \right)^{1/2} = \sqrt{\text{tr}(\mathbf{A}^T \mathbf{A})}.$$

The induced norm measures the maximum factor by which the matrix $\mathbf{A}$ can stretch a vector. It satisfies the important submultiplicative property

$$\|\mathbf{AB}\|_2 \leq \|\mathbf{A}\|_2 \|\mathbf{B}\|_2,$$

whenever the product $\mathbf{AB}$ is defined.

Unless a subscript is shown, $\|\cdot\|$ denotes the Euclidean norm for vectors and the spectral norm for matrices. A displayed subscript such as 1, 2, ∞ , or F always takes precedence over this convention.

In numerical analysis, the spectral norm is especially important because it has a direct relationship with singular values. This makes it particularly suitable for studying the stability of pseudoinverse computation.

3.3 Singular Values and the Spectral Norm

Let

$$\mathbf{A} \in \mathbb{R}^{m \times n}$$

have singular value decomposition

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T,$$

where $\mathbf{U}$ and $\mathbf{V}$ are orthogonal matrices and

$$\mathbf{\Sigma} = \text{diag}(\sigma_1, \sigma_2, \dots, \sigma_r)$$

contains the nonzero singular values of $\mathbf{A}$, ordered so that

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0.$$

The singular values provide a complete spectral description of the action of $\mathbf{A}$. In particular [2, p. 34],

$$\|\mathbf{A}\|_2 = \sigma_1.$$

If $\mathbf{A}$ has full rank, then the smallest singular value σ_r measures how far the matrix is from losing rank.

The Moore–Penrose inverse is

$$\mathbf{A}^+ = \mathbf{V}\Sigma^+\mathbf{U}^T,$$

where Σ^+ is obtained by replacing each nonzero singular value σ_i with its reciprocal $1/\sigma_i$. Therefore,

$$\|\mathbf{A}^+\|_2 = \frac{1}{\sigma_r}.$$

This identity is fundamental. It shows that the pseudoinverse becomes large when the smallest singular value is small. Hence the numerical difficulty of pseudoinverse computation is governed directly by the singular-value spectrum of the matrix.

3.4 Condition Number

The condition number is the central concept used to measure the sensitivity of a numerical problem to perturbation.

Definition 3.5

[2, p. 94] Let $\mathbf{A} \in \mathbb{R}^{n \times n}$ be nonsingular. The spectral condition number of $\mathbf{A}$ is defined by

$$\kappa_2(\mathbf{A}) = \|\mathbf{A}\|_2 \|\mathbf{A}^{-1}\|_2.$$

If $\mathbf{A}$ has singular values $\sigma_1 \geq \cdots \geq \sigma_n > 0$, then

$$\kappa_2(\mathbf{A}) = \frac{\sigma_1}{\sigma_n}.$$

A matrix with small condition number is called *well-conditioned*, while a matrix with very large condition number is called *ill-conditioned*. Intuitively, the condition number measures how strongly errors in the input may be amplified in the solution.

For rank-deficient matrices, the classical inverse condition number is infinite. When only the nonzero singular subspace is considered, the effective spectral condition number is

$$\frac{\sigma_1}{\sigma_r}$$

where σ_r is the smallest nonzero singular value. If σ_r is close to zero, then the problem becomes highly sensitive on that subspace.

3.4.1 Interpretation

If $\kappa_2(\mathbf{A})$ is close to 1, the matrix behaves like an orthogonal transformation and is numerically well behaved. If $\kappa_2(\mathbf{A})$ is very large, then the matrix maps some directions much more strongly than others, and inversion becomes unstable.

This observation is particularly important in ELM. Since the output weights are computed from the hidden-layer matrix $\mathbf{H}$, the condition number of $\mathbf{H}$ largely determines how stable the learned weights can be.

3.5 Conditioning of Linear Systems

Consider the linear system

$$\mathbf{A}\mathbf{x} = \mathbf{b},$$

where $\mathbf{A}$ is nonsingular. Suppose that $\mathbf{b}$ is perturbed to $\mathbf{b} + \Delta\mathbf{b}$, and let the corresponding solution be $\mathbf{x} + \Delta\mathbf{x}$. Then

$$\mathbf{A}(\mathbf{x} + \Delta\mathbf{x}) = \mathbf{b} + \Delta\mathbf{b},$$

which implies

$$\mathbf{A}\Delta\mathbf{x} = \Delta\mathbf{b}.$$

Hence

$$\Delta\mathbf{x} = \mathbf{A}^{-1}\Delta\mathbf{b},$$

and therefore

$$\|\Delta\mathbf{x}\|_2 \leq \|\mathbf{A}^{-1}\|_2 \|\Delta\mathbf{b}\|_2.$$

Since $\mathbf{b} = \mathbf{A}\mathbf{x}$, we also have

$$\|\mathbf{b}\|_2 \leq \|\mathbf{A}\|_2 \|\mathbf{x}\|_2,$$

so combining the two inequalities yields

$$\frac{\|\Delta\mathbf{x}\|_2}{\|\mathbf{x}\|_2} \leq \kappa_2(\mathbf{A}) \frac{\|\Delta\mathbf{b}\|_2}{\|\mathbf{b}\|_2}.$$

This classical bound [2, p. 94] shows that the relative error in the solution is magnified by a factor proportional to the condition number of the matrix. Thus, even if an algorithm were perfectly stable, an ill-conditioned linear system would still be intrinsically sensitive.

3.6 Conditioning of Least-Squares and Pseudoinverse Problems

The previous discussion concerns nonsingular square matrices. In this thesis, however, the main object of interest is the pseudoinverse, which applies to rectangular or singular matrices. The same principle remains true: small singular values cause instability.

Let

$$\mathbf{A} = \mathbf{U}_r \Sigma_r \mathbf{V}_r^T$$

be the compact SVD of $\mathbf{A}$, with

$$\Sigma_r = \text{diag}(\sigma_1, \dots, \sigma_r), \quad \sigma_1 \geq \dots \geq \sigma_r > 0.$$

Then

$$\mathbf{A}^+ = \mathbf{V}_r \Sigma_r^{-1} \mathbf{U}_r^T, \quad \|\mathbf{A}^+\|_2 = \frac{1}{\sigma_r}.$$

If σ_r is very small, then the pseudoinverse must contain a very large factor $1/\sigma_r$. This means that perturbations in the data or in floating-point arithmetic can produce large changes in the solution $\mathbf{A}^+ \mathbf{b}$.

For least-squares problems,

$$\min_x \|\mathbf{A}\mathbf{x} - \mathbf{b}\|_2,$$

the Moore–Penrose inverse gives the minimum-norm least-squares solution

$$\mathbf{x}^* = \mathbf{A}^+ \mathbf{b}.$$

Therefore the sensitivity of least-squares solutions is also governed by the spectral behavior of $\mathbf{A}$, especially by its smallest nonzero singular value.

3.7 Forward Error, Backward Error, and Residual

When analyzing numerical algorithms, it is important to distinguish different types of error. We use the standard terminology of forward error and backward error from Higham [3, Sec. 1.5].

Definition 3.6

Let $\mathbf{x}^*$ be the exact solution of a problem, and let $\hat{\mathbf{x}}$ be the computed solution. The *forward error* is the difference

$$\|\hat{\mathbf{x}} - \mathbf{x}^*\|.$$

Forward error measures how far the computed answer is from the true answer.

Definition 3.7

[2, p. 77] The *residual* of an approximate solution $\hat{\mathbf{x}}$ to the linear system $\mathbf{A}\mathbf{x} = \mathbf{b}$ is

$$\mathbf{r} = \mathbf{b} - \mathbf{A}\hat{\mathbf{x}}.$$

The residual measures how well the computed answer satisfies the original equations. However, a small residual does not necessarily imply a small forward error when the problem is ill-conditioned.

Definition 3.8

The *backward error* of a computed solution $\hat{\mathbf{x}}$ is the size of the smallest perturbation to the input data for which $\hat{\mathbf{x}}$ becomes the exact solution of the perturbed problem.

Backward error is often more meaningful than forward error because it separates two effects:

- the intrinsic conditioning of the problem,
- the numerical quality of the algorithm.

A small backward error means that the algorithm solves a nearby problem accurately. If the original problem is well-conditioned, then small backward error usually implies small forward error. If the original problem is ill-conditioned, even a backward-stable algorithm may still have noticeable forward error, but that error is then due to the problem, not to the algorithm.

3.8 Algorithmic Stability

The conditioning of a problem and the stability of an algorithm are not the same thing [3, Sec. 1.6].

- **Conditioning** describes how sensitive the exact mathematical problem is to perturbation.

- **Stability** describes how much additional error a numerical algorithm introduces while solving that problem.

An algorithm is called *backward stable* if its computed output is the exact solution to a nearby problem with only a small perturbation in the input data [3, Sec. 1.5]. This is usually the ideal notion of stability in numerical linear algebra.

A problem may be ill-conditioned even when solved by a stable algorithm. Conversely, an unstable algorithm may produce large errors even on a well-conditioned problem. Therefore, a complete numerical analysis must study both conditioning and stability.

3.9 Orthogonal Transformations and Numerical Stability

Orthogonal matrices play a special role in numerical linear algebra because they preserve the Euclidean norm. If $\mathbf{Q}$ is orthogonal, then

$$\mathbf{Q}^T \mathbf{Q} = \mathbf{I}, \quad \|\mathbf{Q}\mathbf{x}\|_2 = \|\mathbf{x}\|_2$$

for every vector $\mathbf{x}$. It follows that

$$\|\mathbf{Q}\|_2 = 1.$$

This norm-preserving property makes orthogonal transformations extremely stable from a numerical point of view [2]. Algorithms based on Householder reflections [4], Givens rotations [5], and QR-type iterations [6] therefore tend to have very favorable stability behavior.

This explains why SVD-based methods are usually more reliable than methods based on repeated explicit inverse approximations. The SVD is built from orthogonal operations, which do not amplify error unnecessarily.

3.10 Perturbation of Singular Values

The singular values of a matrix are themselves stable under perturbation. If $\mathbf{A}$ is perturbed to

$$\mathbf{A} + \mathbf{E},$$

then the singular values change by at most

$$|\tilde{\sigma}_i - \sigma_i| \leq \|\mathbf{E}\|_2.$$

This means that small perturbations in the matrix lead to proportionally small changes in the singular values.

However, the effect on the pseudoinverse may still be large when the smallest singular value is small. Indeed, because

$$\|\mathbf{A}^+\|_2 = \frac{1}{\sigma_r},$$

a small change in σ_r may produce a substantial change in $\mathbf{A}^+$ when σ_r is close to zero.

Thus, the singular values are numerically stable quantities, but the inverse operation applied to them is not. This is the central reason why ill-conditioned pseudoinverse problems are difficult.

3.11 Conditioning of the Hidden-Layer Matrix in ELM

In Extreme Learning Machine, the hidden-layer output matrix $\mathbf{H}$ plays the same role as the coefficient matrix in a least-squares problem. The output weights are computed by

$$\beta = \mathbf{H}^+ \mathbf{T}.$$

Therefore,

$$\|\beta\|_2 \leq \|\mathbf{H}^+\|_2 \|\mathbf{T}\|_2 = \frac{\|\mathbf{T}\|_2}{\sigma_r(\mathbf{H})}.$$

This shows that the smallest singular value of $\mathbf{H}$ directly controls the sensitivity of the ELM solution. If $\sigma_r(\mathbf{H})$ is very small, then the learned output weights may become very large and highly sensitive to perturbation.

Since $\mathbf{H}$ is generated from randomly assigned hidden weights and biases, its quality is not guaranteed a priori. Depending on the activation function, the number of hidden nodes, and the dataset, $\mathbf{H}$ may be well-conditioned, nearly rank-deficient, or strongly ill-conditioned. Consequently, ELM is numerically stable only to the extent that the hidden-layer matrix $\mathbf{H}$ and the pseudoinverse algorithm permit.

This observation motivates the later chapters of the thesis. Once ELM is recognized as a pseudoinverse-based learning method, the numerical study of $\mathbf{H}$ becomes essential.

3.12 Implications for SVD and Hyperpower Methods

The theory developed above already suggests a qualitative comparison between the two main families of pseudoinverse solvers studied in this thesis.

SVD-based methods are built from orthogonal transformations and compute singular values explicitly. As a result, they are typically backward stable and allow the smallest singular

directions to be identified directly. This makes them particularly suitable for ill-conditioned problems.

Hyperpower iterative methods, such as Newton–Schulz and its higher-order extensions, do not expose the singular values explicitly. Their convergence depends on initialization, spectral properties, and iteration control. When the matrix is well-conditioned, they may converge rapidly and efficiently. When the matrix is ill-conditioned, however, their convergence may slow down, stagnate, or become more sensitive to roundoff error.

Therefore, the numerical theory already indicates why SVD is generally the safer approach, while hyperpower methods require more careful control and may benefit from additional stabilization mechanisms such as scaling or regularization.

3.13 Summary

This chapter introduced the numerical concepts needed for the rest of the thesis. We reviewed matrix norms, singular values, spectral norms, condition numbers, forward and backward error, residuals, and the basic notion of numerical stability. The main conclusions are as follows.

First, the conditioning of inversion, least-squares, and pseudoinverse problems is governed primarily by the singular values of the matrix, especially the smallest nonzero singular value. Second, the pseudoinverse becomes sensitive when this singular value is small, because the factor $1/\sigma_r$ appears explicitly in A^+ . Third, algorithmic stability is distinct from problem conditioning: a stable algorithm solves a nearby problem accurately, but cannot eliminate the intrinsic sensitivity of an ill-conditioned problem. Finally, these concepts apply directly to Extreme Learning Machine, because the output weights are computed through

$$\beta = H^+T.$$

This chapter therefore provides the theoretical bridge from exact generalized inverse theory to the later study of numerical methods, stability analysis, and stabilized pseudoinverse-based ELM.

Literature Review

Chapter Overview:

This chapter surveys the main strands of literature relevant to the thesis. It reviews classical work on generalized inverses and pseudoinverse computation, including SVD-based methods and hyperpower iterative schemes. It also examines the development of Extreme Learning Machine as an analytically trained single-hidden-layer neural network and discusses prior studies on stability, conditioning, and regularization in ELM. The chapter concludes by identifying the research gap addressed in this thesis: the need for a unified analysis of pseudoinverse computation, numerical stability, and ELM performance.

4.1 Overview of Generalized Inverses

The problem of extending matrix inversion beyond nonsingular square matrices has been studied for more than a century. In many applications, the coefficient matrix is either rectangular or singular, so the classical inverse does not exist. This limitation motivated the development of the *generalized inverse*, which provides an algebraic tool for describing solutions of linear systems in a broader setting.

Among the many forms of generalized inverses, the Moore–Penrose inverse is the most important and most widely used. The Moore–Penrose inverse was introduced through the pioneering work of Moore [7], Bjerhammar [8], Penrose [9] and was later developed systematically in the classical monograph of Ben-Israel and Greville [1]. Unlike a general algebraic

inverse, the Moore–Penrose inverse is uniquely characterized by four defining equations, and it plays a central role in least-squares theory, minimum-norm solutions, and numerical linear algebra.

The importance of the Moore–Penrose inverse comes from both theory and application. From a theoretical perspective, it provides a unified solution concept for inconsistent, under-determined, and rank-deficient linear systems. From a practical perspective, it is widely used in statistics, optimization, control, signal processing, and machine learning. Because of this dual role, the generalized inverse has become a standard tool in modern applied mathematics.

4.2 Moore–Penrose Inverse and Least-Squares Problems

One of the main reasons the Moore–Penrose inverse is so important is its close connection to least-squares problems. For a general linear system

$$\mathbf{A}\mathbf{x} = \mathbf{b}, \quad \mathbf{A} \in \mathbb{R}^{m \times n}, \mathbf{b} \in \mathbb{R}^m,$$

an exact solution may not exist, or may not be unique. In such cases, one seeks a vector $\mathbf{x}$ that minimizes the residual norm

$$\min_{\mathbf{x} \in \mathbb{R}^n} \|\mathbf{A}\mathbf{x} - \mathbf{b}\|_2.$$

The Moore–Penrose inverse provides a canonical solution to this problem. In particular, $\mathbf{A}^+\mathbf{b}$ gives the minimum-norm least-squares solution, and when the system is consistent, it gives the exact solution of smallest Euclidean norm [1].

This property makes the pseudoinverse especially attractive in computational settings. Instead of treating overdetermined and underdetermined systems separately, the same operator $\mathbf{A}^+$ can be used in both cases. As a result, a large body of work in numerical linear algebra focuses on how to compute $\mathbf{A}^+$ efficiently and accurately.

4.3 SVD-Based Methods for Computing the Pseudoinverse

4.3.1 Singular Value Decomposition as a Standard Approach

The most classical and reliable approach for computing the Moore–Penrose inverse is through the Singular Value Decomposition (SVD). If

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$$

is the singular value decomposition of $\mathbf{A}$, then the pseudoinverse is given by

$$\mathbf{A}^+ = \mathbf{V}\mathbf{\Sigma}^+\mathbf{U}^T.$$

This formula is mathematically elegant and computationally powerful, since it applies to arbitrary matrices, including rectangular and rank-deficient ones [1].

The SVD is particularly important because it reveals the spectral structure of the matrix. The singular values indicate the rank, conditioning, and numerical stability of the linear system. Small singular values correspond to directions in which the problem becomes sensitive to perturbations. Therefore, the SVD is not only a computational method for obtaining $\mathbf{A}^+$, but also a diagnostic tool for understanding the difficulty of the inverse problem itself.

4.3.2 Classical SVD Algorithms

A straightforward way to derive singular values is through the eigendecomposition of $\mathbf{A}^T\mathbf{A}$. However, this approach is often numerically unsatisfactory because forming $\mathbf{A}^T\mathbf{A}$ squares the condition number and may severely degrade the accuracy of small singular values [10]. For this reason, more sophisticated SVD algorithms were developed.

A major breakthrough was the Golub-Kahan and Golub-Reinsch framework [10, 11], which avoids explicit formation of $\mathbf{A}^T\mathbf{A}$. The Golub-Reinsch algorithm first reduces a dense matrix to bidiagonal form using Householder reflections, and then applies an implicit QR iteration to compute the singular values. This method became the foundation of many standard numerical libraries and is regarded as one of the most robust general-purpose SVD algorithms.

To improve efficiency on large dense matrices, divide-and-conquer methods were later proposed, notably by Gu and Eisenstat [12]. These methods recursively decompose the problem and solve smaller secular equations, often achieving better performance than classical bidiagonal QR on large inputs. Another important class of methods is the one-sided Jacobi algorithm, originally studied by Hestenes [13] and further improved by de Rijk [14]. Although Jacobi-type methods are usually more computationally expensive, they are known for excellent orthogonality and high numerical accuracy.

4.3.3 Advantages and Limitations of SVD-Based Methods

SVD-based methods are generally regarded as the gold standard for pseudoinverse computation. Their main advantage is numerical robustness: they can handle rectangular, rank-deficient, and ill-conditioned matrices in a stable way. In particular, they provide direct access to the singular values, which allows very small values to be detected, discarded, or regularized when necessary.

However, this accuracy comes at a computational cost. Full SVD computation is expensive for large matrices, especially in repeated or real-time applications. This computational

burden has motivated the search for faster alternatives, particularly iterative schemes that approximate the inverse or pseudoinverse using only matrix multiplications.

4.4 Iterative and Hyperpower Methods for the Pseudoinverse

4.4.1 Early Iterative Approaches

Iterative methods for inverse computation have a long history. Hestenes [13] and Altman [15] studied iterative inversion schemes in Hilbert spaces, while Ben-Israel [16, 17] established that related techniques could also be used to compute generalized inverses of matrices. These methods are attractive because they replace direct factorization by repeated matrix operations, which may be advantageous on modern hardware and for certain matrix structures.

One of the most well-known examples is the Newton–Schulz method,

$$\mathbf{X}_{k+1} = \mathbf{X}_k(2\mathbf{I} - \mathbf{A}\mathbf{X}_k),$$

which is a quadratically convergent scheme when the initial approximation is sufficiently close to the desired inverse. The Newton–Schulz iteration has served as the basis for a large family of higher-order methods.

4.4.2 Hyperpower and Higher-Order Iterations

A general hyperpower iteration has the form

$$\mathbf{X}_{k+1} = \mathbf{X}_k \left(\mathbf{I} + R_k + R_k^2 + \cdots + R_k^p \right), \quad R_k = \mathbf{I} - \mathbf{A}\mathbf{X}_k,$$

With this notation, $p = 1$ gives the Newton–Schulz method and the local error order is $p + 1$. Higher-order schemes seek to improve convergence speed by using higher-degree matrix polynomials.

Several authors have proposed third-, fourth-, and fifth-order variants. Chebyshev-type methods have been developed for accelerating convergence [18], while other higher-order formulas have been proposed by Esmaeili et al. [19], Kaur et al. [20], and Sheikhi and Esmaeili [21]. A common idea in these works is to increase the local order of convergence while factorizing the polynomial expressions in order to reduce the number of matrix–matrix multiplications per iteration.

4.4.3 Advantages and Challenges of Iterative Methods

The main attraction of hyperpower methods is computational efficiency. Since they are built almost entirely from matrix multiplications and additions, they can be well suited for parallel architectures and batched computation. In favorable settings, they may perform competitively with or even faster than SVD-based methods.

However, the literature also shows that iterative methods are more sensitive to initialization, stopping criteria, and matrix conditioning. Their convergence depends on spectral properties of the matrix and on whether the initial approximation lies in the appropriate convergence region. As the matrix becomes ill-conditioned, convergence may slow down significantly, and roundoff errors may accumulate more strongly. Thus, while hyperpower methods are attractive from a computational perspective, their numerical stability is generally less robust than that of SVD-based methods.

4.5 Extreme Learning Machine in the Literature

4.5.1 Analytical Training of Single-Hidden-Layer Networks

Traditional training of single-hidden-layer feedforward neural networks relies on gradient-based optimization. Although backpropagation-based methods can achieve high accuracy, they are often computationally expensive, require tuning of learning rates and stopping criteria, and may suffer from slow convergence or poor local minima.

A different paradigm was proposed by Huang, Zhu, and Siew [22] under the name *Extreme Learning Machine* (ELM). In ELM, the input weights and hidden-node biases are randomly assigned and then fixed. Only the output weights are learned. Once the hidden-layer output matrix $\mathbf{H}$ is formed, training reduces to solving the linear system

$$\mathbf{H}\beta = \mathbf{T},$$

where $\mathbf{T}$ is the target matrix and β is the output weight matrix. The output weights are then computed analytically by

$$\beta = \mathbf{H}^+ \mathbf{T}.$$

This formulation makes ELM fundamentally different from conventional gradient-based training. The learning process becomes a problem in numerical linear algebra rather than nonlinear optimization. As a result, training is often much faster than backpropagation.

4.5.2 Theoretical and Practical Development of ELM

Since its introduction, ELM has been studied from both theoretical and practical viewpoints. Huang et al. [22] argued that random hidden-node assignment, together with analytical solution of the output layer, can yield good generalization performance while drastically reducing training time. Later works extended the ELM framework to online learning, stochastic variants, and different application domains.

At the same time, the theoretical foundations of ELM have continued to be discussed. Recent work by Perfilieva et al. [23] provides a critical re-examination of the theoretical assumptions often used in the ELM literature. This line of research suggests that, while ELM is practically effective, some classical theoretical claims should be interpreted carefully and under precise assumptions.

Stochastic variants of ELM have also been proposed. For example, Janakiraman et al. [24] studied stochastic gradient-based ELM formulations for online learning. Such approaches aim to combine some of the efficiency of ELM with the adaptability of iterative learning methods, although they may sacrifice part of the simplicity of the original analytical framework.

4.6 Numerical Stability in Pseudoinverse-Based Learning

4.6.1 Why Stability Matters

Because ELM computes its output weights through

$$\beta = \mathbf{H}^+ \mathbf{T},$$

the success of the method depends strongly on the numerical quality of the computed pseudoinverse. If the hidden-layer output matrix $\mathbf{H}$ is ill-conditioned or nearly rank-deficient, small perturbations in the data or in floating-point arithmetic can produce large changes in β . Therefore, the stability of the pseudoinverse algorithm directly affects the stability of the trained model [25].

This issue is not unique to ELM. In numerical linear algebra, the Moore–Penrose inverse is known to be sensitive to small singular values, because the inverse operation amplifies directions associated with them. In ELM, such amplification may appear as unstable output weights, reduced robustness, or degradation in generalization when the hidden representation matrix $\mathbf{H}$ is poorly conditioned.

4.6.2 SVD Versus Hyperpower Methods in the Stability Context

The literature suggests a natural contrast between SVD-based and hyperpower-based pseudoinverse computation. SVD-based methods are more computationally expensive, but they provide direct access to singular values and are widely regarded as the most reliable choice for rank-deficient and ill-conditioned problems. Hyperpower methods, in contrast, may achieve attractive performance on well-conditioned problems, but they rely on convergence behavior that is more sensitive to the spectrum of the matrix and to the quality of the initial approximation.

This contrast is particularly important for ELM, because the matrix $\mathbf{H}$ is generated from random hidden parameters and may vary greatly in quality depending on the activation function, number of hidden units, and dataset. As a consequence, the numerical method used to compute $\mathbf{H}^+$ should not be regarded as a minor implementation detail; it is a core component that influences both training stability and predictive performance.

4.7 Research Gap and Position of This Thesis

The existing literature provides strong foundations in three separate directions. First, classical numerical linear algebra offers a mature theory of generalized inverses and SVD-based computation [1, 10–13]. Second, iterative hyperpower methods have been actively developed to improve computational efficiency in generalized inverse computation [15–21]. Third, the ELM literature has shown that analytical training based on the Moore–Penrose inverse can provide an efficient alternative to gradient-based neural network training [22–24].

However, these three lines of work are often studied separately. The literature does not always provide a sufficiently detailed comparison of how different pseudoinverse algorithms behave when used specifically inside the ELM framework, especially under ill-conditioned hidden-layer matrices. In particular, there remains a need for a unified study that connects:

- ▶ the mathematical role of the Moore–Penrose inverse,
- ▶ the computational trade-off between SVD and hyperpower methods,
- ▶ and the stability of ELM when these methods are used in practice.

This thesis is positioned to address that gap. It does not only review generalized inverses as an abstract linear algebra topic, nor does it only implement ELM as a machine learning model. Instead, it studies the interaction between pseudoinverse computation and ELM training, with special emphasis on numerical stability, matrix conditioning, and the practical

trade-off between speed and accuracy.

In particular, this thesis compares SVD-based and hyperpower-based methods for computing the pseudoinverse, evaluates their performance on matrices of different structures, and investigates how these computational choices affect the behavior of ELM. By doing so, the thesis aims to clarify not only which method is faster, but also which method is more reliable in the presence of rank deficiency and ill-conditioning.

Extreme Learning Machine and Pseudoinverse Formulation

Chapter Overview:

This chapter develops the connection between generalized inverses and Extreme Learning Machine. It first reviews the structure of single-hidden-layer feedforward neural networks and the limitations of gradient-based training. It then presents ELM as a non-iterative learning framework in which the hidden-layer parameters are fixed randomly and the output weights are computed analytically. The chapter shows that ELM training reduces to solving the linear system $\mathbf{H}\beta = \mathbf{T}$, with the canonical solution $\beta = \mathbf{H}^+\mathbf{T}$, thereby establishing the pseudoinverse as the core mathematical mechanism of the model.

5.1 Single Hidden Layer Feedforward Networks

Single Hidden Layer Feedforward Networks (SLFNs) are among the most fundamental architectures in neural network theory. An SLFN consists of an input layer, a single hidden layer, and an output layer. Let the training set be

$$\{(\mathbf{x}_j, \mathbf{t}_j)\}_{j=1}^N,$$

where $\mathbf{x}_j \in \mathbb{R}^n$ is the input vector of the j -th sample and $\mathbf{t}_j \in \mathbb{R}^m$ is the corresponding target vector. For an SLFN with L hidden neurons, the network output for the j -th input sample is

given by

$$f_L(\mathbf{x}_j) = \sum_{i=1}^L \beta_i g(\mathbf{w}_i^T \mathbf{x}_j + b_i), \quad j = 1, \dots, N,$$

where $\mathbf{w}_i \in \mathbb{R}^n$ is the input weight vector connecting the input layer to the i -th hidden neuron, $b_i \in \mathbb{R}$ is the bias of the i -th hidden neuron, $g(\cdot)$ is the activation function, and $\beta_i \in \mathbb{R}^m$ is the output weight vector connecting the i -th hidden neuron to the output layer.

By collecting the hidden-layer outputs for all training samples, the system can be written compactly in matrix form as

$$\mathbf{H}\beta = \mathbf{T},$$

where

$$\mathbf{H} = \begin{bmatrix} g(\mathbf{w}_1^T \mathbf{x}_1 + b_1) & \cdots & g(\mathbf{w}_L^T \mathbf{x}_1 + b_L) \\ g(\mathbf{w}_1^T \mathbf{x}_2 + b_1) & \cdots & g(\mathbf{w}_L^T \mathbf{x}_2 + b_L) \\ \vdots & \ddots & \vdots \\ g(\mathbf{w}_1^T \mathbf{x}_N + b_1) & \cdots & g(\mathbf{w}_L^T \mathbf{x}_N + b_L) \end{bmatrix} \in \mathbb{R}^{N \times L}$$

is the hidden-layer output matrix,

$$\beta = \begin{bmatrix} \beta_1^T \\ \beta_2^T \\ \vdots \\ \beta_L^T \end{bmatrix} \in \mathbb{R}^{L \times m}, \quad \mathbf{T} = \begin{bmatrix} \mathbf{t}_1^T \\ \mathbf{t}_2^T \\ \vdots \\ \mathbf{t}_N^T \end{bmatrix} \in \mathbb{R}^{N \times m}.$$

This matrix formulation is crucial, because once the hidden-layer parameters $\{\mathbf{w}_i, b_i\}_{i=1}^L$ are fixed, the training of the output layer becomes a linear algebra problem. In particular, learning the network reduces to solving the matrix equation $\mathbf{H}\beta = \mathbf{T}$, which immediately connects SLFN training to the theory of least-squares and generalized inverses developed in the previous chapter.

5.2 Gradient-Based Training of SLFNs

Traditionally, SLFNs are trained by optimizing all parameters of the network, including the input weights, hidden biases, and output weights. This leads to the optimization problem

$$\min_{\mathbf{w}, b, \beta} \|\mathbf{H}\beta - \mathbf{T}\|_F,$$

where $\mathbf{H}$ itself depends nonlinearly on the hidden-layer parameters through

$$\mathbf{H}_{ji} = g(\mathbf{w}_i^T \mathbf{x}_j + b_i).$$

Equivalently, one may define the training error by the objective function

$$E = \sum_{j=1}^N \left\| \sum_{i=1}^L \beta_i g(\mathbf{w}_i^T \mathbf{x}_j + b_i) - \mathbf{t}_j \right\|_2^2.$$

Gradient-based methods, such as backpropagation or adaptive first-order optimizers, update the network parameters iteratively according to a rule of the form

$$\mathbf{W}^{(k+1)} = \mathbf{W}^{(k)} - \eta \nabla E(\mathbf{W}^{(k)}),$$

where $\mathbf{W}$ denotes the collection of trainable parameters and $\eta > 0$ is the learning rate.

Although this approach is standard in neural network training, it has several well-known limitations. First, the quality of convergence depends strongly on hyperparameter choices such as learning rate, initialization, batch size, and stopping criteria. Second, the optimization process may be computationally expensive because it requires repeated forward and backward passes through the training data. Third, the training dynamics may be unstable when the learning rate is too large, while excessively small learning rates result in very slow convergence. Finally, iterative training may lead to overfitting if the optimization proceeds for too many epochs without appropriate regularization or validation.

These difficulties motivate alternative training paradigms in which the nonlinear hidden representation is constructed first, and the output layer is then solved analytically. This is precisely the idea behind the Extreme Learning Machine.

5.3 Extreme Learning Machine

Extreme Learning Machine (ELM) was introduced as a fast learning framework for single-hidden-layer feedforward networks. The key idea is to avoid iterative optimization of all network parameters. Instead, the input weights and hidden-node biases are randomly assigned and then kept fixed throughout training. Once these hidden-layer parameters have been chosen, the hidden-layer output matrix $\mathbf{H}$ is determined, and the learning problem reduces to solving the linear system

$$\mathbf{H}\beta = \mathbf{T}.$$

Thus, in ELM, only the output weights β are learned. This is a major departure from conventional gradient-based neural network training. The nonlinear feature mapping induced by the hidden layer is created randomly, and the final learning stage becomes a purely linear problem. The practical consequence is a substantial reduction in training time.

Under suitable assumptions on the activation function and the sampling process of hidden parameters, classical ELM theory states that SLFNs with randomly assigned hidden nodes can approximate training data well, and in some ideal settings can interpolate the data exactly. These results provide the theoretical foundation for replacing iterative training by analytical output-weight computation. However, from a numerical viewpoint, the matrix $\mathbf{H}$ generated in this way is not guaranteed to be well-conditioned. Therefore, although ELM simplifies training conceptually, it shifts the core computational challenge to the computation of the output weights from the hidden-layer matrix.

The standard ELM training procedure can be summarized as follows:

1. Randomly assign the input weights and hidden-layer biases.
2. Compute the hidden-layer output matrix $\mathbf{H}$.
3. Determine the output weights β by solving $\mathbf{H}\beta = \mathbf{T}$.

The third step is where the generalized inverse becomes essential.

5.4 Output Weight Computation via the Moore–Penrose Inverse

In general, the hidden-layer output matrix $\mathbf{H}$ is neither square nor invertible in the classical sense. Depending on the number of training samples N and hidden nodes L , the system $\mathbf{H}\beta = \mathbf{T}$ may be overdetermined, underdetermined, or rank-deficient. Therefore, the output-weight computation must be formulated as a generalized linear system rather than as a standard matrix inversion problem.

From the theory developed in Chapter 2, the Moore–Penrose inverse provides the natural solution to this problem. The ELM output weights are computed as

$$\beta = \mathbf{H}^+\mathbf{T}.$$

This formula has a precise interpretation:

- if the system $\mathbf{H}\beta = \mathbf{T}$ is consistent, then $\mathbf{H}^+\mathbf{T}$ is an exact solution of minimum Euclidean norm,
- if the system is inconsistent, then $\mathbf{H}^+\mathbf{T}$ is the minimum-norm least-squares solution,

Hence the ELM solution inherits both the least-squares optimality and the minimum-norm property of the Moore–Penrose inverse. This is particularly valuable in machine learning, because the minimum-norm solution is often associated with better numerical behavior

and may contribute to improved generalization compared with arbitrary solutions of the same linear system.

When $\mathbf{H}$ has full column rank and $N \geq L$, one may write

$$\mathbf{H}^+ = (\mathbf{H}^T \mathbf{H})^{-1} \mathbf{H}^T,$$

and thus

$$\beta = (\mathbf{H}^T \mathbf{H})^{-1} \mathbf{H}^T \mathbf{T}.$$

However, this representation is valid only under the full-column-rank assumption. For arbitrary matrices, including rectangular and rank-deficient cases, the general formula

$$\mathbf{H}^+ = \mathbf{V} \Sigma^+ \mathbf{U}^T$$

obtained from the singular value decomposition

$$\mathbf{H} = \mathbf{U} \Sigma \mathbf{V}^T$$

is the appropriate expression.

Therefore, the pseudoinverse is not merely a computational convenience in ELM. It is the central mathematical mechanism that turns random-feature construction into a complete analytical learning algorithm.

5.5 Role and Structure of the Hidden-Layer Output Matrix

The hidden-layer output matrix $\mathbf{H}$ is the core object that determines the behavior of ELM. Its size is $N \times L$, where N is the number of training samples and L is the number of hidden nodes. Each row of $\mathbf{H}$ represents the hidden-layer response to one training sample, while each column represents the activation pattern of one hidden neuron over the entire training set.

The geometry and numerical properties of $\mathbf{H}$ govern both the solvability and the stability of the ELM training problem. Several cases are particularly important:

- **Square case** ($N = L$): if $\mathbf{H}$ has full rank, then the system may be solved exactly and $\mathbf{H}$ is invertible in the classical sense,
- **Overdetermined case** ($N > L$): the number of training equations exceeds the number of hidden nodes, and the output weights are typically obtained as a least-squares solution,

- **Underdetermined case** ($N < L$): the system has infinitely many exact solutions when consistent, and the pseudoinverse selects the solution with minimum Euclidean norm,
- **Rank-deficient case**: some hidden nodes may generate linearly dependent or nearly dependent columns, causing $\mathbf{H}$ to lose rank or become nearly singular,

Because the hidden parameters are chosen randomly, the matrix $\mathbf{H}$ depends on the activation function, the input distribution, the hidden-layer width, and the random seed. Consequently, even when the theoretical approximation power of ELM is strong, the actual numerical quality of $\mathbf{H}$ may vary substantially from one configuration to another.

This observation shows that ELM is fundamentally a *matrix-driven* learning method: its effectiveness is determined not only by network architecture, but also by the spectral properties of the hidden-layer output matrix.

5.6 Ill-Conditioning and Motivation for Stability Analysis

A major numerical issue in ELM is the possible ill-conditioning of the hidden-layer matrix $\mathbf{H}$. Let the singular value decomposition of $\mathbf{H}$ be

$$\mathbf{H} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T, \quad \mathbf{\Sigma} = \text{diag}(\sigma_1, \sigma_2, \dots, \sigma_r), \quad \sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0.$$

Then

$$\mathbf{H}^+ = \mathbf{V}\mathbf{\Sigma}^+\mathbf{U}^T, \quad \|\mathbf{H}^+\|_2 = \frac{1}{\sigma_r}.$$

This identity shows that when the smallest nonzero singular value σ_r is very small, the norm of the pseudoinverse becomes very large. As a result, small perturbations in the matrix $\mathbf{H}$, in the target matrix $\mathbf{T}$, or in floating-point arithmetic may be strongly amplified in the computed output weights β .

This issue is especially relevant in ELM, because $\mathbf{H}$ is not designed through optimization but generated from random hidden parameters. Consequently, ELM may produce hidden-layer matrices that are nearly rank-deficient or poorly conditioned, even when the network is theoretically expressive enough to fit the data.

The choice of pseudoinverse algorithm therefore becomes a crucial part of the ELM framework. Different numerical methods for computing $\mathbf{H}^+$ may produce the same theoretical solution in exact arithmetic, but their behavior can differ significantly in finite precision. In particular, SVD-based methods are generally more robust because they reveal the singular-value structure of $\mathbf{H}$ explicitly, whereas iterative inverse approximations may be more sensitive to conditioning, initialization, and stopping criteria.

This motivates the next chapters of the thesis. After formulating ELM as a pseudoinverse-based analytical learning model, it becomes necessary to study how the pseudoinverse is computed in practice, how different computational methods compare, and how numerical stability affects the overall behavior of ELM. In this sense, the matrix $\mathbf{H}$ serves as the main link between generalized inverse theory, numerical linear algebra, and machine learning application.

5.7 Summary

This chapter has established the connection between single-hidden-layer neural networks, Extreme Learning Machine, and the Moore–Penrose inverse. Once the hidden-layer parameters are fixed, the ELM training problem reduces to the linear system

$$\mathbf{H}\beta = \mathbf{T},$$

whose canonical analytical solution is

$$\beta = \mathbf{H}^+\mathbf{T}.$$

This reformulation transforms neural network training from a nonlinear iterative optimization problem into a linear algebra problem. At the same time, it highlights the central importance of the hidden-layer output matrix $\mathbf{H}$, whose rank, dimension, and conditioning determine the numerical behavior of the method.

Therefore, understanding ELM requires not only neural network theory, but also a careful study of pseudoinverse computation. The next chapter develops the numerical methods used to compute the Moore–Penrose inverse, while the subsequent stability analysis will clarify how these choices influence the reliability and performance of ELM in practice.

Numerical Methods for Pseudoinverse Computation

Chapter Overview:

This chapter reviews the principal numerical approaches used to compute the Moore–Penrose pseudoinverse. The first part focuses on SVD-based methods, including the mathematical construction of the pseudoinverse from singular values and practical algorithms such as Golub–Reinsch, divide-and-conquer SVD, and one-sided Jacobi SVD. The second part presents iterative approaches, especially Newton–Schulz and higher-order hyperpower methods. The chapter compares these families conceptually in terms of computational structure, convergence behavior, and practical suitability for ELM.

6.1 Introduction

The Moore–Penrose inverse plays a central role in the analytical training of Extreme Learning Machines. Once the hidden-layer output matrix $\mathbf{H}$ has been constructed, the training problem reduces to the linear system

$$\mathbf{H}\boldsymbol{\beta} = \mathbf{T},$$

whose canonical solution is

$$\boldsymbol{\beta} = \mathbf{H}^+\mathbf{T}.$$

Therefore, the effectiveness of ELM in practice depends not only on the theoretical properties of the pseudoinverse, but also on how $\mathbf{H}^+$ is computed numerically.

In exact arithmetic, the Moore–Penrose inverse is uniquely defined. In finite-precision computation, however, different algorithms may yield solutions with different numerical stability, computational cost, and sensitivity to conditioning. This is especially important when the matrix is large, rectangular, rank-deficient, or ill-conditioned. In such cases, the choice of numerical method can strongly affect both runtime and the quality of the computed solution.

This chapter presents the principal numerical approaches used to compute the pseudoinverse. We begin with singular value decomposition (SVD), which provides the most standard and mathematically transparent construction of $\mathbf{A}^+$. We then discuss practical algorithms for computing the SVD, including Golub–Reinsch, divide-and-conquer, and one-sided Jacobi methods. After that, we consider iterative schemes for directly approximating the inverse or pseudoinverse, focusing on the hyperpower family and its higher-order variants. Finally, we compare these approaches from the perspective of accuracy, efficiency, and suitability for ELM.

6.2 Pseudoinverse from Singular Value Decomposition

The most fundamental numerical method for computing the Moore–Penrose inverse is based on the singular value decomposition. Let

$$\mathbf{A} \in \mathbb{R}^{m \times n}$$

have full SVD

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T,$$

where

$$\mathbf{U} \in \mathbb{R}^{m \times m}, \quad \mathbf{V} \in \mathbb{R}^{n \times n}$$

are orthogonal matrices and

$$\mathbf{\Sigma} \in \mathbb{R}^{m \times n}$$

is a rectangular diagonal matrix whose diagonal entries are the singular values

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0,$$

with the remaining diagonal entries equal to zero.

As established in Chapter 2, the Moore–Penrose inverse is then given by

$$\mathbf{A}^+ = \mathbf{V}\mathbf{\Sigma}^+\mathbf{U}^T,$$

where Σ^+ is obtained by replacing each nonzero singular value σ_i with its reciprocal $1/\sigma_i$, while zero singular values remain zero.

This formula is valid for arbitrary matrices, including rectangular and rank-deficient ones. For this reason, SVD-based pseudoinverse computation is regarded as the most general and reliable approach. It automatically handles all of the cases that arise in practice:

- ▶ square and nonsingular matrices,
- ▶ overdetermined systems,
- ▶ underdetermined systems,
- ▶ rank-deficient matrices,
- ▶ ill-conditioned matrices.

Moreover, the SVD reveals the spectral structure of the matrix explicitly. Since

$$\|\mathbf{A}^+\|_2 = \frac{1}{\sigma_r},$$

where σ_r is the smallest nonzero singular value, the SVD provides direct insight into conditioning and numerical sensitivity. This makes it particularly well suited for stability analysis, which will be developed in the next chapter.

6.3 Direct Construction via the Gram Matrix

A conceptually simple route to the singular value decomposition starts from the matrix [2, p. 235]

$$\mathbf{A}^T \mathbf{A}.$$

Because $\mathbf{A}^T \mathbf{A}$ is symmetric positive semidefinite, it admits an orthogonal eigendecomposition

$$\mathbf{A}^T \mathbf{A} = \mathbf{V} \mathbf{\Lambda} \mathbf{V}^T,$$

where the eigenvalues are nonnegative. If

$$\mathbf{\Lambda} = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_n),$$

then the singular values of $\mathbf{A}$ are

$$\sigma_i = \sqrt{\lambda_i}.$$

Once $\mathbf{V}$ and the singular values are known, one may define the left singular vectors by

$$\mathbf{u}_i = \frac{\mathbf{A} \mathbf{v}_i}{\sigma_i}, \quad \sigma_i > 0,$$

and thereby recover the factorization

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T.$$

This construction is mathematically natural and useful for theoretical exposition. However, it is not generally preferred in numerical practice. The main reason is that forming $\mathbf{A}^T\mathbf{A}$ may worsen conditioning. Indeed, if the singular values of $\mathbf{A}$ are σ_i , then the eigenvalues of $\mathbf{A}^T\mathbf{A}$ are σ_i^2 . Consequently, small singular values are squared before being recovered through a square root, which may amplify roundoff effects and reduce accuracy for ill-conditioned problems.

Therefore, although the Gram-matrix approach is pedagogically valuable, modern numerical linear algebra avoids it when a high-quality SVD is required. Instead, practical algorithms compute the singular values more directly.

6.4 Golub–Reinsch Algorithm

The classical high-quality algorithm for computing the singular value decomposition of a dense matrix is the Golub–Reinsch method [10, 11]. Its importance lies in the fact that it avoids forming $\mathbf{A}^T\mathbf{A}$ explicitly and instead transforms the matrix through orthogonal operations, which are numerically stable [2, p. 234].

The method consists of two major stages.

6.4.1 Bidiagonalization

In the first stage, the matrix $\mathbf{A}$ is reduced to bidiagonal form by a sequence of Householder reflections. More precisely, one computes orthogonal matrices

$$\mathbf{Q}_1 \in \mathbb{R}^{m \times m}, \quad \mathbf{Q}_2 \in \mathbb{R}^{n \times n}$$

such that

$$\mathbf{Q}_1^T \mathbf{A} \mathbf{Q}_2 = \mathbf{B},$$

where $\mathbf{B}$ is upper bidiagonal. Because Householder transformations are orthogonal, they preserve the singular values of the matrix and introduce only small backward error.

The bidiagonalization stage is crucial because the singular value decomposition of a bidiagonal matrix is much easier to compute than that of a general dense matrix [2, p. 236].

6.4.2 Diagonalization of the Bidiagonal Matrix

In the second stage, the bidiagonal matrix is reduced to diagonal form using an implicit QR-type iteration, typically with Wilkinson shifts. If

$$\mathbf{B} = \tilde{\mathbf{U}}\tilde{\Sigma}\tilde{\mathbf{V}}^T,$$

then the SVD of the original matrix is

$$\mathbf{A} = (\mathbf{Q}_1\tilde{\mathbf{U}})\tilde{\Sigma}(\mathbf{Q}_2\tilde{\mathbf{V}})^T.$$

This procedure yields the singular values directly, without passing through the eigenvalues of $\mathbf{A}^T\mathbf{A}$. For this reason, the Golub–Reinsch method is considerably more stable than the naive eigendecomposition route. It is the classical foundation of many standard numerical libraries and remains a benchmark method for dense SVD computation.

For pseudoinverse computation, the resulting singular values can be thresholded when necessary, and the pseudoinverse is then assembled as

$$\mathbf{A}^+ = \mathbf{V}\Sigma^+\mathbf{U}^T.$$

6.5 Divide-and-Conquer SVD

For large dense matrices, especially when all singular vectors are required, divide-and-conquer methods are often faster than the classical Golub–Reinsch diagonalization phase. The key idea is to recursively split the bidiagonal matrix into smaller subproblems and then merge the solutions.

After reducing $\mathbf{A}$ to bidiagonal form, one writes the bidiagonal matrix as a structured rank-one modification of two smaller bidiagonal blocks. The smaller subproblems are solved recursively, and the singular values of the full problem are recovered from a secular equation associated with the rank-one update.

Schematically, the method reduces the problem to the analysis of matrices of the form

$$\mathbf{M} = \mathbf{D} + \rho\mathbf{z}\mathbf{z}^T,$$

where $\mathbf{D}$ is diagonal, ρ is a scalar, and $\mathbf{z}$ is a vector. The updated singular values are then obtained by solving a secular equation. A central numerical issue in this step is the preservation of orthogonality of the computed singular vectors. High-quality divide-and-conquer algorithms therefore incorporate correction mechanisms that ensure stable vector reconstruction.

The main advantage of divide-and-conquer SVD is speed. In many practical settings it outperforms classical bidiagonal QR iterations, which is why it is widely used in modern libraries. The main trade-off is that the method is algorithmically more sophisticated and requires careful implementation to maintain numerical robustness.

For pseudoinverse computation, divide-and-conquer remains an SVD-based method and therefore retains the same conceptual advantages as Golub–Reinsch: generality, rank-awareness, and strong numerical reliability.

6.6 One-Sided Jacobi SVD

Another important approach to computing the singular value decomposition is the one-sided Jacobi method. Unlike Golub–Reinsch, which first reduces the matrix to bidiagonal form, the Jacobi method acts directly on the columns of the matrix.

The basic step applies a plane rotation to two columns of the matrix so that they become more nearly orthogonal. Repeating this procedure over many column pairs drives the matrix toward a form whose columns are mutually orthogonal. Once convergence is reached, the singular values are given by the column norms, the normalized columns yield the left singular vectors, and the accumulated rotations yield the right singular vectors.

If $J(p, q, \theta)$ denotes a Jacobi rotation acting on columns p and q , then the iteration has the form

$$A \leftarrow AJ(p, q, \theta),$$

with the angle θ chosen so that the updated columns p and q are orthogonal.

The one-sided Jacobi method is attractive because of its excellent orthogonality properties. In many cases it can compute singular vectors with very high accuracy. This is especially useful when the matrix is ill-conditioned or when the quality of the singular vectors is as important as that of the singular values.

Its main drawback is computational cost. Jacobi-type algorithms are usually slower than Golub–Reinsch or divide-and-conquer on large dense matrices. Nevertheless, optimized variants remain important in practice, especially for small matrices, batched computations, and certain parallel environments.

In modern implementations, additional improvements are often introduced, such as:

- ▶ pre-sorting columns by norm,
- ▶ threshold pivoting to skip nearly orthogonal column pairs,

- vectorized or batched rotations.

These enhancements make one-sided Jacobi attractive in machine learning contexts where many small SVD computations must be performed efficiently.

6.7 Iterative Methods for Inverse and Pseudoinverse Approximation

Besides direct factorization-based methods, one may approximate the inverse or pseudoinverse iteratively. These methods construct a sequence of matrices

$$\{\mathbf{X}_k\}_{k=0}^{\infty}$$

intended to converge to $\mathbf{A}^{-1}$ or $\mathbf{A}^+$, depending on the problem setting.

The appeal of iterative methods lies in their reliance on matrix multiplication, which can be highly efficient on modern hardware. They are particularly attractive when one seeks a fast approximation, when the matrix size is large, or when specialized hardware favors repeated dense linear algebra kernels.

A general hyperpower iteration can be written as

$$\mathbf{X}_{k+1} = \mathbf{X}_k (\mathbf{I} + R_k + R_k^2 + \cdots + R_k^p),$$

where

$$R_k = \mathbf{I} - \mathbf{A}\mathbf{X}_k.$$

If the iteration converges, the residual R_k tends to zero and $\mathbf{X}_k$ approaches the desired inverse-type operator. For rectangular or rank-deficient pseudoinverse problems, this identity residual should be interpreted with care: the limiting product $\mathbf{A}\mathbf{A}^+$ is generally a projector, not the identity. In those cases, Penrose residuals are the appropriate general-purpose convergence measures.

These methods are elegant and often easy to parallelize, but their convergence depends strongly on initialization, the spectral properties of the matrix, and the problem class (square, rectangular, full-rank, rank-deficient, or ill-conditioned). For this reason, they require more careful analysis than SVD-based approaches.

6.8 Newton–Schulz Iteration

The best-known member of the hyperpower family is the Newton–Schulz iteration, obtained when $p = 1$ in the inverse setting or, equivalently, when one writes the quadratic

correction in the form

$$\mathbf{X}_{k+1} = \mathbf{X}_k(2\mathbf{I} - \mathbf{A}\mathbf{X}_k).$$

This method is attractive for several reasons:

- ▶ each step uses only matrix multiplications and additions,
- ▶ it is easy to implement,
- ▶ it can exhibit quadratic convergence near the solution.

A common practical initialization is

$$\mathbf{X}_0 = \frac{\mathbf{A}^T}{\|\mathbf{A}\|_1 \|\mathbf{A}\|_\infty},$$

which scales the starting approximation so that the iteration begins in a stable region for many matrices.

To illustrate the idea, one starts with $\mathbf{X}_0$, forms the residual-like factor

$$2\mathbf{I} - \mathbf{A}\mathbf{X}_0,$$

and updates via

$$\mathbf{X}_1 = \mathbf{X}_0(2\mathbf{I} - \mathbf{A}\mathbf{X}_0).$$

Repeated application of this rule can drive the approximation toward the true inverse or pseudoinverse.

Despite its theoretical attractiveness, Newton–Schulz has an important practical limitation: the early iterations may converge slowly unless the initial approximation is sufficiently good. Moreover, when the matrix is ill-conditioned or the initialization is not well chosen, convergence may stagnate or fail. Thus, although the method is simple and hardware-friendly, its robustness is generally lower than that of SVD-based methods.

6.9 Higher-Order Hyperpower Methods

To improve the convergence speed of Newton–Schulz, researchers have developed higher-order iterative schemes. The idea is straightforward: if each step is constructed from a higher-degree polynomial in the residual, then fewer iterations may be required to reach a prescribed tolerance.

6.9.1 Chebyshev-Type Cubic Iteration

A well-known third-order method has the form

$$\mathbf{X}_{k+1} = \mathbf{X}_k (3\mathbf{I} - \mathbf{A}\mathbf{X}_k (3\mathbf{I} - \mathbf{A}\mathbf{X}_k)).$$

Compared with Newton–Schulz, this method can reduce the iteration count significantly. However, the gain in convergence order comes at the cost of additional matrix products per iteration.

6.9.2 Other Cubic Schemes

Other third-order methods may be derived from midpoint-type or Homeier-type constructions. Typical examples include

$$\mathbf{X}_{k+1} = \left(\mathbf{I} - \frac{1}{4}(\mathbf{I} - \mathbf{X}_k\mathbf{A})(3\mathbf{I} - \mathbf{X}_k\mathbf{A})^2 \right) \mathbf{X}_k$$

and

$$\mathbf{X}_{k+1} = \mathbf{X}_k \left(\mathbf{I} + \frac{1}{2}(\mathbf{I} - \mathbf{A}\mathbf{X}_k)(\mathbf{I} + (2\mathbf{I} - \mathbf{A}\mathbf{X}_k)^2) \right).$$

These methods seek a better balance between convergence speed and per-step cost.

6.9.3 Fourth- and Fifth-Order Methods

Even higher-order formulas have also been proposed. A representative fourth-order scheme is

$$\mathbf{X}_{k+1} = \mathbf{X}_k (9\mathbf{I} - 26\mathbf{A}\mathbf{X}_k + 34(\mathbf{A}\mathbf{X}_k)^2 - 21(\mathbf{A}\mathbf{X}_k)^3 + 5(\mathbf{A}\mathbf{X}_k)^4).$$

Likewise, fifth-order schemes have been designed, for example

$$\mathbf{X}_{k+1} = \mathbf{X}_k (5\mathbf{I} - 10\mathbf{A}\mathbf{X}_k + 10(\mathbf{A}\mathbf{X}_k)^2 - 5(\mathbf{A}\mathbf{X}_k)^3 + (\mathbf{A}\mathbf{X}_k)^4),$$

as well as more elaborate polynomial constructions.

In practice, these expressions are usually factorized to recycle intermediate products and reduce the number of matrix multiplications. This is essential because the nominal order of convergence alone does not determine efficiency. A method with fewer iterations may still be slower overall if each iteration is too expensive.

Thus, the practical performance of high-order methods depends on three competing factors:

1. convergence order,

2. number of matrix multiplications per iteration,
3. numerical robustness in finite precision.

6.10 Stopping Criteria and Numerical Safeguards

Iterative methods require a stopping rule. For square nonsingular inverse problems, a common approach is to terminate the iteration once the identity residual becomes sufficiently small, for example when

$$\|\mathbf{I} - \mathbf{A}\mathbf{X}_k\| < \varepsilon$$

for a prescribed tolerance $\varepsilon > 0$, or after a maximum number of iterations has been reached. This criterion is not universal for pseudoinverses. For rectangular or rank-deficient matrices, $\mathbf{A}\mathbf{A}^+$ and $\mathbf{A}^+\mathbf{A}$ are orthogonal projectors rather than identities. More appropriate residuals are the Penrose residuals

$$\frac{\|\mathbf{A}\mathbf{X}_k\mathbf{A} - \mathbf{A}\|}{\|\mathbf{A}\|}, \quad \frac{\|\mathbf{X}_k\mathbf{A}\mathbf{X}_k - \mathbf{X}_k\|}{\|\mathbf{X}_k\|},$$

together with symmetry residuals for $\mathbf{A}\mathbf{X}_k$ and $\mathbf{X}_k\mathbf{A}$ when the full Moore–Penrose conditions must be checked.

In pseudoinverse computation, several safeguards are important:

- **Scaling and initialization:** poor initialization may prevent convergence.
- **Maximum iteration count:** high-order methods may still stagnate on difficult matrices,
- **Tolerance choice:** excessively tight tolerances may increase runtime with little practical gain,
- **Rank sensitivity:** near rank-deficiency can cause slow or unstable behavior.

For these reasons, iterative schemes are often most successful when combined with careful preprocessing, scaling, and empirical tuning.

6.11 Comparison Between SVD-Based and Iterative Approaches

The two major families of methods discussed above have complementary strengths and weaknesses.

6.11.1 Advantages of SVD-Based Methods

SVD-based methods provide:

- ▶ a mathematically exact representation of the Moore–Penrose inverse,
- ▶ natural handling of rectangular and rank-deficient matrices,
- ▶ direct access to singular values and rank information,
- ▶ strong numerical stability,
- ▶ transparent support for thresholding and regularization.

These properties make SVD-based approaches the standard reference for reliable pseudoinverse computation.

6.11.2 Advantages of Iterative Methods

Iterative methods provide:

- ▶ implementations based primarily on matrix multiplication,
- ▶ potential efficiency on parallel architectures,
- ▶ flexible approximations when an exact high-quality SVD is unnecessary,
- ▶ reduced iteration counts through higher-order updates.

These features make them attractive in hardware-oriented and large-scale settings.

6.11.3 Limitations

However, the limitations are equally important:

- ▶ SVD-based methods may be computationally expensive for very large problems.
- ▶ Iterative methods depend heavily on initialization and conditioning.
- ▶ Higher-order iterations reduce iteration count but increase per-step algebraic cost.
- ▶ Ill-conditioned matrices can severely degrade iterative performance.

Therefore, the choice of method depends on the balance one seeks between reliability, speed, and implementation complexity.

6.12 Implications for Extreme Learning Machine

In ELM, the matrix whose pseudoinverse must be computed is the hidden-layer output matrix

$$\mathbf{H} \in \mathbb{R}^{N \times L}.$$

This matrix is often rectangular and may be ill-conditioned or nearly rank-deficient, depending on the activation function, the hidden-layer width, and the random choice of input weights and biases.

From a numerical standpoint, this makes pseudoinverse computation the core step of the ELM training pipeline. If $\mathbf{H}^+$ is computed inaccurately, then the output weights

$$\beta = \mathbf{H}^+ \mathbf{T}$$

may be highly sensitive to perturbations, which can in turn affect prediction performance.

Because SVD-based methods explicitly reveal the singular value structure of $\mathbf{H}$, they are particularly well suited for ELM. They allow one to detect small singular values, understand the conditioning of the hidden-layer matrix, and obtain stable minimum-norm solutions. Iterative methods, on the other hand, may offer speed advantages in some settings, but their success depends more strongly on the numerical properties of $\mathbf{H}$.

Hence, the comparison of pseudoinverse algorithms is not merely a question of abstract numerical linear algebra. In the context of ELM, it directly determines the quality, stability, and computational efficiency of the learned model.

6.13 Summary

This chapter has presented the main numerical methods used to compute the Moore–Penrose inverse. The SVD-based approach remains the most fundamental and reliable method, since it is valid for arbitrary matrices and provides direct spectral insight. Within this family, practical implementations include the classical Golub–Reinsch algorithm, divide-and-conquer methods, and one-sided Jacobi schemes.

We also examined iterative approaches based on the hyperpower framework, including Newton–Schulz and several higher-order variants. These methods are attractive because they rely mainly on matrix multiplication and may be efficient on modern hardware. However, their behavior depends more strongly on initialization and conditioning.

The main conclusion is that pseudoinverse computation is not a purely formal step. Different numerical methods can behave very differently in finite precision, especially for

large, rank-deficient, or ill-conditioned matrices. Since ELM training is built directly on the formula

$$\beta = \mathbf{H}^+ \mathbf{T},$$

the choice of pseudoinverse algorithm is central to the overall performance of the method.

The next chapter therefore investigates this issue from the viewpoint of numerical stability. In particular, we analyze how conditioning, perturbation amplification, and finite-precision effects influence pseudoinverse computation and, consequently, the behavior of Extreme Learning Machines.

Stability Analysis of Pseudoinverse Computation

Chapter Overview:

This chapter analyzes the numerical stability of pseudoinverse computation and its effect on Extreme Learning Machine. It studies the conditioning of the pseudoinverse problem through the singular values of the hidden-layer matrix and explains why small singular values amplify perturbations in the computed weights. The chapter compares the stability of SVD-based and hyperpower-based solvers, showing that SVD is more robust while hyperpower methods are more sensitive to initialization and ill-conditioning. These results are then connected directly to the stability of ELM, since the output weights are obtained from $\beta = H^+T$.

7.1 Introduction

In the previous chapters, we established that the Moore–Penrose inverse provides the canonical solution of the linear system arising in Extreme Learning Machine (ELM), namely

$$H\beta = T, \quad \beta = H^+T,$$

where H is the hidden-layer output matrix, T is the target matrix, and β is the output weight matrix. We also reviewed two major families of numerical methods for computing the pseu-

doinverse: singular value decomposition (SVD)-based methods and hyperpower iterative methods.

Although these methods are equivalent in exact arithmetic when they converge to the same Moore–Penrose inverse, their behavior in finite precision can differ substantially. In practical machine learning applications, numerical stability is therefore as important as computational efficiency. This is especially true when the matrix $\mathbf{H}$ is ill-conditioned or nearly rank-deficient, since small perturbations may then produce large variations in the computed output weights.

The purpose of this chapter is to analyze the stability of pseudoinverse computation and its impact on ELM. We first examine the conditioning of the pseudoinverse problem itself. We then compare the numerical stability of SVD-based methods and hyperpower iterative methods. Finally, we study how the singular values of the hidden-layer matrix influence the stability of the overall ELM model.

7.2 Conditioning of the Pseudoinverse Problem

Let

$$\mathbf{H} \in \mathbb{R}^{N \times L}$$

be the hidden-layer output matrix of an ELM, and suppose its compact singular value decomposition is

$$\mathbf{H} = \mathbf{U}_r \Sigma_r \mathbf{V}_r^T,$$

where

$$\Sigma_r = \text{diag}(\sigma_1, \sigma_2, \dots, \sigma_r), \quad \sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0,$$

and $r = \text{rank}(\mathbf{H})$.

Then the Moore–Penrose inverse is

$$\mathbf{H}^+ = \mathbf{V}_r \Sigma_r^{-1} \mathbf{U}_r^T,$$

and its spectral norm is

$$\|\mathbf{H}^+\|_2 = \frac{1}{\sigma_r}.$$

Thus, the smallest nonzero singular value σ_r controls the sensitivity of the inverse problem. If σ_r is very small, then $\|\mathbf{H}^+\|_2$ becomes very large, which means that perturbations in the data may be amplified strongly.

To see this, consider a perturbation $\Delta \mathbf{T}$ in the target matrix. The induced perturbation in the output weights is

$$\Delta \beta = \mathbf{H}^+ \Delta \mathbf{T},$$

so

$$\|\Delta \beta\|_2 \leq \|\mathbf{H}^+\|_2 \|\Delta \mathbf{T}\|_2 = \frac{\|\Delta \mathbf{T}\|_2}{\sigma_r}.$$

Hence small singular values cause instability in the learned weights.

Likewise, if the matrix $\mathbf{H}$ itself is perturbed to $\mathbf{H} + \Delta \mathbf{H}$, then the computed pseudoinverse may change significantly when σ_r is small. Therefore, before studying any specific numerical algorithm, one must recognize that the pseudoinverse problem is intrinsically sensitive when the hidden-layer matrix is ill-conditioned.

7.2.1 A Random Matrix Justification for the Width–Stability Bound

The hidden-layer matrix in ELM is given by

$$\mathbf{H}_{ji} = g(\mathbf{w}_i^\top \mathbf{x}_j + b_i), \quad j = 1, \dots, N, \quad i = 1, \dots, L.$$

Since the hidden weights $\mathbf{w}_i$ and biases b_i are randomly generated, the matrix $\mathbf{H}$ is a random feature matrix. However, its entries are not independent in full generality, because all entries in the i -th column depend on the same random pair $(\mathbf{w}_i, b_i)$.

Therefore, to obtain a rigorous random matrix estimate, we introduce the following idealized assumption.

Definition 7.1 – Sub-Gaussian norm

For a real-valued random variable Z , its sub-Gaussian or ψ_2 -norm is defined by

$$\|Z\|_{\psi_2} = \inf \left\{ t > 0 : \mathbb{E} \exp \left(\frac{Z^2}{t^2} \right) \leq 2 \right\}.$$

A random variable is called sub-Gaussian when its ψ_2 -norm is finite.

Assumption 7.1 – Sub-Gaussian hidden-layer matrix

Let $\mathbf{H} \in \mathbb{R}^{N \times L}$, with $N > L$, be a random hidden-layer matrix. Assume that the normalized entries of $\mathbf{H}$ are independent, centered, sub-Gaussian random variables with unit variance. More precisely, assume that

$$\mathbb{E} \mathbf{H}_{ji} = 0, \quad \mathbb{E} \mathbf{H}_{ji}^2 = 1,$$

and that there exists a constant $K > 0$ such that

$$\|\mathbf{H}_{ji}\|_{\psi_2} \leq K$$

for all $j = 1, \dots, N$ and $i = 1, \dots, L$.

Theorem 7.1 – Width-dependent lower bound for the smallest singular value

Let $\mathbf{H} \in \mathbb{R}^{N \times L}$ satisfy the above sub-Gaussian assumption, with $N > L$. Then there exist constants $c > 0$ and $C > 0$, depending only on the sub-Gaussian norm bound K , such that

$$\mathbb{P}\left(\sigma_{\min}(\mathbf{H}) \leq c(\sqrt{N} - \sqrt{L})\right) \leq 2 \exp(-C(N - L)).$$

Equivalently, with probability at least

$$1 - 2 \exp(-C(N - L)),$$

we have

$$\sigma_{\min}(\mathbf{H}) \geq c(\sqrt{N} - \sqrt{L}).$$

Consequently,

$$\|\mathbf{H}^+\|_2 \leq \frac{1}{c(\sqrt{N} - \sqrt{L})}.$$

Proof: We provide the proof by invoking the classical smallest-singular-value estimate for rectangular sub-Gaussian random matrices.

Let $\mathbf{H} \in \mathbb{R}^{N \times L}$ with $N > L$. Under the stated assumptions, $\mathbf{H}$ is a rectangular random matrix whose entries are independent, centered, sub-Gaussian random variables with unit variance.

A standard theorem [26] in random matrix theory states that for such a matrix, there exist constants $c > 0$ and $C > 0$, depending only on the sub-Gaussian norm of the entries, such that

$$\mathbb{P}\left(\sigma_{\min}(\mathbf{H}) \leq c(\sqrt{N} - \sqrt{L})\right) \leq 2 \exp(-C(N - L)).$$

This estimate means that the smallest singular value of $\mathbf{H}$ is bounded from below by a quantity of order $\sqrt{N} - \sqrt{L}$ with high probability.

Therefore, with probability at least

$$1 - 2 \exp(-C(N - L)),$$

we obtain

$$\sigma_{\min}(\mathbf{H}) \geq c(\sqrt{N} - \sqrt{L}).$$

Now recall the spectral identity for the Moore–Penrose pseudoinverse:

$$\|\mathbf{H}^+\|_2 = \frac{1}{\sigma_{\min}(\mathbf{H})}$$

whenever $\mathbf{H}$ has full column rank. Since the previous estimate implies $\sigma_{\min}(\mathbf{H}) > 0$ with high probability, $\mathbf{H}$ has full column rank with high probability.

Hence,

$$\|\mathbf{H}^+\|_2 = \frac{1}{\sigma_{\min}(\mathbf{H})} \leq \frac{1}{c(\sqrt{N} - \sqrt{L})}.$$

This proves the theorem. $\square$

Remark 7.1

The theorem itself is a classical result about smallest singular value estimates for rectangular sub-Gaussian random matrices. The novelty in the present work lies in interpreting these spectral estimates within the framework of pseudoinverse-based ELM training, thereby establishing a width-dependent perspective on the conditioning and stability of random hidden-layer representations.

7.3 Stability of the SVD-Based Pseudoinverse

7.3.1 Backward Stability of the SVD

The SVD-based pseudoinverse is obtained from the factorization

$$\mathbf{H} = \mathbf{U}\Sigma\mathbf{V}^T$$

through the formula

$$\mathbf{H}^+ = \mathbf{V}\Sigma^+\mathbf{U}^T.$$

This approach is generally considered the most numerically reliable, because SVD algorithms are based on orthogonal transformations, such as Householder reflections, Givens rotations, and bidiagonal QR iterations. Orthogonal transformations preserve the Euclidean norm and do not magnify errors significantly.

In particular, if a numerical SVD algorithm produces

$$\hat{\mathbf{H}} = \hat{\mathbf{U}}\hat{\Sigma}\hat{\mathbf{V}}^T,$$

then the computed factorization is typically the exact SVD of a nearby matrix

$$\mathbf{H} + \mathbf{E},$$

where $\|\mathbf{E}\|$ is small relative to $\|\mathbf{H}\|$. This is the standard backward-stability property of SVD-based methods [2, p. 104]. As a consequence, the computed pseudoinverse

$$\widehat{\mathbf{H}}^+ = \widehat{\mathbf{V}}\widehat{\Sigma}^+\widehat{\mathbf{U}}^T$$

is the exact Moore–Penrose inverse of a slightly perturbed matrix, rather than an uncontrolled approximation.

This property is one of the main reasons why SVD is the standard reference method for pseudoinverse computation.

7.3.2 Sensitivity Through Singular Values

Although SVD is algorithmically stable, it cannot remove the intrinsic instability of the problem itself [2, p. 89]. If $\mathbf{H}$ contains very small singular values, then the reciprocals

$$\frac{1}{\sigma_i}$$

must appear in $\mathbf{H}^+$. Hence the corresponding directions are amplified.

Suppose $\mathbf{H}$ is perturbed by a small matrix $\mathbf{E}$. By singular value perturbation theory, the singular values of $\mathbf{H} + \mathbf{E}$ differ from those of $\mathbf{H}$ by at most $\|\mathbf{E}\|_2$. Therefore, if σ_r is close to zero, even a small perturbation may noticeably change the smallest singular directions. This means that the pseudoinverse is most sensitive in the directions associated with the smallest singular values.

The crucial point is that SVD exposes this instability transparently. Since the singular values are computed explicitly, one can detect near rank-deficiency and, if desired, truncate or regularize the smallest singular values. Thus, SVD does not hide the numerical difficulty; it reveals it in a controlled manner.

7.3.3 Implication for ELM

For ELM, this means that if the hidden-layer matrix $\mathbf{H}$ is ill-conditioned, SVD-based training may still be trusted to compute the correct least-squares or minimum-norm solution of a nearby problem. The instability that remains is mainly due to the hidden-layer matrix itself, not to the numerical algorithm used to compute the pseudoinverse.

Therefore, from a stability point of view, SVD-based ELM inherits the conditioning of the learning problem, but introduces relatively little additional numerical error.

7.4 Stability of Hyperpower Iterative Methods

7.4.1 Basic Error Propagation

Consider the Newton–Schulz iteration

$$\mathbf{X}_{k+1} = \mathbf{X}_k(2\mathbf{I} - \mathbf{H}\mathbf{X}_k),$$

which is the best-known member of the hyperpower family. Let

$$\mathbf{E}_k = \mathbf{I} - \mathbf{H}\mathbf{X}_k.$$

Then

$$\mathbf{X}_{k+1} = \mathbf{X}_k(\mathbf{I} + \mathbf{E}_k),$$

and therefore

$$\mathbf{I} - \mathbf{H}\mathbf{X}_{k+1} = \mathbf{I} - \mathbf{H}\mathbf{X}_k(2\mathbf{I} - \mathbf{H}\mathbf{X}_k) = (\mathbf{I} - \mathbf{H}\mathbf{X}_k)^2 = \mathbf{E}_k^2.$$

Thus, in exact arithmetic, the residual satisfies

$$\mathbf{E}_{k+1} = \mathbf{E}_k^2.$$

This explains the quadratic convergence of the method when $\|\mathbf{E}_0\| < 1$. For rectangular or rank-deficient pseudoinverse problems, this identity-residual argument must be restricted to the singular subspace on which $\mathbf{H}\mathbf{X}_k$ approximates the projector $\mathbf{H}\mathbf{H}^+$. Outside $\text{Col}(\mathbf{H})$, $\mathbf{H}\mathbf{X}_k$ approaches the orthogonal projector, not the identity, so $\|\mathbf{I} - \mathbf{H}\mathbf{X}_k\|_2$ is not a universal convergence measure. Penrose residuals such as

$$\frac{\|\mathbf{H}\mathbf{X}_k\mathbf{H} - \mathbf{H}\|}{\|\mathbf{H}\|}, \quad \frac{\|\mathbf{X}_k\mathbf{H}\mathbf{X}_k - \mathbf{X}_k\|}{\|\mathbf{X}_k\|}$$

are better suited for checking pseudoinverse convergence.

More generally, for higher-order hyperpower methods, the residual behaves like

$$\mathbf{E}_{k+1} \approx \mathbf{E}_k^p,$$

where p is the order of the method. Hence the theoretical attraction of these schemes lies in their potentially rapid local convergence.

7.4.2 Dependence on Initialization and Spectrum

Despite their fast local convergence, hyperpower methods are much more sensitive than SVD to initialization and matrix conditioning. For the Newton–Schulz iteration, a common choice is

$$\mathbf{X}_0 = \alpha\mathbf{H}^T.$$

If

$$\mathbf{H} = \mathbf{U}_r \Sigma_r \mathbf{V}_r^T,$$

then

$$\mathbf{H}\mathbf{X}_0 = \alpha \mathbf{H}\mathbf{H}^T = \mathbf{U}_r \text{diag}(\alpha \sigma_1^2, \dots, \alpha \sigma_r^2) \mathbf{U}_r^T.$$

Therefore the initial residual factor along the i -th singular direction is

$$1 - \alpha \sigma_i^2.$$

The worst-case contraction factor is thus

$$q(\alpha) = \max_i |1 - \alpha \sigma_i^2|.$$

The optimal scaling is obtained by balancing the extreme singular values, leading to

$$\alpha^* = \frac{2}{\sigma_1^2 + \sigma_r^2}.$$

Substituting this into the contraction factor yields

$$q^* = \frac{\sigma_1^2 - \sigma_r^2}{\sigma_1^2 + \sigma_r^2} = \frac{\kappa_2(\mathbf{H})^2 - 1}{\kappa_2(\mathbf{H})^2 + 1},$$

where

$$\kappa_2(\mathbf{H}) = \frac{\sigma_1}{\sigma_r}$$

is the effective spectral condition number of $\mathbf{H}$ on its nonzero singular subspace. This contraction argument is restricted to the singular subspace of $\mathbf{H}$; outside the column space, $\mathbf{H}\mathbf{X}_k$ approaches an orthogonal projector rather than the identity.

This expression shows that when $\mathbf{H}$ is well-conditioned, q^* is comfortably below 1 and convergence is rapid. However, when $\mathbf{H}$ is ill-conditioned, $\kappa_2(\mathbf{H})$ is large and q^* approaches 1. In that regime, the iteration contracts the error slowly and may require many steps before entering the fast asymptotic regime.

Thus, the stability and efficiency of hyperpower methods deteriorate as the condition number of the matrix increases.

7.4.3 Finite-Precision Effects

In finite precision, the iteration is affected by roundoff:

$$\widehat{\mathbf{X}}_{k+1} = \text{fl}(\widehat{\mathbf{X}}_k(2\mathbf{I} - \mathbf{H}\widehat{\mathbf{X}}_k)).$$

This means that the exact recurrence

$$E_{k+1} = E_k^2$$

is replaced by a perturbed version of the form

$$\widehat{E}_{k+1} = \widehat{E}_k^2 + \Delta_k,$$

where Δ_k represents floating-point error.

When the matrix is ill-conditioned, $\|\mathbf{H}^+\|_2 = 1/\sigma_r$ becomes large, and the iterates $\widehat{\mathbf{X}}_k$ may themselves attain large norms. This can magnify the effect of floating-point perturbations and produce an error floor below which the iteration cannot improve. In practice, the method may stagnate before reaching high accuracy, especially for nearly singular matrices.

Therefore, unlike SVD, hyperpower methods are not only affected by the conditioning of the problem, but also by the convergence dynamics of the algorithm. This makes them less robust for difficult matrices.

7.5 Effect of Singular Values and Ill-Conditioning

7.5.1 Spectral Interpretation

The singular values of $\mathbf{H}$ determine both the difficulty of the inverse problem and the behavior of the numerical method. The largest singular value σ_1 controls the scale of the matrix, while the smallest nonzero singular value σ_r controls the inverse amplification through

$$\|\mathbf{H}^+\|_2 = \frac{1}{\sigma_r}.$$

Thus, the condition number

$$\kappa_2(\mathbf{H}) = \frac{\sigma_1}{\sigma_r}$$

summarizes how stretched the singular spectrum is.

If $\kappa_2(\mathbf{H})$ is moderate, both SVD-based and iterative methods can perform well. If $\kappa_2(\mathbf{H})$ is very large, the pseudoinverse problem becomes highly sensitive, and iterative methods tend to suffer more severely.

7.5.2 Worst-Case Reconstruction Error

Let $\mathbf{X}$ be an approximation of $\mathbf{H}^+$. For a target matrix $\mathbf{T}$, define the reconstructed output by

$$\widehat{\mathbf{T}} = \mathbf{H}\mathbf{X}\mathbf{T}.$$

Then the reconstruction error is

$$\mathbf{T} - \widehat{\mathbf{T}} = (\mathbf{I} - \mathbf{H}\mathbf{X})\mathbf{T}.$$

Therefore

$$\|\mathbf{T} - \widehat{\mathbf{T}}\|_2 \leq \|\mathbf{I} - \mathbf{H}\mathbf{X}\|_2 \|\mathbf{T}\|_2.$$

For rectangular or rank-deficient $\mathbf{H}$, this expression includes both the irreducible projection error for target components outside $\text{Col}(\mathbf{H})$ and the solver error inside the singular subspace. Therefore, $\|\mathbf{I} - \mathbf{H}\mathbf{X}\|_2$ should not be used by itself as a stopping criterion for pseudoinverse convergence.

For the exact SVD pseudoinverse $\mathbf{X} = \mathbf{H}^+$, one has

$$\mathbf{H}\mathbf{H}^+ = \mathbf{P}_{\text{Col}(\mathbf{H})},$$

the orthogonal projector onto the column space of $\mathbf{H}$. Thus, for all targets in the column space of $\mathbf{H}$, reconstruction is exact.

For a Newton–Schulz-type method after k iterations with optimal scaling, the projector residual restricted to the singular subspace is approximately

$$\|\mathbf{P}_{\text{Col}(\mathbf{H})} - \mathbf{H}\mathbf{X}_k\|_2 \approx \left(\frac{\sigma_1^2 - \sigma_r^2}{\sigma_1^2 + \sigma_r^2} \right)^{2^k}.$$

Hence, if σ_r is very small, this quantity decreases slowly for fixed k . In the worst ill-conditioned case, the method may remain far from the true pseudoinverse unless a large number of iterations is used.

This explains, at the spectral level, why iterative methods become unreliable on ill-conditioned matrices.

7.6 Stability of ELM Based on Pseudoinverse Solvers

7.6.1 Propagation of Solver Error to Output Weights

Let $\widehat{\mathbf{H}}^+$ be the pseudoinverse produced by a numerical algorithm. The computed ELM weights are

$$\widehat{\boldsymbol{\beta}} = \widehat{\mathbf{H}}^+ \mathbf{T}.$$

Then

$$\widehat{\boldsymbol{\beta}} - \boldsymbol{\beta} = (\widehat{\mathbf{H}}^+ - \mathbf{H}^+) \mathbf{T}.$$

Thus,

$$\|\widehat{\beta} - \beta\|_2 \leq \|\widehat{\mathbf{H}^+} - \mathbf{H}^+\|_2 \|\mathbf{T}\|_2.$$

Therefore, any instability in pseudoinverse computation is transferred directly to the learned output weights.

Since ELM makes no iterative correction after computing β , the quality of the pseudoinverse step is decisive. In other words, ELM compresses the whole training problem into a single linear algebra operation, so the stability of that operation becomes the stability of the model itself.

7.6.2 SVD-Based ELM

SVD-based ELM is the more stable formulation. The reasons are:

- ▶ the SVD is backward stable;
- ▶ the singular values reveal the conditioning of $\mathbf{H}$;
- ▶ small singular values can be treated explicitly through truncation or regularization;
- ▶ the computed solution remains the minimum-norm least-squares solution of a nearby problem,

Hence, SVD-based ELM tends to fail gracefully: when the hidden-layer matrix is difficult, the SVD reflects that difficulty but does not introduce unnecessary algorithmic instability.

7.6.3 Hyperpower-Based ELM

Hyperpower-based ELM is more sensitive. In addition to the intrinsic instability caused by small singular values, it also depends on:

- ▶ the initial approximation $\mathbf{X}_0$,
- ▶ the convergence tolerance,
- ▶ the maximum number of iterations,
- ▶ the accumulation of floating-point error,

As a consequence, the method may produce output weights that are less stable than those from SVD, particularly when $\mathbf{H}$ is ill-conditioned. Even when the final training error appears similar, the underlying coefficients may be more sensitive to perturbation.

Under exact-pseudoinverse computation and the unregularized settings studied here, SVD-based ELM is expected to be more robust than the tested hyperpower-based implementations.

7.7 Regularization as a Stabilization Strategy

The previous analysis shows that instability is fundamentally driven by small singular values. A natural way to improve robustness is therefore to regularize the inverse problem. Instead of using the exact pseudoinverse solution

$$\beta = \mathbf{H}^+ \mathbf{T},$$

one may consider the Tikhonov-regularized solution

$$\beta_\lambda = (\mathbf{H}^T \mathbf{H} + \lambda \mathbf{I})^{-1} \mathbf{H}^T \mathbf{T}, \quad \lambda > 0.$$

If

$$\mathbf{H} = \mathbf{U}_r \Sigma_r \mathbf{V}_r^T,$$

then

$$\beta_\lambda = \mathbf{V}_r \operatorname{diag} \left(\frac{\sigma_i}{\sigma_i^2 + \lambda} \right) \mathbf{U}_r^T \mathbf{T}.$$

Compared with the exact pseudoinverse factor $1/\sigma_i$, the regularized factor

$$\frac{\sigma_i}{\sigma_i^2 + \lambda}$$

remains bounded even when σ_i is very small. Hence regularization reduces the instability caused by near rank-deficiency.

This idea is relevant for hyperpower-based methods, but it must be used carefully. Regularization may improve conditioning, but forming the normal-equation matrix $\mathbf{H}^T \mathbf{H}$ can square the condition number before regularization. In practice, ridge solutions should preferably be computed by SVD, QR, Cholesky with care, or by iterative solvers applied to the regularized system

$$\mathbf{H}^T \mathbf{H} + \lambda \mathbf{I}.$$

Thus, although regularization can reduce sensitivity to small singular values, it should not be presented as automatically making hyperpower iterations stable.

7.8 Summary

This chapter analyzed the numerical stability of pseudoinverse computation and its effect on Extreme Learning Machine. The main conclusions are as follows.

First, the stability of the pseudoinverse problem is governed by the singular values of the matrix $\mathbf{H}$, especially the smallest nonzero singular value σ_r . When σ_r is small, the pseudoinverse norm $\|\mathbf{H}^+\|_2 = 1/\sigma_r$ becomes large, and the inverse problem becomes sensitive to perturbation.

Second, SVD-based methods are algorithmically more stable than hyperpower iterative methods. The SVD is backward stable, reveals the singular-value structure explicitly, and allows near-singular directions to be treated in a controlled way.

Third, hyperpower methods, although attractive for their matrix-multiplication-based structure and high local convergence order, are more sensitive to conditioning, initialization, stopping criteria, and finite-precision effects. Their stability deteriorates significantly when the matrix is ill-conditioned.

Fourth, these matrix-level stability properties transfer directly to ELM, because the learned output weights are computed through

$$\beta = \mathbf{H}^+ \mathbf{T}.$$

Thus, the numerical stability of the pseudoinverse algorithm is a decisive factor in the stability of the overall learning model.

In summary, the theoretical analysis and the experiments in this thesis indicate that SVD-based ELM is the more robust formulation in the tested unregularized settings, while hyperpower-based ELM may be competitive only when the hidden-layer matrix is sufficiently well-conditioned and the stopping rule is matched to pseudoinverse residuals.

Experimental Analysis

Chapter Overview:

This chapter reports the experimental evaluation of the pseudoinverse methods studied in the previous chapters. Experiment A uses synthetic matrices with controlled singular-value spectra to isolate numerical behavior. Experiment B applies the same solver families inside an Extreme Learning Machine (ELM) classifier on capped subsets of MNIST, Fashion-MNIST, and ISOLET. Together, the experiments show that the hyperpower methods can be accurate on benign synthetic spectra, but that the SVD-based solvers are the most reliable choice among the tested configurations when the hidden-layer matrix is generated by sampled classification data.

8.1 Overview

The purpose of the experimental analysis is to connect the numerical discussion in Chapters 6 and 7 with observable behavior in computation. The central question is not only whether a method can produce a pseudoinverse, but also how accurately it satisfies the Moore–Penrose relations, how quickly it does so, and how its behavior changes when the matrix is ill-conditioned, rank deficient, or embedded in a learning problem.

The chapter contains two experiments. Experiment A is a controlled synthetic matrix benchmark. Its role is to separate numerical effects caused by matrix spectra from effects caused by a particular data set. Experiment B is an ELM dataset benchmark. Its role is to test whether the same pseudoinverse methods remain useful when the matrix is the hidden-layer

design matrix of a classifier.

The reported aggregate statistics are computed across tested configurations from a single random seed, not across repeated random seeds. Thus means, standard deviations, and medians summarize variation across matrix, spectrum, data-set, hidden-width, initialization, and activation configurations; they should not be interpreted as estimates of seed-to-seed variability.

8.2 Experiment A: Synthetic Matrix Benchmark

8.2.1 Design

For each trial, a matrix

$$\mathbf{A} \in \mathbb{R}^{m \times n}$$

is generated with a prescribed singular-value spectrum. The implementation constructs

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T,$$

where $\mathbf{U}$ and $\mathbf{V}$ are obtained from QR orthogonalization of random Gaussian matrices, and $\mathbf{\Sigma}$ contains the requested singular values. A random target matrix

$$\mathbf{B} \in \mathbb{R}^{m \times k}$$

is then generated, where k is a fixed positive integer representing the number of target vectors. In the experiments, we set $k=8$. Each method computes an approximate pseudoinverse $\widehat{\mathbf{A}}^+$. The computed solution is

$$\widehat{\mathbf{X}} = \widehat{\mathbf{A}}^+ \mathbf{B} \in \mathbb{R}^{n \times k}.$$

The benchmark uses the six matrix sizes

$$(m, n) \in \{(100, 100), (500, 500), (500, 200), (200, 500), (1000, 300), (300, 1000)\}.$$

These include square, tall, and wide matrices. The singular-value spectra are:

- ▶ well-conditioned: singular values decay linearly from 1 to 10^{-1} ;
- ▶ moderately ill-conditioned: singular values decay logarithmically from 1 to 10^{-6} ;
- ▶ severely ill-conditioned: singular values decay logarithmically from 1 to 10^{-12} ;
- ▶ rank deficient: the last 10% of singular values are set exactly to zero;
- ▶ clustered small: the smallest 10% of singular values are clustered near machine precision.

The evaluated methods are two SVD-based solvers, `svd_default` and `svd_jacobian`, and six iterative hyperpower solvers: `newton_schulz`, `chebyshev`, `homeier`, `esmaeili`, `kaur`, and `sheikhi`. The two result folders correspond to two fixed-point convergence thresholds:

$$\varepsilon_{\text{conv}} = 10^{-10} \quad \text{and} \quad \varepsilon_{\text{conv}} = 10^{-7}.$$

The implementation uses the combined Penrose residual for reporting, no separate residual-tolerance cutoff, a maximum residual guard of 10^3 , and 50 iterations for iterative solvers.

8.2.2 Metrics

The first metric is the relative least-squares residual

$$r_{\text{ls}} = \frac{\|\mathbf{A}\widehat{\mathbf{X}} - \mathbf{B}\|_F}{\|\mathbf{B}\|_F}.$$

This measures the quality of the solution of $\mathbf{A}\mathbf{X} \approx \mathbf{B}$. For rank-deficient or rectangular matrices this residual should not be interpreted as a pure measure of pseudoinverse accuracy, because $\mathbf{B}$ may contain components outside the range of $\mathbf{A}$.

The benchmark therefore also reports two relative Penrose residuals:

$$r_1 = \frac{\|\mathbf{A}\widehat{\mathbf{A}}^+\mathbf{A} - \mathbf{A}\|_F}{\|\mathbf{A}\|_F}, \quad r_2 = \frac{\|\widehat{\mathbf{A}}^+\mathbf{A}\widehat{\mathbf{A}}^+ - \widehat{\mathbf{A}}^+\|_F}{\|\widehat{\mathbf{A}}^+\|_F}.$$

Runtime, iteration count, final solver residual, convergence delta, and stopping reason are also recorded.

8.2.3 Results

Tables 8.1–8.5 summarize the method-level results separately for each prescribed spectrum. Each table reports medians over the six matrix-size configurations for that spectrum and threshold.

With the stricter threshold 10^{-10} , all methods behave reliably on the well-conditioned and moderately ill-conditioned spectra. In those cases, the iterative methods satisfy the Penrose relation $\mathbf{A}\mathbf{A}^+\mathbf{A} \approx \mathbf{A}$ to small residuals, often around 10^{-11} or smaller. The difficult cases are rank-deficient, clustered-small, and severely ill-conditioned matrices.

Relaxing the threshold to 10^{-7} increases the number of successful iterative runs from 96 out of 180 to 144 out of 180. The improvement comes from the rank-deficient and clustered-small spectra, as shown in Table 8.6.

Table 8.1: Experiment A median method summary for the well-conditioned spectrum.

Threshold	Method	Success	Runtime (s)	r_{ls}	r_1	r_2
10^{-10}	chebyshev	6/6	1.02×10^{-1}	1.16×10^{-15}	9.96×10^{-16}	8.83×10^{-16}
	esmaeili	6/6	8.32×10^{-2}	2.14×10^{-15}	2.15×10^{-15}	2.07×10^{-15}
	homeier	6/6	1.03×10^{-1}	1.21×10^{-15}	1.05×10^{-15}	9.40×10^{-16}
	kaur	6/6	8.48×10^{-2}	1.61×10^{-15}	1.55×10^{-15}	1.45×10^{-15}
	newton_schulz	6/6	1.27×10^{-1}	1.21×10^{-15}	8.47×10^{-16}	8.06×10^{-16}
	sheikhi	6/6	7.28×10^{-2}	1.22×10^{-15}	1.07×10^{-15}	1.12×10^{-15}
	svd_default	6/6	1.65×10^{-1}	7.54×10^{-15}	4.36×10^{-15}	3.74×10^{-15}
	svd_jacobian	6/6	1.58×10^{-1}	7.54×10^{-15}	4.36×10^{-15}	3.74×10^{-15}
10^{-7}	chebyshev	6/6	1.01×10^{-1}	1.23×10^{-15}	1.09×10^{-15}	8.97×10^{-16}
	esmaeili	6/6	8.10×10^{-2}	2.27×10^{-15}	2.25×10^{-15}	2.19×10^{-15}
	homeier	6/6	1.03×10^{-1}	1.34×10^{-15}	1.17×10^{-15}	9.40×10^{-16}
	kaur	6/6	8.68×10^{-2}	1.61×10^{-15}	1.55×10^{-15}	1.45×10^{-15}
	newton_schulz	6/6	1.29×10^{-1}	1.32×10^{-15}	9.16×10^{-16}	8.31×10^{-16}
	sheikhi	6/6	7.43×10^{-2}	1.22×10^{-15}	1.07×10^{-15}	1.12×10^{-15}
	svd_default	6/6	2.06×10^{-1}	7.54×10^{-15}	4.36×10^{-15}	3.74×10^{-15}
	svd_jacobian	6/6	1.58×10^{-1}	7.54×10^{-15}	4.36×10^{-15}	3.74×10^{-15}

Table 8.2: Experiment A median method summary for the moderately ill-conditioned spectrum.

Threshold	Method	Success	Runtime (s)	r_{ls}	r_1	r_2
10^{-10}	chebyshev	6/6	2.48×10^{-1}	2.56×10^{-11}	1.97×10^{-11}	1.03×10^{-11}
	esmaeili	6/6	1.69×10^{-1}	2.58×10^{-11}	2.38×10^{-11}	1.05×10^{-11}
	homeier	6/6	2.49×10^{-1}	2.58×10^{-11}	2.19×10^{-11}	1.01×10^{-11}
	kaur	6/6	1.99×10^{-1}	2.47×10^{-11}	2.44×10^{-11}	1.06×10^{-11}
	newton_schulz	6/6	3.14×10^{-1}	2.78×10^{-11}	1.74×10^{-11}	1.05×10^{-11}
	sheikhi	6/6	1.58×10^{-1}	2.78×10^{-11}	2.52×10^{-11}	1.04×10^{-11}
	svd_default	6/6	1.91×10^{-1}	1.32×10^{-10}	8.29×10^{-12}	1.42×10^{-11}
	svd_jacobian	6/6	1.90×10^{-1}	1.32×10^{-10}	8.29×10^{-12}	1.42×10^{-11}
10^{-7}	chebyshev	6/6	2.47×10^{-1}	2.56×10^{-11}	2.06×10^{-11}	1.03×10^{-11}
	esmaeili	6/6	1.70×10^{-1}	2.58×10^{-11}	2.38×10^{-11}	1.05×10^{-11}
	homeier	6/6	2.49×10^{-1}	2.42×10^{-11}	2.24×10^{-11}	1.04×10^{-11}
	kaur	6/6	1.98×10^{-1}	2.52×10^{-11}	2.45×10^{-11}	1.06×10^{-11}
	newton_schulz	6/6	3.15×10^{-1}	2.55×10^{-11}	1.72×10^{-11}	1.03×10^{-11}
	sheikhi	6/6	1.58×10^{-1}	2.81×10^{-11}	2.49×10^{-11}	1.04×10^{-11}
	svd_default	6/6	1.91×10^{-1}	1.32×10^{-10}	8.29×10^{-12}	1.42×10^{-11}
	svd_jacobian	6/6	1.91×10^{-1}	1.32×10^{-10}	8.29×10^{-12}	1.42×10^{-11}

Table 8.3: Experiment A median method summary for the severely ill-conditioned spectrum.

Threshold	Method	Success	Runtime (s)	r_{ls}	r_1	r_2
10^{-10}	chebyshev	0/6	3.85×10^{-1}	2.00×10^{-1}	2.63×10^{-6}	4.63×10^{-1}
	esmaeili	0/6	4.84×10^{-1}	1.46×10^{-5}	1.28×10^{-5}	5.49×10^{-6}
	homeier	0/6	4.40×10^{-1}	1.51×10^{-5}	1.34×10^{-5}	5.44×10^{-6}
	kaur	0/6	4.41×10^{-1}	1.52×10^{-5}	1.29×10^{-5}	5.45×10^{-6}
	newton_schulz	0/6	3.15×10^{-1}	6.34×10^{-1}	6.50×10^{-8}	4.95×10^{-1}
	sheikhi	0/6	4.79×10^{-1}	1.64×10^{-5}	1.53×10^{-5}	5.47×10^{-6}
	svd_default	6/6	2.50×10^{-1}	7.44×10^{-5}	4.39×10^{-6}	7.89×10^{-6}
	svd_jacobian	6/6	2.50×10^{-1}	7.44×10^{-5}	4.39×10^{-6}	7.89×10^{-6}
10^{-7}	chebyshev	0/6	3.85×10^{-1}	2.00×10^{-1}	2.63×10^{-6}	4.63×10^{-1}
	esmaeili	0/6	4.87×10^{-1}	1.46×10^{-5}	1.28×10^{-5}	5.49×10^{-6}
	homeier	0/6	4.40×10^{-1}	1.51×10^{-5}	1.34×10^{-5}	5.44×10^{-6}
	kaur	0/6	4.42×10^{-1}	1.52×10^{-5}	1.29×10^{-5}	5.45×10^{-6}
	newton_schulz	0/6	3.17×10^{-1}	6.34×10^{-1}	6.50×10^{-8}	4.95×10^{-1}
	sheikhi	0/6	4.80×10^{-1}	1.64×10^{-5}	1.53×10^{-5}	5.47×10^{-6}
	svd_default	6/6	2.50×10^{-1}	7.44×10^{-5}	4.39×10^{-6}	7.89×10^{-6}
	svd_jacobian	6/6	2.50×10^{-1}	7.44×10^{-5}	4.39×10^{-6}	7.89×10^{-6}

Table 8.4: Experiment A median method summary for the rank deficient spectrum.

Threshold	Method	Success	Runtime (s)	r_{ls}	r_1	r_2
10^{-10}	chebyshev	2/6	2.39×10^{-1}	3.32×10^{-1}	1.71×10^{-10}	1.00×10^0
	esmaeili	0/6	3.71×10^{-1}	5.43×10^8	4.63×10^8	1.12×10^9
	homeier	2/6	4.39×10^{-1}	3.32×10^{-1}	3.47×10^{-7}	1.00×10^0
	kaur	1/6	4.42×10^{-1}	1.38×10^1	1.36×10^1	1.51×10^1
	newton_schulz	5/6	2.08×10^{-1}	3.32×10^{-1}	1.96×10^{-14}	1.34×10^{-10}
	sheikhi	0/6	3.35×10^{-1}	1.43×10^7	1.68×10^7	3.68×10^7
	svd_default	6/6	1.82×10^{-1}	1.07×10^0	1.56×10^{-1}	6.94×10^{-1}
	svd_jacobian	6/6	1.82×10^{-1}	1.07×10^0	1.56×10^{-1}	6.94×10^{-1}
10^{-7}	chebyshev	6/6	1.63×10^{-1}	3.32×10^{-1}	2.35×10^{-14}	1.11×10^{-10}
	esmaeili	6/6	1.22×10^{-1}	3.32×10^{-1}	4.12×10^{-14}	5.65×10^{-9}
	homeier	6/6	1.63×10^{-1}	3.32×10^{-1}	2.48×10^{-14}	1.73×10^{-10}
	kaur	6/6	1.34×10^{-1}	3.32×10^{-1}	2.86×10^{-14}	2.21×10^{-10}
	newton_schulz	6/6	2.02×10^{-1}	3.32×10^{-1}	1.97×10^{-14}	6.68×10^{-11}
	sheikhi	6/6	1.07×10^{-1}	3.32×10^{-1}	3.43×10^{-14}	3.20×10^{-9}
	svd_default	6/6	1.82×10^{-1}	1.07×10^0	1.56×10^{-1}	6.94×10^{-1}
	svd_jacobian	6/6	1.82×10^{-1}	1.07×10^0	1.56×10^{-1}	6.94×10^{-1}

Table 8.5: Experiment A median method summary for the clustered small spectrum.

Threshold	Method	Success	Runtime (s)	r_{ls}	r_1	r_2
10^{-10}	chebyshev	6/6	1.00×10^{-1}	3.32×10^{-1}	7.58×10^{-15}	1.32×10^{-11}
	esmaeili	0/6	4.85×10^{-1}	1.92×10^{-2}	1.82×10^{-2}	7.96×10^{-3}
	homeier	6/6	1.02×10^{-1}	3.32×10^{-1}	7.58×10^{-15}	2.02×10^{-11}
	kaur	6/6	8.50×10^{-2}	3.32×10^{-1}	7.99×10^{-15}	3.37×10^{-11}
	newton_schulz	6/6	1.24×10^{-1}	3.32×10^{-1}	7.53×10^{-15}	6.52×10^{-12}
	sheikhi	0/6	4.79×10^{-1}	2.28×10^{-2}	2.21×10^{-2}	8.26×10^{-3}
	svd_default	6/6	2.20×10^{-1}	8.25×10^{-2}	6.13×10^{-3}	1.15×10^{-2}
	svd_jacobian	6/6	2.21×10^{-1}	8.25×10^{-2}	6.13×10^{-3}	1.15×10^{-2}
10^{-7}	chebyshev	6/6	1.02×10^{-1}	3.32×10^{-1}	7.57×10^{-15}	1.11×10^{-11}
	esmaeili	6/6	8.05×10^{-2}	3.32×10^{-1}	1.12×10^{-14}	3.58×10^{-10}
	homeier	6/6	1.03×10^{-1}	3.32×10^{-1}	7.58×10^{-15}	1.67×10^{-11}
	kaur	6/6	8.65×10^{-2}	3.32×10^{-1}	7.99×10^{-15}	3.37×10^{-11}
	newton_schulz	6/6	1.22×10^{-1}	3.32×10^{-1}	7.53×10^{-15}	6.10×10^{-12}
	sheikhi	6/6	7.32×10^{-2}	3.32×10^{-1}	7.68×10^{-15}	4.07×10^{-10}
	svd_default	6/6	2.21×10^{-1}	8.25×10^{-2}	6.13×10^{-3}	1.15×10^{-2}
	svd_jacobian	6/6	2.21×10^{-1}	8.25×10^{-2}	6.13×10^{-3}	1.15×10^{-2}

Table 8.6: Iterative-method convergence by spectrum. Each entry counts successful iterative runs.

Spectrum	10^{-10} threshold	10^{-7} threshold
Well-conditioned	36/36	36/36
Moderately ill-conditioned	36/36	36/36
Severely ill-conditioned	0/36	0/36
Rank deficient	10/36	36/36
Clustered small	24/36	36/36

The relaxed threshold also reduces the median runtime for several higher-order methods on the cases where convergence changes. On the rank-deficient spectrum, the median runtime for `esmaeili` decreases from 3.71×10^{-1} seconds to 1.22×10^{-1} seconds, while `sheikhi` decreases from 3.35×10^{-1} seconds to 1.07×10^{-1} seconds. On clustered small spectra, the same methods move from failed strict-threshold runs to successful relaxed-threshold runs. The severe ill-conditioning case remains unresolved: no iterative method satisfies the stopping criterion on that spectrum under either threshold. This indicates that the limitation is not merely an overly strict stopping threshold, but the interaction between the spectrum, initialization, and iteration budget.

8.2.4 Runtime and Trace Behavior

Figures 8.1 and 8.2 show the runtime comparison generated by the benchmark. Runtime depends strongly on matrix shape. For small square matrices, iterative methods are often faster than SVD-based methods. For larger rectangular matrices, the advantage is less uniform because each iteration requires repeated matrix multiplications.

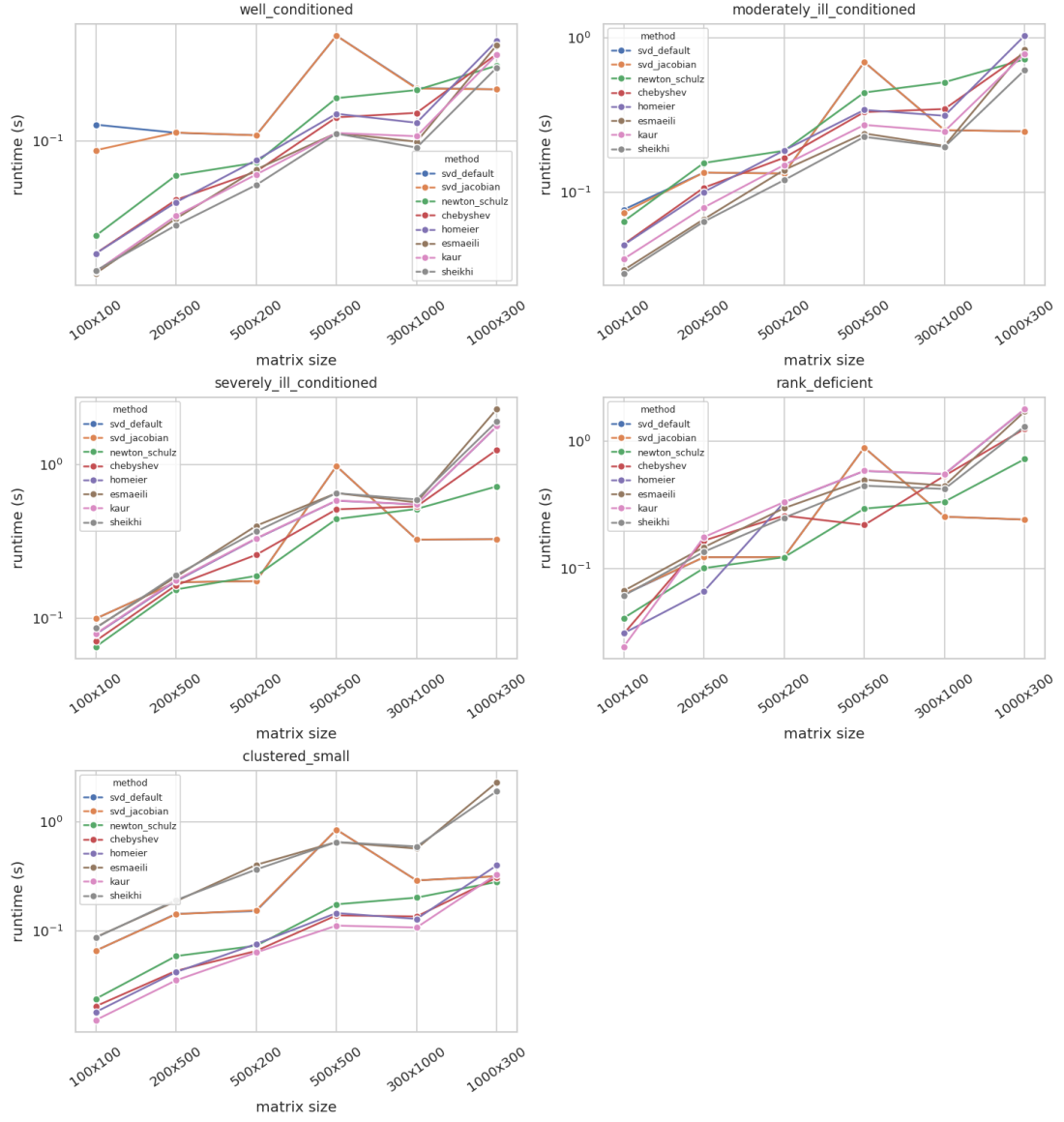


Figure 8.1: Runtime comparison for Experiment A under $\epsilon_{\text{conv}} = 10^{-10}$.

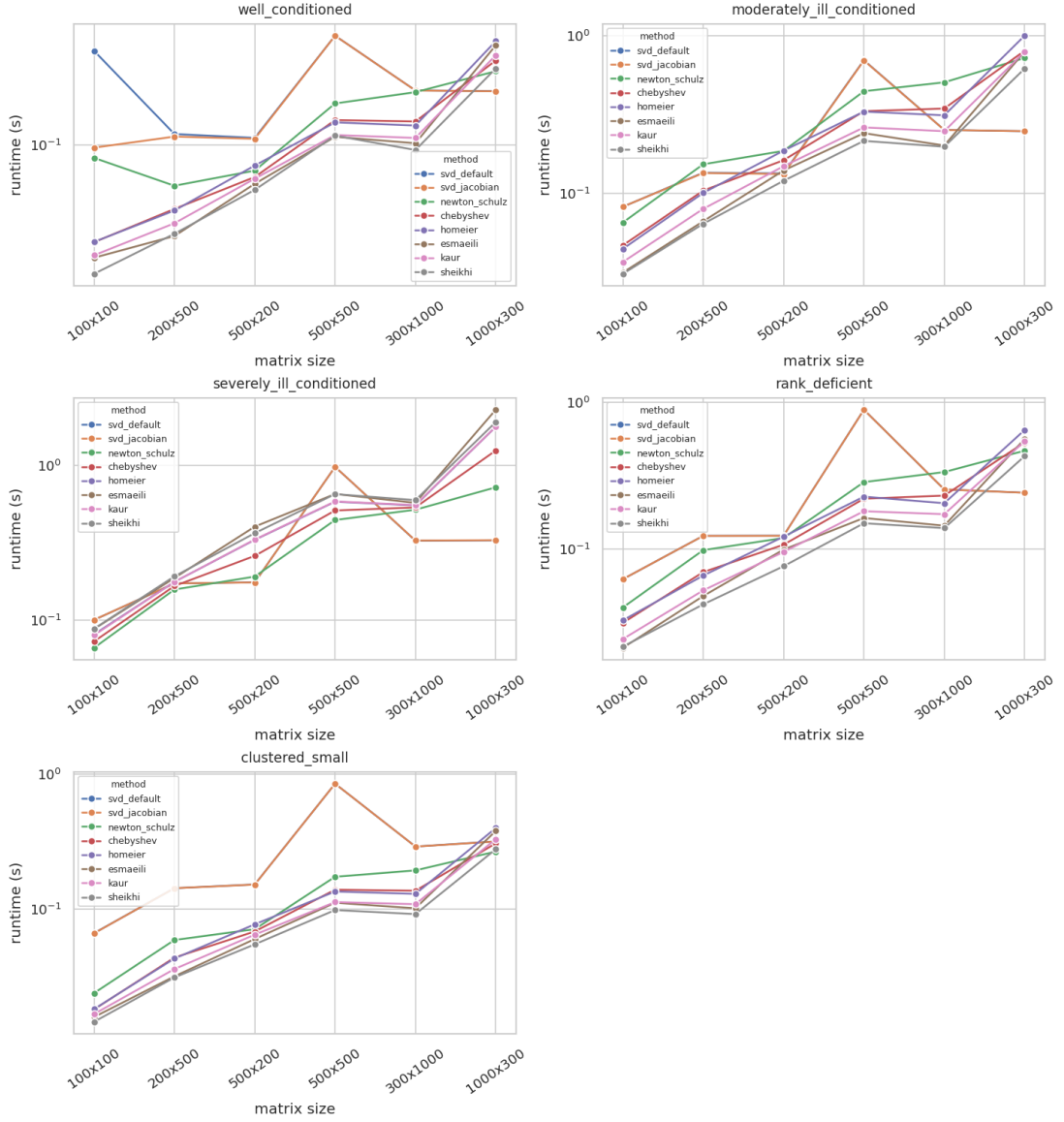


Figure 8.2: Runtime comparison for Experiment A under $\varepsilon_{\text{conv}} = 10^{-7}$.

The residual trace plots provide additional insight into convergence behavior. To keep this chapter focused on aggregate results, the representative trace plots are reported in Appendix B. They show rapid residual decrease for benign spectra and stagnation for the severely ill-conditioned spectrum, matching the zero success rate in Table 8.6.

8.3 Experiment B: ELM Dataset Benchmark

8.3.1 Design

Experiment B evaluates the same pseudoinverse solvers inside an ELM classifier. For a data matrix $\mathbf{X}$, the ELM first constructs a hidden-layer matrix

$$\mathbf{H} = g(\mathbf{X}\mathbf{W} + \mathbf{b}),$$

where g is the activation function and the hidden weights are fixed after random initialization. Training then solves for the output weights $\mathbf{W}_{\text{out}}$. The pseudoinverse-based methods compute

$$\mathbf{W}_{\text{out}} = \mathbf{H}^+ \mathbf{Y},$$

while the Adam [27] baselines optimize the output layer directly for a fixed number of epochs.

The implemented benchmark uses capped subsets of three data sets: MNIST, Fashion-MNIST [28], and ISOLET [29]. Each data set is limited to at most 10,000 training samples and 2,000 test samples after the train–test split or original train–test partition; smaller splits use all available samples below these caps. Each configuration uses hidden widths 128, 256, and 512, one of three hidden-weight initializations (gaussian, he, and xavier), and one of four activations: relu, elu, sigmoid, and tanh. The solver set contains the two SVD methods, the six hyperpower methods, and five Adam baselines with 5, 10, 20, 50, and 100 epochs. The pseudoinverse configuration uses the combined Penrose residual, convergence threshold 10^{-7} , maximum residual 10^3 , maximum iteration count 30, and a runtime guard that stops an iterative pseudoinverse method after it exceeds twice the corresponding default-SVD runtime.

The pseudoinverse experiments are intentionally unregularized. The purpose is to isolate the numerical behavior of the tested algorithms for the Moore–Penrose pseudoinverse itself. Adding Tikhonov regularization would replace the exact pseudoinverse problem with a different optimization objective, introduce an additional hyperparameter λ , and require a separate model-selection procedure. In that setting, observed improvements could come either from the pseudoinverse algorithm or from changing the learning problem through regularization. For this reason, regularization is discussed theoretically as a stabilization mechanism, but its experimental validation is left outside the scope of the present benchmark.

The recorded metrics include training accuracy, test accuracy, micro-F1, macro-F1, balanced accuracy, Matthews correlation coefficient, training time, hidden-matrix condition number, hidden rank, output-weight Frobenius norm, and training residual.

8.3.2 Method-Level Results

Table 8.7 reports the main successful baselines. The two SVD solvers have identical median test accuracy in this run and are successful on all 108 dataset, width, initialization,

and activation configurations. Adam approaches the SVD test accuracy as the epoch budget increases, although it is not guaranteed to recover the same least-squares solution unless the loss and optimization setting match the closed-form objective.

Table 8.7: *Experiment B aggregate results for SVD and Adam methods. Accuracy and runtime are reported as medians across data sets, hidden widths, initializations, and activations.*

Method	Test accuracy	Runtime (s)	Success
svd_default	0.8590	6.570×10^{-1}	108/108
svd_jacobian	0.8590	6.566×10^{-1}	108/108
adam_100_epochs	0.8420	3.082×10^{-1}	108/108
adam_50_epochs	0.8240	1.637×10^{-1}	108/108
adam_20_epochs	0.7846	7.678×10^{-2}	108/108
adam_10_epochs	0.7445	4.793×10^{-2}	108/108
adam_5_epochs	0.6923	3.367×10^{-2}	108/108

The Adam results show the expected trade-off between runtime and accuracy. With only five or ten epochs, Adam is fast but substantially below the SVD solution. At 50 and 100 epochs, it becomes competitive on many configurations while still having a lower median runtime than the SVD baselines. The best individual cases for Adam occur on ISOLET and on selected elu, relu, or tanh configurations, but the SVD solution remains the strongest aggregate baseline.

Table 8.8 summarizes the hyperpower methods in the dataset benchmark. None of these methods meets the implemented convergence and runtime policy on any of the 108 configurations. Their reported accuracies therefore should not be interpreted as converged pseudoinverse ELM solutions.

Table 8.8: *Experiment B aggregate results for iterative hyperpower methods. Accuracy and runtime are reported as medians; all methods are unsuccessful under the implemented stopping policy.*

Method	Test accuracy	Runtime (s)	Success
newton_schulz	0.6260	1.601×10^0	0/108
chebyshev	0.6320	9.450×10^0	0/108
homeier	0.6333	1.816×10^1	0/108
kaur	0.6378	1.821×10^1	0/108
esmaeili	0.6487	2.688×10^1	0/108
sheikhi	0.6650	1.829×10^1	0/108

This is the main difference between Experiments A and B. In the synthetic benchmark, the iterative methods can converge on well-conditioned and moderately ill-conditioned spectra. In the ELM benchmark, the hidden matrices generated by real data, nonlinear activations, initialization choices, and larger hidden widths are not favorable enough for these iterative methods under the configured 30-iteration budget and runtime guard. The direct SVD methods are slower than short Adam runs, but they produce stable closed-form ELM training across every configuration.

8.3.3 Dataset Results

Table 8.9 compares the strongest method families by data set. The SVD value is the mean for `svd_default`; `svd_jacobian` matches it in these runs. These means are taken over the hidden-width, initialization, and activation configurations for the seed used in the benchmark.

Table 8.9: Experiment B mean test accuracy by data set. Values are averaged over hidden widths, initializations, and activation functions.

Data set	SVD	Adam 100	Adam 50	Hyperpower success
Fashion-MNIST	0.7903	0.7732	0.7513	0/216
ISOLET	0.8995	0.8941	0.8718	0/216
MNIST	0.8527	0.8427	0.8269	0/216

ISOLET is the easiest of the three data sets for the tested ELM configurations, with SVD mean accuracy 0.8995. Fashion-MNIST is the most difficult, with SVD mean accuracy 0.7903. MNIST lies between them at 0.8527. Adam with 100 epochs is close to SVD on ISOLET, but the average gap is larger on MNIST and Fashion-MNIST.

The paired comparisons against `svd_default` give the same conclusion. Across the 108 matched configurations from this single-seed run, `adam_100_epochs` is lower by 0.0108 ± 0.0188 test accuracy, and `adam_50_epochs` is lower by 0.0308 ± 0.0412 . Here the “ $\pm$ ” values are standard deviations across configuration-level paired differences. Among the failed hyperpower methods, the strongest average accuracy is obtained by `sheikhi`, but it is still 0.1731 ± 0.0629 below `svd_default` and has no successful convergence cases.

8.3.4 Conditioning and Hidden-Layer Spectra

The hidden-layer matrices are all full rank for the tested hidden widths, but their condition numbers vary by data set, activation, initialization, and width. Averaged over SVD rows, the sigmoid activation has the largest hidden condition number, about 1.65×10^2 , while `elu`,

relu, and tanh are lower, at approximately 3.40×10^1 , 3.78×10^1 , and 2.18×10^1 , respectively. This conditioning difference does not by itself determine accuracy: elu has the highest average SVD accuracy in the current run, while the best-conditioned tanh rows are not the most accurate on average. The result suggests that hidden representation quality and numerical conditioning must be interpreted together.

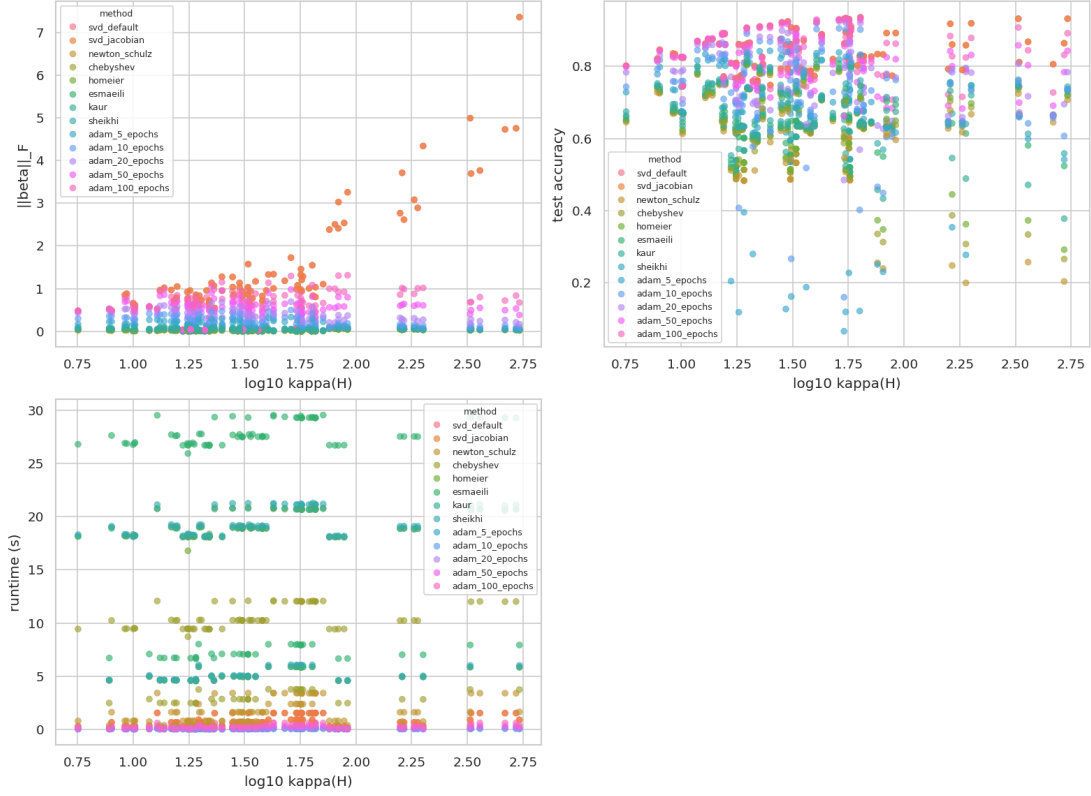


Figure 8.3: Experiment B relationship between hidden-matrix conditioning, output-weight norm, test accuracy, and training runtime.

The hidden-width sweep shows the expected benefit of a larger hidden representation. For `svd_default`, mean test accuracy increases from 0.8164 at width 128 to 0.8509 at width 256 and 0.8752 at width 512. The same pattern appears for `adam_100_epochs`, which increases from 0.8058 to 0.8400 and then to 0.8642, but remains below the SVD baseline at each width.

The smallest singular value of the hidden matrix decreases as the hidden width increases, as shown in Figure 8.5. This trend is consistent with the random-matrix intuition developed in Chapter 7: for a fixed sample size N , increasing the hidden width L narrows the sample-width margin $\sqrt{N} - \sqrt{L}$. Figures 8.6 and 8.7 summarize this relation across data sets, where N denotes the post-sampling training size. The experiments support the spectral interpretation proposed in Chapter 7: better-conditioned hidden-layer matrices lead to more stable pseudoinverse solutions and improved robustness.

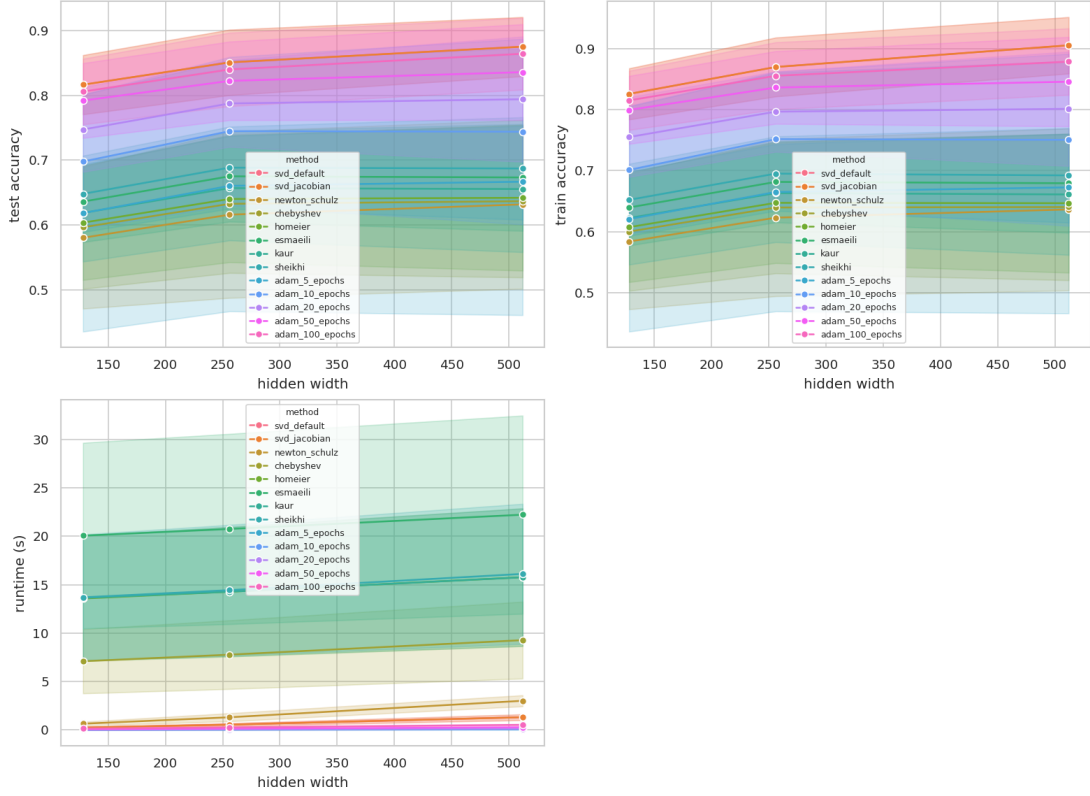


Figure 8.4: Experiment B method comparison across hidden widths 128, 256, and 512

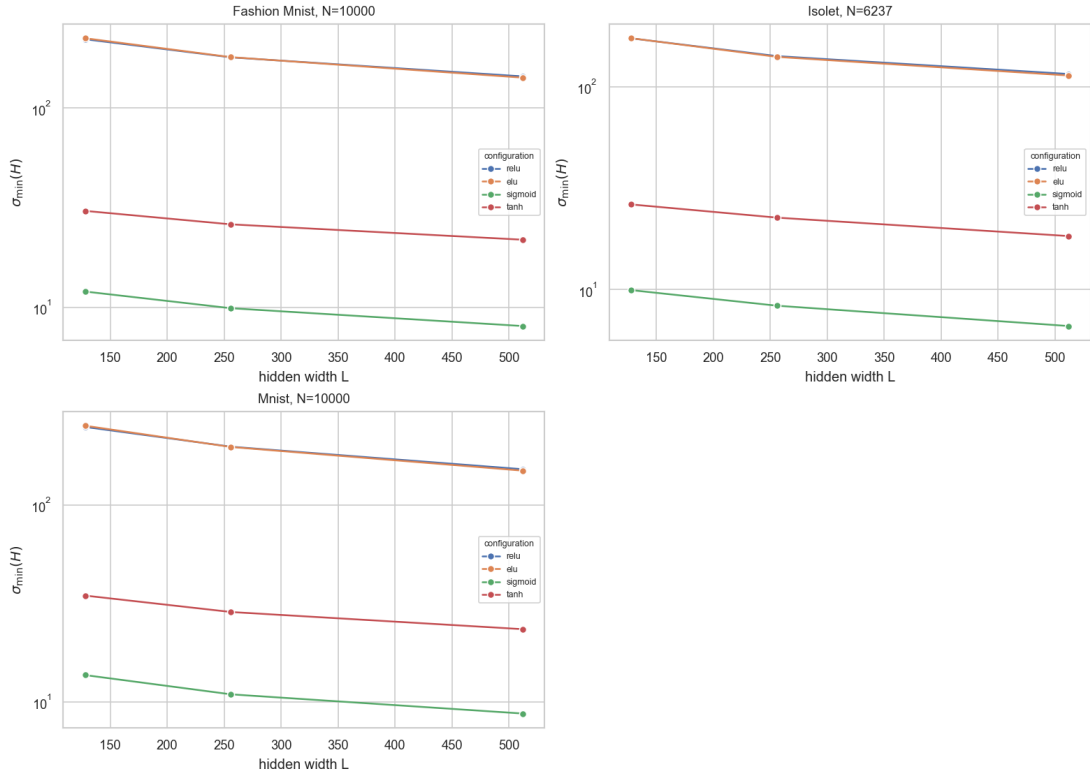


Figure 8.5: Experiment B relationship between hidden width L and the smallest singular value $\sigma_{\min}(H)$ of the hidden-layer matrix.

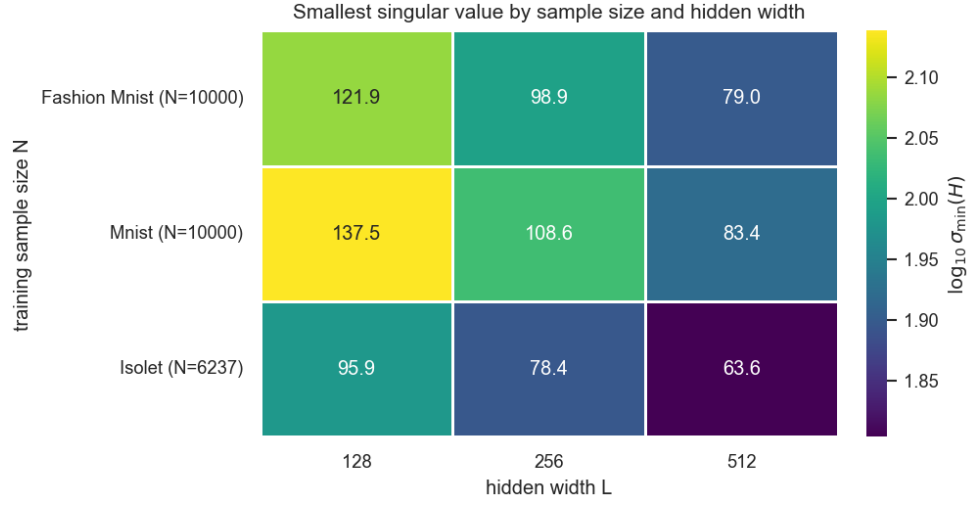


Figure 8.6: Experiment B mean smallest hidden singular value by training sample size N and hidden width L .

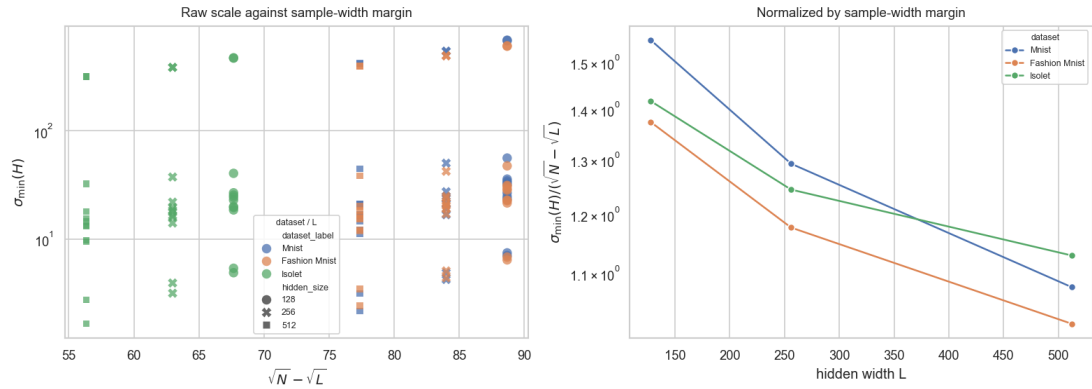


Figure 8.7: Experiment B relationship between $\sigma_{\min}(\mathbf{H})$, the sample-width margin $\sqrt{N} - \sqrt{L}$, and hidden width.

Figures 8.8–8.10 show representative hidden-layer singular-value spectra. These plots support the interpretation that the ELM hidden matrices are numerically manageable for SVD, but not easy enough for the tested hyperpower methods under the implemented stopping policy.

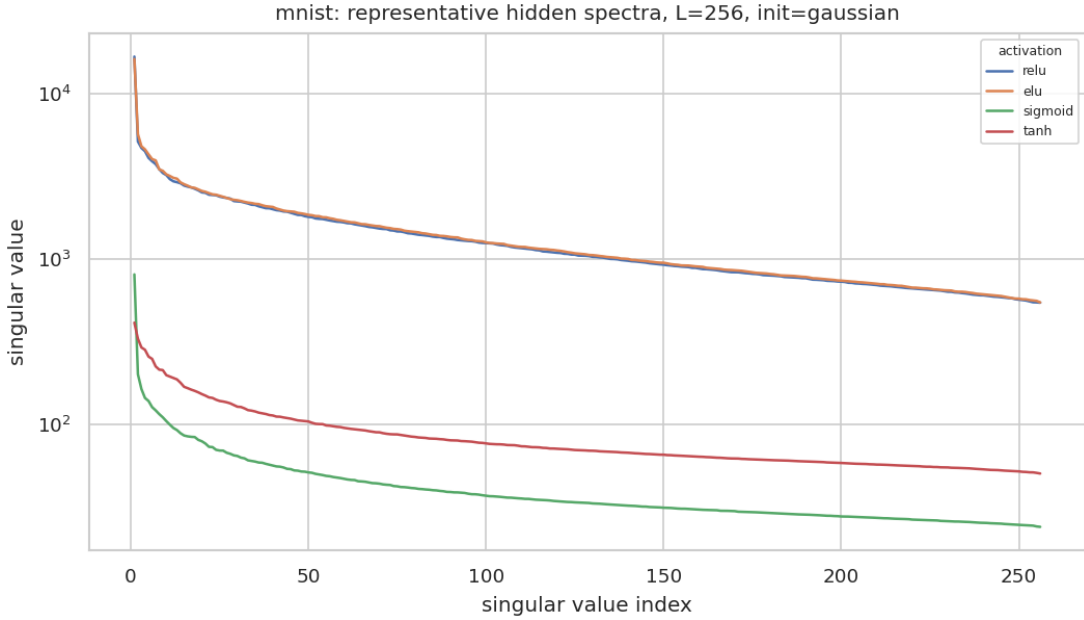


Figure 8.8: Representative hidden-layer singular-value spectrum for MNIST.

8.4 Discussion

Experiment A shows that the hyperpower methods are viable when the singular spectrum is benign. They converge on all well-conditioned and moderately ill-conditioned synthetic matrices, and the relaxed 10^{-7} threshold improves convergence on rank-deficient and clustered-small spectra. However, the same methods fail on severely ill-conditioned spectra under both thresholds.

Experiment B shows that this controlled success does not automatically transfer to ELM training. In the single-seed configuration sweep, the direct SVD solvers are successful in all 108 ELM configurations and give the best aggregate accuracy. Adam provides a useful speed-accuracy trade-off: short runs are fast but inaccurate, while 50 or 100 epochs approach the SVD solution on some dataset, width, initialization, and activation configurations. The iterative hyperpower methods are not competitive in the current ELM configuration because they do not meet the convergence policy before the runtime guard stops them.

The combined result is that SVD remains the reference method for reliable ELM output weight computation. Hyperpower iterations can still be useful as numerical methods, but their

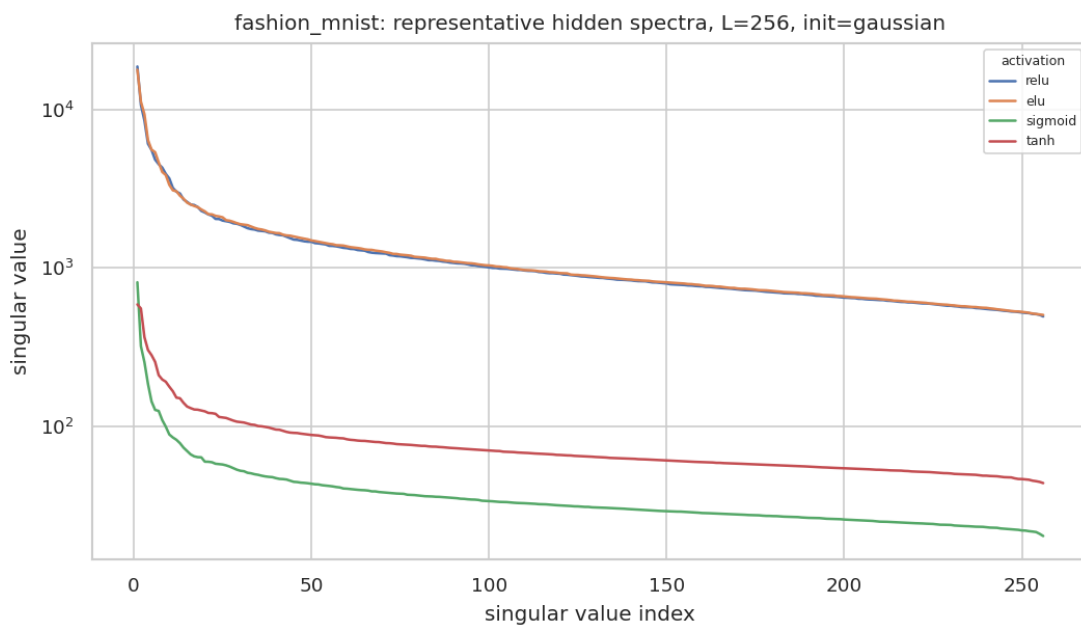


Figure 8.9: Representative hidden-layer singular-value spectrum for Fashion-MNIST.

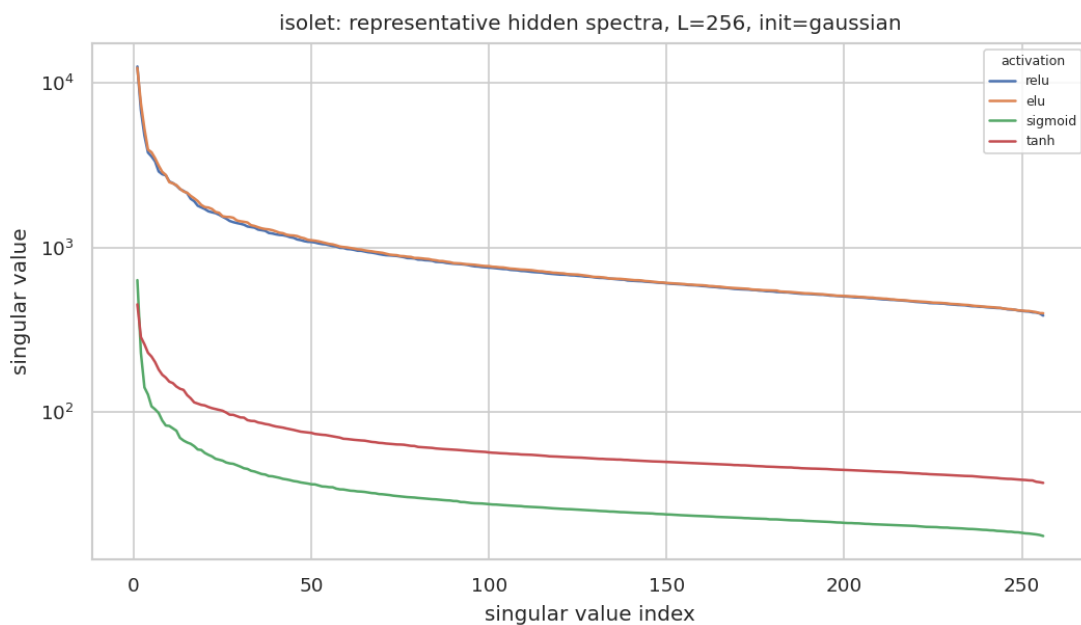


Figure 8.10: Representative hidden-layer singular-value spectrum for ISOLET.

practical value depends strongly on the hidden matrix spectrum, initialization, stopping rule, and permitted runtime. For ELM training, these conditions are not guaranteed by the data sets and activations tested here.

8.5 Chapter Summary

The experiments support three main conclusions. First, matrix conditioning dominates pseudoinverse solver behavior: benign spectra allow accurate hyperpower convergence, whereas severe ill-conditioning defeats the tested iterative methods. Second, relaxing the convergence threshold from 10^{-10} to 10^{-7} improves practical convergence on difficult synthetic spectra without resolving the most ill-conditioned case. Third, in this ELM benchmark, SVD-based pseudoinverse training is the most reliable and accurate among the tested configurations, while Adam offers a tunable approximation and the tested hyperpower solvers fail under the implemented stopping policy.

Conclusion and Future Work

Chapter Overview:

This chapter summarizes the main findings of the thesis. It reviews the theoretical role of generalized inverses, the numerical behavior of SVD-based and hyperpower-based pseudoinverse methods, and the implications for Extreme Learning Machine. The chapter also states the limitations of the current work and outlines future directions for regularized, large-scale, and more extensively validated ELM experiments.

9.1 Conclusion

This thesis studied generalized inverses from both mathematical and computational perspectives, with emphasis on the Moore–Penrose pseudoinverse and its application to Extreme Learning Machine (ELM). The starting point was the observation that classical matrix inversion is not available for rectangular, singular, or rank-deficient matrices, whereas many practical learning and optimization problems naturally lead to such matrices. The Moore–Penrose inverse provides a canonical way to solve least-squares and minimum-norm problems in this setting.

The theoretical chapters established the core linear algebra needed for the rest of the work: generalized inverses, projection matrices, orthogonal decompositions, least-squares solutions, and the singular value decomposition. The SVD was shown to be especially important

because it exposes the singular values directly, and these values control both the size of the pseudoinverse and the sensitivity of the computed solution. Small singular values make the pseudoinverse large and can strongly amplify perturbations.

The numerical analysis chapters then compared SVD-based pseudoinverse computation with hyperpower iterative methods. SVD-based methods are more expensive in some settings, but they are stable, rank-aware, and reliable for rectangular and rank-deficient matrices. Hyperpower methods are attractive because they use matrix multiplications and may converge rapidly in favorable cases, but their behavior depends strongly on initialization, the fixed-point convergence tolerance, and the singular-value spectrum.

These conclusions carry directly into ELM. In ELM, the output weights are computed from

$$\beta = \mathbf{H}^+ \mathbf{T},$$

where $\mathbf{H}$ is the hidden-layer output matrix. Therefore, instability in pseudoinverse computation becomes instability in the learned output weights. The thesis showed that the conditioning of $\mathbf{H}$, especially its smallest nonzero singular value, is a central factor in the reliability of ELM training.

The study suggests that numerical stability in pseudoinverse-based learning should be understood not merely as an algorithmic issue, but fundamentally as a spectral property of the hidden-layer representation.

9.2 Experimental Findings

Experiment A used synthetic matrices with controlled spectra. The results show that the tested hyperpower methods can converge accurately on well-conditioned and moderately ill-conditioned matrices. Relaxing the convergence threshold from 10^{-10} to 10^{-7} improved practical convergence on rank-deficient and clustered-small spectra. However, severe ill-conditioning remained difficult for all iterative methods. The SVD-based solvers were successful across all tested matrix structures.

Experiment B evaluated the same solver families inside ELM classifiers on capped subsets of MNIST, Fashion-MNIST, and ISOLET. Its aggregate means, standard deviations, and medians summarize variation across data-set, hidden-width, initialization, and activation configurations. In this benchmark, the SVD-based methods were the most reliable and accurate among the tested configurations. Adam-trained output layers approached SVD test accuracy as the epoch budget increased, but they did not represent the same closed-form least-squares computation. The tested hyperpower pseudoinverse methods did not satisfy the implemented

convergence and runtime policy on the dataset benchmark, which confirms that success on controlled synthetic matrices does not automatically transfer to ELM hidden-layer matrices.

9.3 Limitations

The main limitation of the experiments is that the reported runs use a single random seed and capped data-set subsets. Reported standard deviations therefore measure variation across tested configurations, not variability across independent random seeds. The results are best interpreted as benchmark evidence for the tested configurations, not as a full statistical comparison across random initializations or full-data training regimes. Additional seeds and larger uncapped runs would be needed to estimate variability in accuracy, runtime, and hidden-matrix conditioning.

A second limitation is that regularization is discussed theoretically but not validated as a separate experimental method. This choice is intentional: the experiments isolate the numerical behavior of algorithms for the unregularized Moore–Penrose pseudoinverse. Adding Tikhonov regularization would introduce a different optimization objective, an additional hyperparameter λ , and a separate model-selection problem. It would then be harder to distinguish whether observed improvements come from the pseudoinverse algorithm itself or from changing the learning problem through regularization. Therefore, regularization is treated as a theoretical stabilization mechanism in this thesis, while its experimental validation is left as future work.

A third limitation is the convergence policy for iterative pseudoinverse methods. The experiments used fixed-point convergence as the implemented condition, together with practical iteration and runtime safeguards; residuals were used for diagnostics rather than as a residual-tolerance stopping condition. The analysis shows that convergence deltas alone do not fully characterize Moore–Penrose accuracy for rectangular or rank-deficient pseudoinverse problems, so Penrose residuals and projector-aware diagnostics should be developed further for future iterative implementations.

9.4 Future Work

Future work should first extend the experiments to multiple random seeds, uncapped or larger data-set subsets, additional hidden widths, and more data sets. This would make it possible to separate stable method-level behavior from seed-specific variation in the hidden-layer matrix.

Second, regularized ELM should be evaluated experimentally. This includes comparing SVD-based ridge solutions, QR or Cholesky-based solvers used with care, and iterative methods applied to regularized systems. Such experiments would test whether regularization can make iterative approaches practically competitive without hiding conditioning problems caused by forming normal equations.

Third, convergence assessment for hyperpower pseudoinverse methods should be refined. Future implementations should keep the fixed-point convergence condition explicit and report Penrose residuals, symmetry residuals, and projector-aware diagnostics as separate accuracy measures, rather than relying on identity residuals that only apply cleanly to square nonsingular or full-row-rank right-inverse settings.

Finally, larger-scale implementations could study whether matrix-multiplication-heavy hyperpower methods become more attractive on GPUs or batched workloads. This would help clarify the practical trade-off between the robustness of SVD and the hardware-friendly structure of iterative methods.

9.5 Final Remarks

The main conclusion of the thesis is that the pseudoinverse is not merely a formal linear algebra object in ELM training. It is a numerical computation whose stability depends on the matrix spectrum and on the algorithm used to compute it. SVD remains the most reliable reference method in the settings studied here, while hyperpower methods require favorable spectra, careful initialization, and appropriate convergence tolerances to be useful in practice.

Appendix A

Algorithms

This appendix gives the algorithmic forms referred to in the thesis. The purpose is not to reproduce a production numerical library, but to make the computational steps clear: how an SVD is obtained, how the pseudoinverse is assembled from singular values, and how the iterative hyperpower solvers used in the experiments are organized.

The notation follows the main chapters. For a matrix $\mathbf{A}$, $\mathbf{A}_{:,j}$ denotes its j -th column, $\|\cdot\|_F$ is the Frobenius norm, and τ denotes a singular-value or convergence tolerance. All indices are written using one-based notation.

A.1 Golub–Reinsch SVD

The Golub–Reinsch method computes the SVD in two conceptually separate phases. First, orthogonal Householder transformations reduce the dense matrix to bidiagonal form. Then implicit shifted QR iterations diagonalize the bidiagonal matrix. Since both phases use orthogonal transformations, the method avoids forming $\mathbf{A}^T \mathbf{A}$ and is therefore much more stable than computing singular values through the normal equations.

The algorithm may be summarized as

$$\mathbf{A} = \mathbf{U}_0 \mathbf{B} \mathbf{V}_0^T,$$

where $\mathbf{B}$ is bidiagonal, followed by an orthogonal diagonalization of $\mathbf{B}$. This is the algorithmic foundation behind many stable dense SVD implementations.

Algorithm A.1. Golub–Reinsch SVD

Input: Matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$, $m \geq n$, and tolerance τ

Output: Matrices $\mathbf{U}$, Σ , and $\mathbf{V}^T$ such that $\mathbf{A} \approx \mathbf{U}\Sigma\mathbf{V}^T$

```

1  $\mathbf{U} \leftarrow \mathbf{I}_m, \mathbf{V} \leftarrow \mathbf{I}_n, \mathbf{B} \leftarrow \mathbf{A};$ 
2 for  $j \leftarrow 1$  to  $n$  do
3   Construct a Householder reflector  $\mathbf{P}_j$  that zeros the entries below  $B_{j,j}$ ;
4    $\mathbf{B} \leftarrow \mathbf{P}_j\mathbf{B}, \mathbf{U} \leftarrow \mathbf{U}\mathbf{P}_j^T;$ 
5   if  $j < n$  then
6     Construct a Householder reflector  $\mathbf{Q}_j$  that zeros the entries right of  $B_{j,j+1}$ ;
7      $\mathbf{B} \leftarrow \mathbf{B}\mathbf{Q}_j, \mathbf{V} \leftarrow \mathbf{V}\mathbf{Q}_j;$ 
8 while  $\mathbf{B}$  has a nonzero superdiagonal entry above tolerance  $\tau$  do
9   Identify the largest unreduced bidiagonal block of  $\mathbf{B}$ ;
10  Compute a Wilkinson shift from the trailing  $2 \times 2$  block of  $\mathbf{B}^T\mathbf{B}$ ;
11  Apply one implicit shifted QR sweep to the active bidiagonal block;
12  Accumulate the right Givens rotations into  $\mathbf{V}$ ;
13  Accumulate the left Givens rotations into  $\mathbf{U}$ ;
14  Set negligible superdiagonal entries to zero for deflation;
15  $\Sigma \leftarrow |\text{diag}(\mathbf{B})|;$ 
16 Move signs into  $\mathbf{U}$  or  $\mathbf{V}$  and sort singular values in descending order;
17 return  $\mathbf{U}, \Sigma, \mathbf{V}^T$ 

```

A.2 Divide-and-Conquer SVD

Divide-and-conquer SVD also begins with bidiagonal reduction. The difference lies in the second phase: instead of repeatedly applying QR sweeps to the whole bidiagonal matrix, the method splits the bidiagonal problem into smaller subproblems and then merges them through rank-one corrections. This is why divide-and-conquer implementations are often faster when the singular vectors are required.

Algorithm A.2. Divide-and-Conquer SVD

Input: Matrix $A \in \mathbb{R}^{m \times n}$, tolerance τ , and cutoff size $n_{\min}$

Output: Matrices U , Σ , and V^T

- 1 $(U_0, B, V_0) \leftarrow \text{HouseholderBidiagonalization}(A);$
 - 2 $(Q, \Sigma, P^T) \leftarrow \text{BidiagonalDCSVD}(B, \tau, n_{\min});$
 - 3 $U \leftarrow U_0 Q;$
 - 4 $V^T \leftarrow P^T V_0^T;$
 - 5 **return** U, Σ, V^T
-

Algorithm A.3. Bidiagonal Divide-and-Conquer SVD

Input: Bidiagonal matrix B , tolerance τ , and cutoff size $n_{\min}$

Output: SVD factors Q , Σ , and P^T

- 1 **if** $\text{size}(B) \leq n_{\min}$ **then**
 - 2 **return** $\text{GolubReinschSVD}(B, \tau)$
 - 3 Choose a split index that separates B into two bidiagonal blocks;
 - 4 Represent the remaining coupling as a rank-one correction;
 - 5 Compute the SVD of the first block recursively;
 - 6 Compute the SVD of the second block recursively;
 - 7 Transform the coupling vector into the singular-vector bases of the two subproblems;
 - 8 Deflate negligible coupling components and repeated singular values;
 - 9 Solve the secular equation for the merged singular values;
 - 10 Reconstruct the merged singular vectors from the secular-vector formula;
 - 11 Assemble Q , Σ , and P^T ;
 - 12 **return** Q, Σ, P^T
-

The central numerical task in the merge step is the solution of a secular equation for

a diagonal matrix plus a rank-one update. In practical libraries, deflation is essential: small coupling entries and nearly repeated singular values reduce the size of the merge problem and improve both speed and accuracy.

A.3 One-Sided Jacobi SVD

The one-sided Jacobi method applies plane rotations from the right until the columns of the working matrix are nearly orthogonal. After convergence, the singular values are the column norms, the normalized columns give the left singular vectors, and the accumulated rotations give the right singular vectors.

The orthogonality test in Algorithm A.4 is based on the column inner product c . When $|c|$ is small relative to $\sqrt{ab}$, the columns are already sufficiently orthogonal and no rotation is applied.

A.4 Thresholded SVD Pseudoinverse

The SVD-based solvers used in the experiments first compute an SVD and then invert only the singular values above a tolerance. This is the direct and rank-aware construction of the Moore–Penrose pseudoinverse.

In the implementation, `svd_default` uses the default SVD routine provided by the linear algebra backend, while `svd_jacobian` requests a Jacobi SVD driver when it is available and falls back to the default driver otherwise. Both methods use the same thresholded reconstruction in Algorithm A.5.

A.5 Hyperpower Pseudoinverse Iteration

The iterative methods in the experiments share the same outer solver structure. They start from a scaled transpose

$$\mathbf{X}_0 = \frac{\mathbf{A}^T}{\|\mathbf{A}\|_1 \|\mathbf{A}\|_\infty},$$

apply a method-specific polynomial update, and stop according to residual, convergence, or safeguard criteria.

For rectangular or rank-deficient matrices, an identity residual is not a universal measure of pseudoinverse convergence. The experiments therefore record Penrose residuals, especially

$$r_1(\mathbf{X}) = \frac{\|\mathbf{AXA} - \mathbf{A}\|_F}{\|\mathbf{A}\|_F}, \quad r_2(\mathbf{X}) = \frac{\|\mathbf{XAX} - \mathbf{X}\|_F}{\|\mathbf{X}\|_F},$$

Algorithm A.4. One-Sided Jacobi SVD**Input:** Matrix $A \in \mathbb{R}^{m \times n}$, tolerance τ , and maximum sweeps $s_{\max}$ **Output:** Matrices U , Σ , and V^T

```

1  $G \leftarrow A, V \leftarrow I_n, s \leftarrow 0;$ 
2 repeat
3    $s \leftarrow s + 1;$ 
4   for  $p \leftarrow 1$  to  $n - 1$  do
5     for  $q \leftarrow p + 1$  to  $n$  do
6        $a \leftarrow \|G_{:,p}\|_2^2, b \leftarrow \|G_{:,q}\|_2^2;$ 
7        $c \leftarrow G_{:,p}^T G_{:,q};$ 
8       if  $|c| > \tau \sqrt{ab}$  then
9         Compute a Jacobi rotation  $J_{pq}$  that diagonalizes the  $2 \times 2$  Gram
          submatrix;
10         $G \leftarrow G J_{pq};$ 
11         $V \leftarrow V J_{pq};$ 
12 until all column pairs satisfy the orthogonality test or  $s = s_{\max};$ 
13 for  $j \leftarrow 1$  to  $n$  do
14    $\sigma_j \leftarrow \|G_{:,j}\|_2;$ 
15    $\Sigma_{j,j} \leftarrow \sigma_j;$ 
16   if  $\sigma_j > \tau$  then
17      $U_{:,j} \leftarrow G_{:,j} / \sigma_j;$ 
18   else
19     Choose  $U_{:,j}$  as a vector orthogonal to the previously accepted columns;
20 Sort singular values in descending order and apply the same permutation to  $U$  and  $V$ ;
21 return  $U, \Sigma, V^T$ 

```

Algorithm A.5. Thresholded SVD Pseudoinverse**Input:** Matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ and tolerance τ **Output:** Pseudoinverse approximation $\mathbf{X} \approx \mathbf{A}^+$

```

1  $(\mathbf{U}, \Sigma, \mathbf{V}^T) \leftarrow \text{SVD}(\mathbf{A});$ 
2  $\Sigma^+ \leftarrow \mathbf{0}_{n \times m};$ 
3 for each singular value  $\sigma_i$  do
4   if  $\sigma_i > \tau$  then
5      $(\Sigma^+)_{i,i} \leftarrow 1/\sigma_i;$ 
6   else
7      $(\Sigma^+)_{i,i} \leftarrow 0;$ 
8  $\mathbf{X} \leftarrow \mathbf{V}\Sigma^+\mathbf{U}^T;$ 
9 return  $\mathbf{X}$ 

```

Algorithm A.6. Safeguarded Hyperpower Pseudoinverse Iteration**Input:** Matrix $\mathbf{A}$, update map Φ , tolerance τ , maximum iterations $k_{\max}$, and failure threshold $\rho_{\max}$ **Output:** Approximation $\mathbf{X} \approx \mathbf{A}^+$ and stopping information

```

1  $\mathbf{X} \leftarrow \mathbf{A}^T / (\|\mathbf{A}\|_1 \|\mathbf{A}\|_\infty);$ 
2  $r_{\text{old}} \leftarrow \text{Residual}(\mathbf{A}, \mathbf{X});$ 
3 for  $k \leftarrow 1$  to  $k_{\max}$  do
4    $\mathbf{X}_{\text{old}} \leftarrow \mathbf{X};$ 
5    $\mathbf{X} \leftarrow \Phi(\mathbf{A}, \mathbf{X}_{\text{old}});$ 
6    $r \leftarrow \text{Residual}(\mathbf{A}, \mathbf{X});$ 
7    $\delta \leftarrow \|\mathbf{X} - \mathbf{X}_{\text{old}}\|_F / (1 + \|\mathbf{X}_{\text{old}}\|_F);$ 
8   if  $r$  or  $\delta$  is not finite then
9     Stop and mark the run as failed;
10  if  $r > \rho_{\max}$  then
11    Stop and mark the run as failed;
12  if  $r < \tau$  or  $\delta < \tau$  then
13    Stop and mark the run as converged;
14   $r_{\text{old}} \leftarrow r;$ 
15 return  $\mathbf{X}$  and the final stopping status

```

and use their maximum for the combined Penrose residual.

The update map Φ in Algorithm A.6 depends on the chosen method. With $S_k = AX_k$, the update polynomials used in the implementation include the following cases:

Table A.1: Hyperpower update formulas used in the implementation.

Method	Update formula
newton_schulz	$X_{k+1} = X_k(2I - S_k)$
chebyshev	$X_{k+1} = X_k(3I - S_k(3I - S_k))$
homeier	$X_{k+1} = X_k(I + \frac{1}{2}(I - S_k)(I + (2I - S_k)^2))$
esmaeili	$X_{k+1} = X_k(9I - 26S_k + 34S_k^2 - 21S_k^3 + 5S_k^4)$
kaur	$X_{k+1} = X_k(5I - 10S_k + 10S_k^2 - 5S_k^3 + S_k^4)$
sheikhi	$Y_k = 7S_k^2 - 16S_k + 11I, X_{k+1} = \frac{1}{4}X_k(4I - Y_kS_k)Y_k$

The formulas are implemented in factorized or nested form so that intermediate products can be reused. This matters because a higher convergence order does not automatically imply faster runtime: each iteration may require several matrix multiplications, and the iteration can still fail when the spectrum or initialization is unfavorable.

Experiment A Residual Traces

This appendix contains representative residual trace plots for the iterative pseudoinverse methods in Experiment A. They complement the aggregate results in Chapter 8 by showing how the residual evolves over iterations for different singular-value spectra and convergence thresholds.

B.1 Convergence Threshold 1e-10

The following traces correspond to the stricter convergence threshold

$$\varepsilon_{\text{conv}} = 10^{-10}.$$

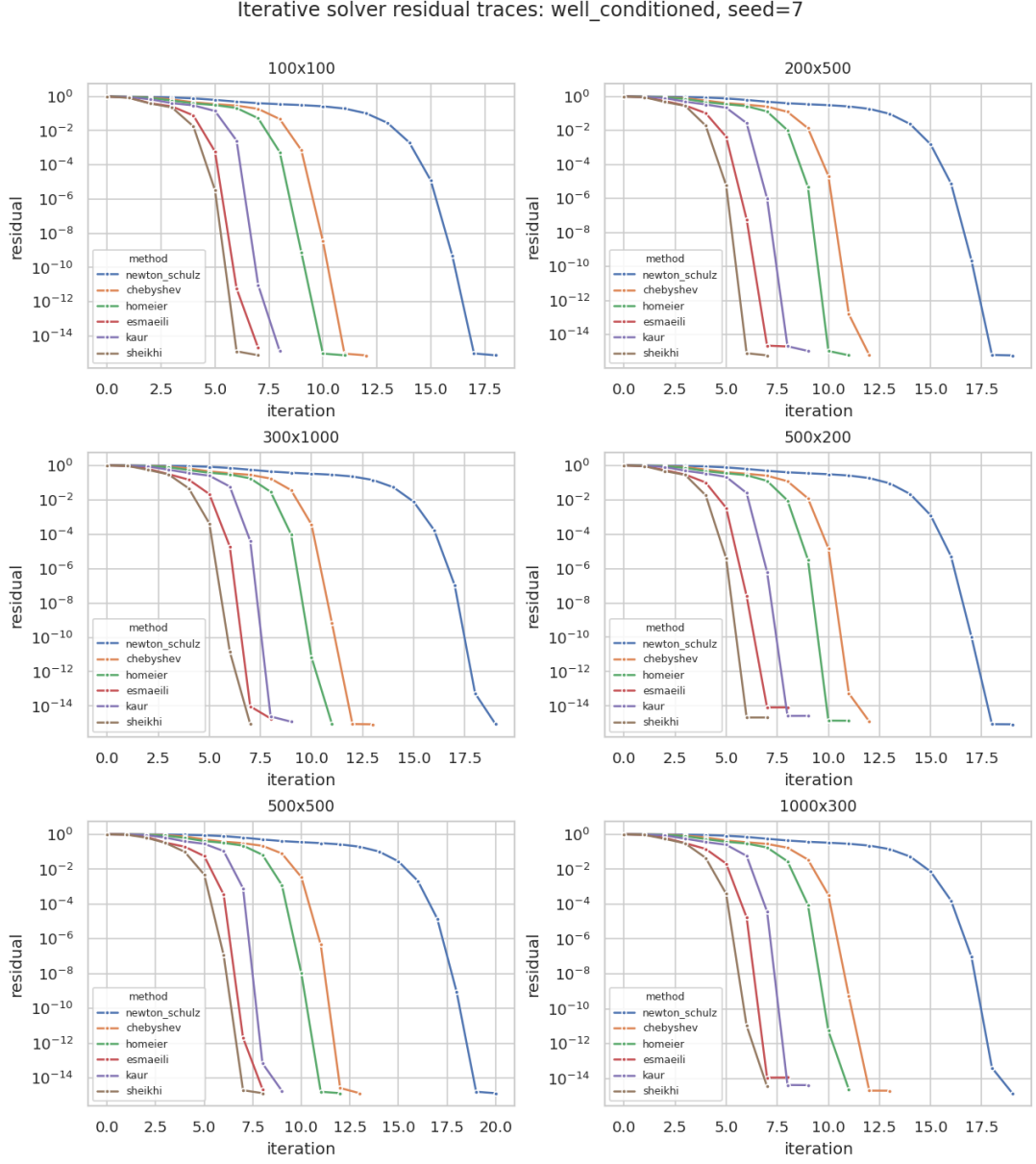


Figure B.1: Residual traces for the well-conditioned spectrum under $\varepsilon_{\text{conv}} = 10^{-10}$.

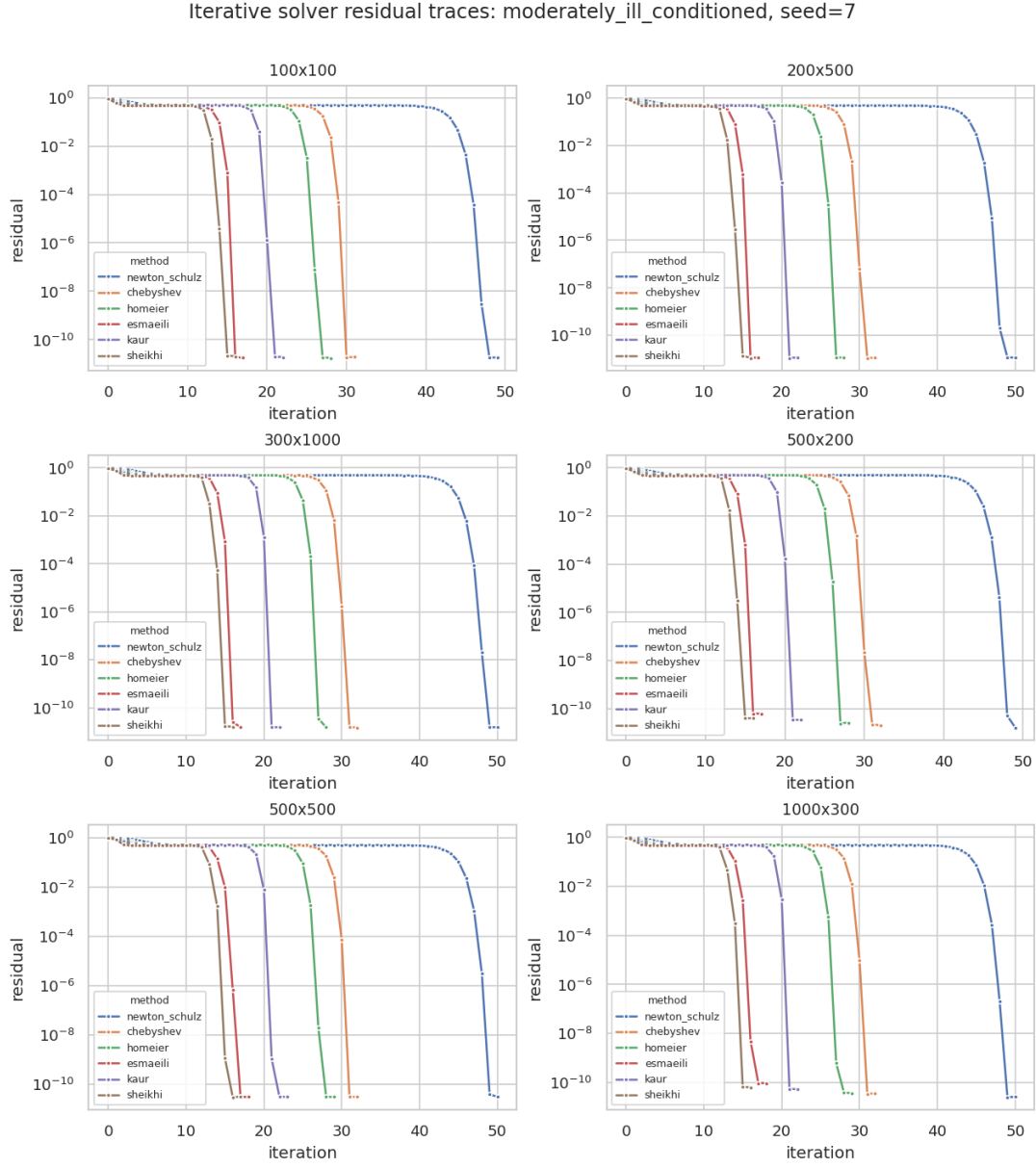
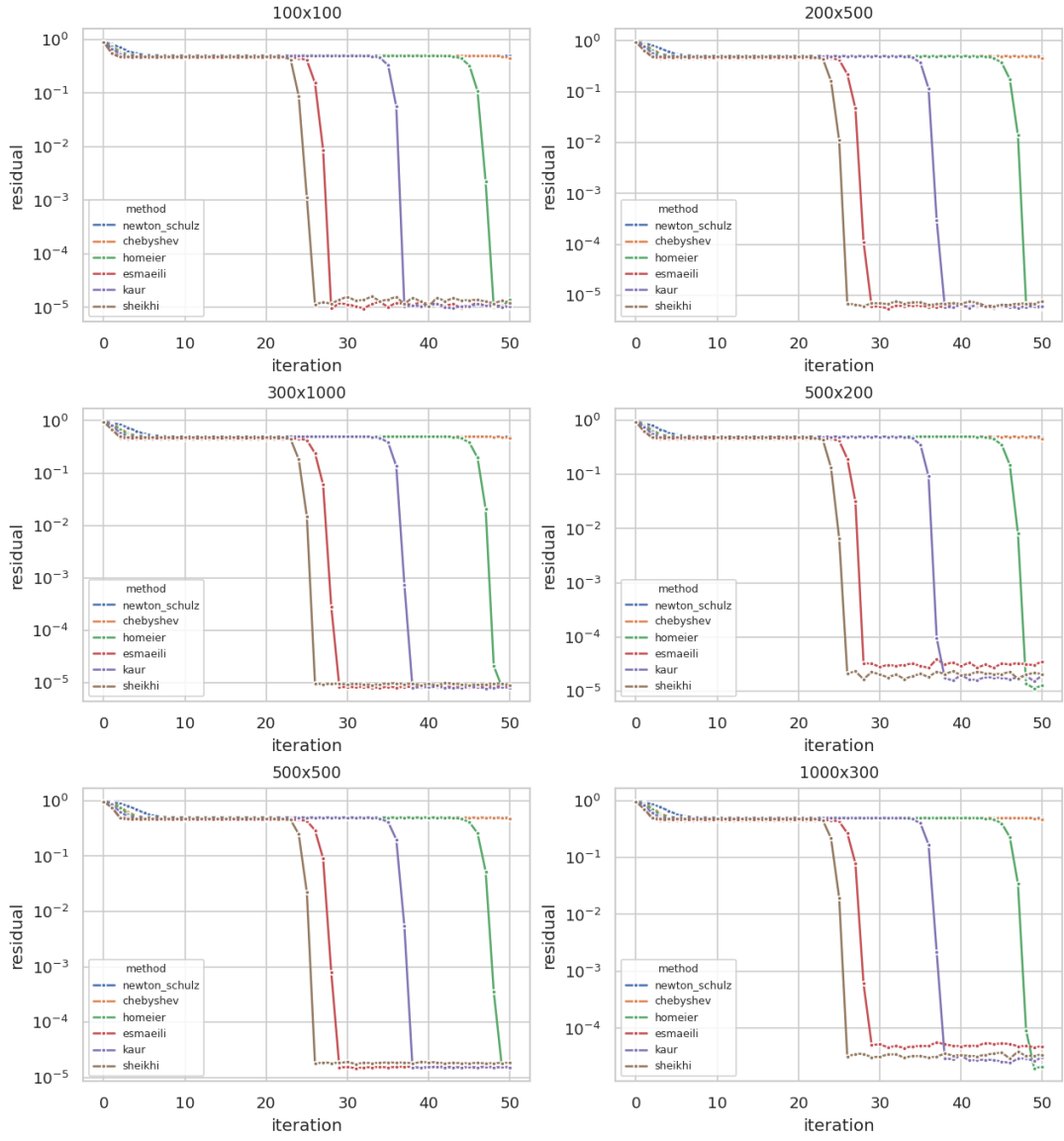
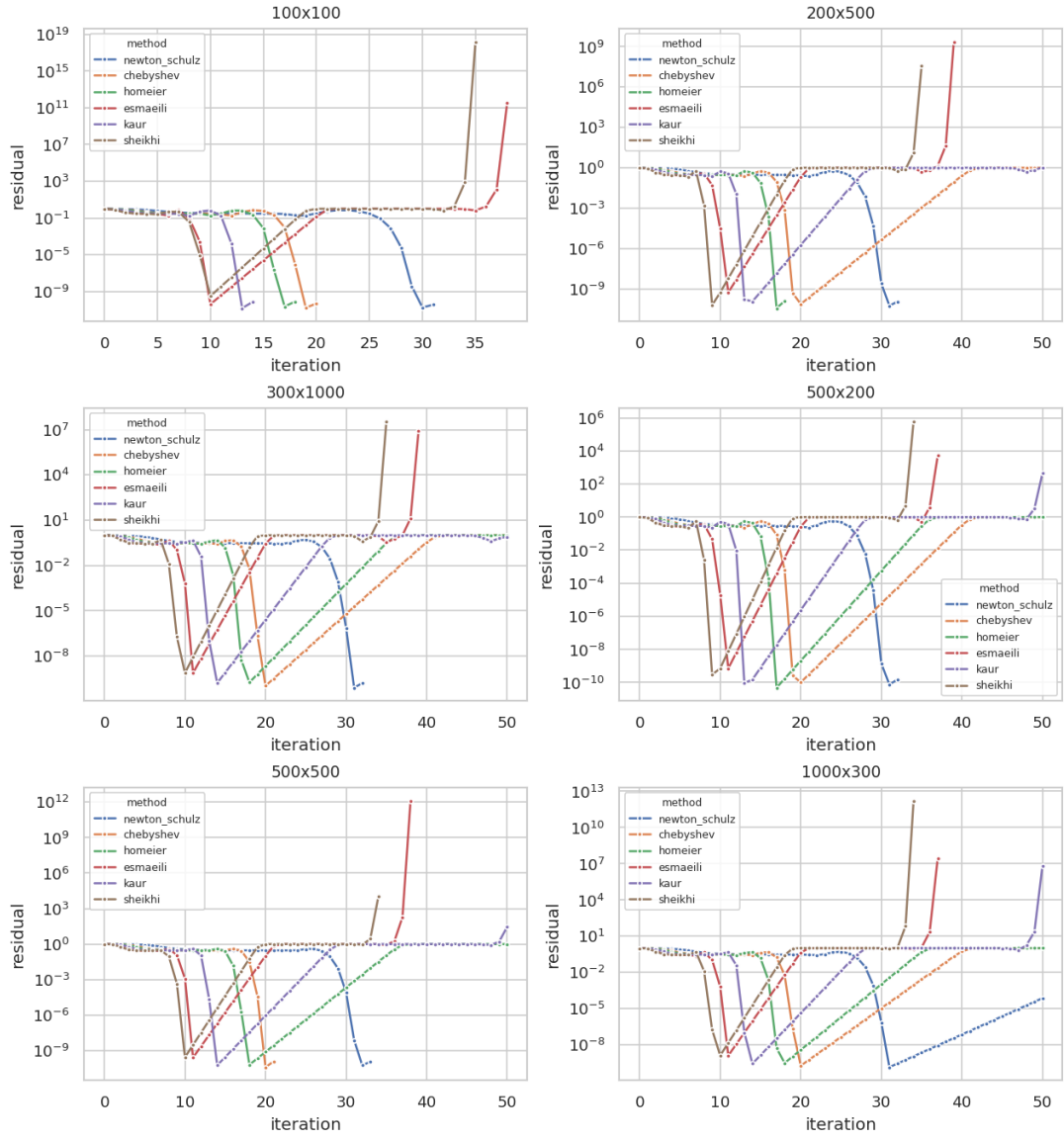


Figure B.2: Residual traces for the moderately ill-conditioned spectrum under $\epsilon_{\text{conv}} = 10^{-10}$.

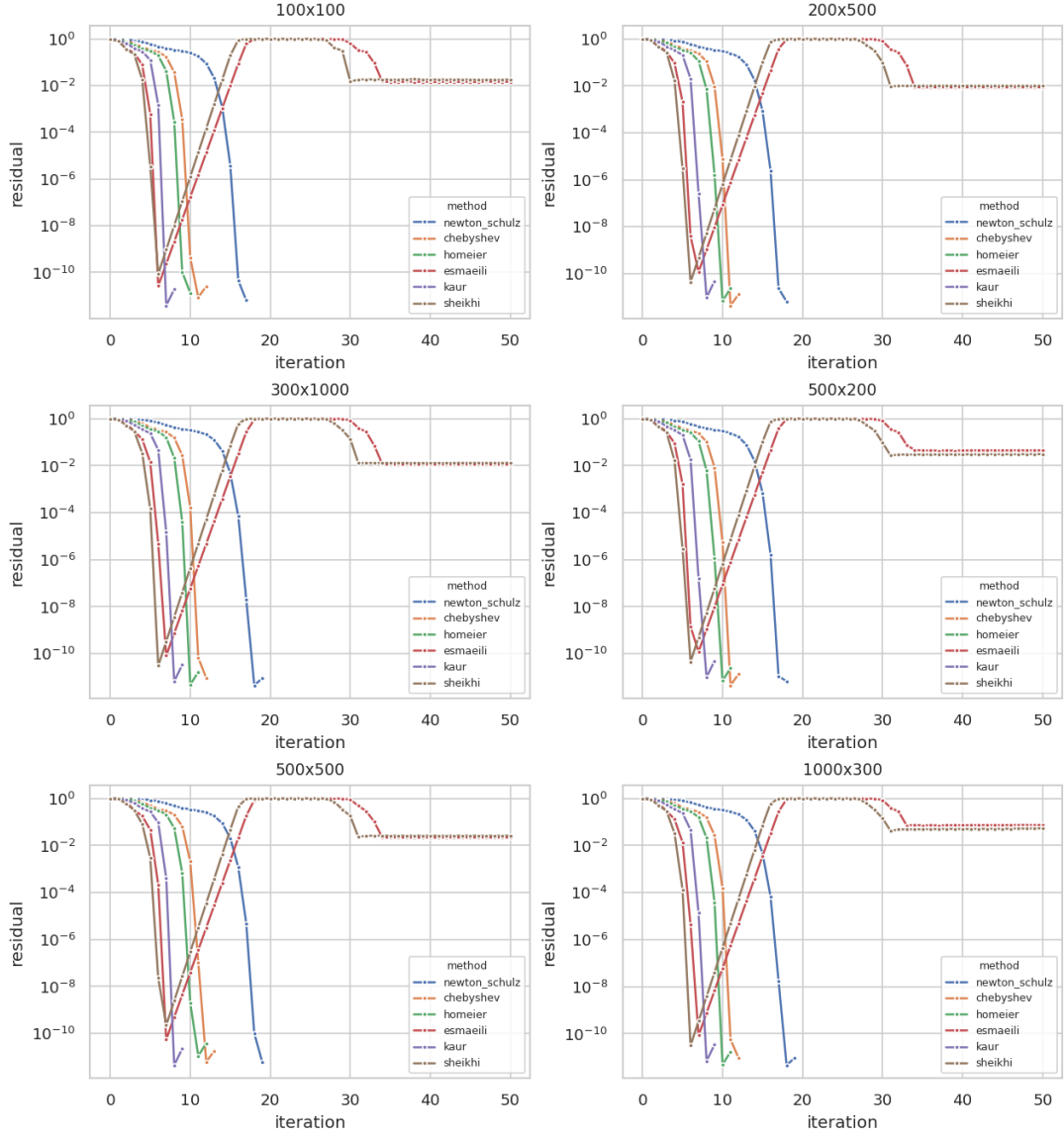
Iterative solver residual traces: severely_ill_conditioned, seed=7

**Figure B.3:** Residual traces for the severely ill-conditioned spectrum under $\varepsilon_{\text{conv}} = 10^{-10}$.

Iterative solver residual traces: rank_deficient, seed=7

**Figure B.4:** Residual traces for the rank-deficient spectrum under $\varepsilon_{\text{conv}} = 10^{-10}$.

Iterative solver residual traces: clustered_small, seed=7

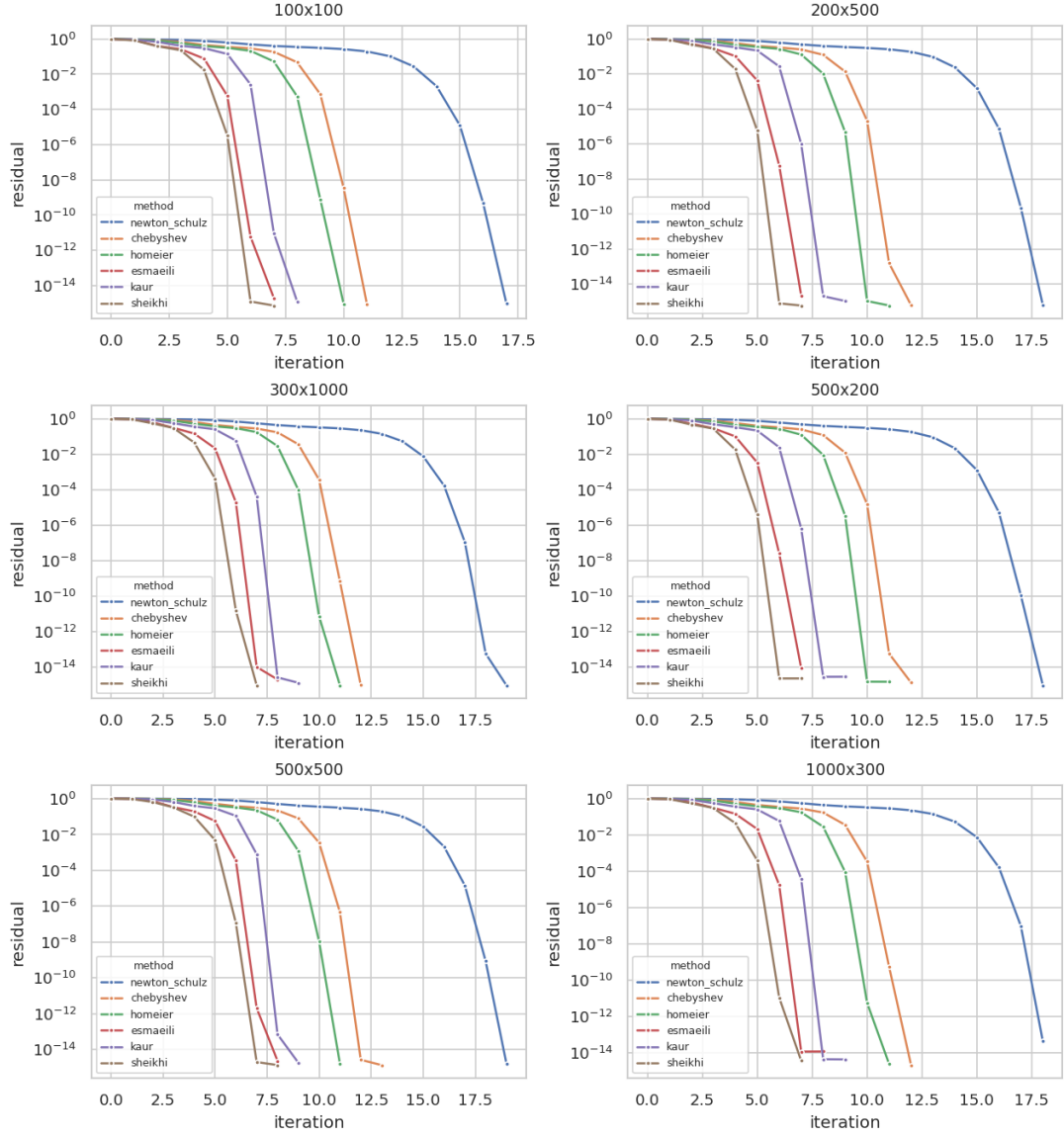
**Figure B.5:** Residual traces for clustered small singular values under $\epsilon_{\text{conv}} = 10^{-10}$.

B.2 Convergence Threshold $1e-7$

The following traces correspond to the relaxed convergence threshold

$$\epsilon_{\text{conv}} = 10^{-7}.$$

Iterative solver residual traces: well_conditioned, seed=7

**Figure B.6:** Residual traces for the well-conditioned spectrum under $\varepsilon_{\text{conv}} = 10^{-7}$.

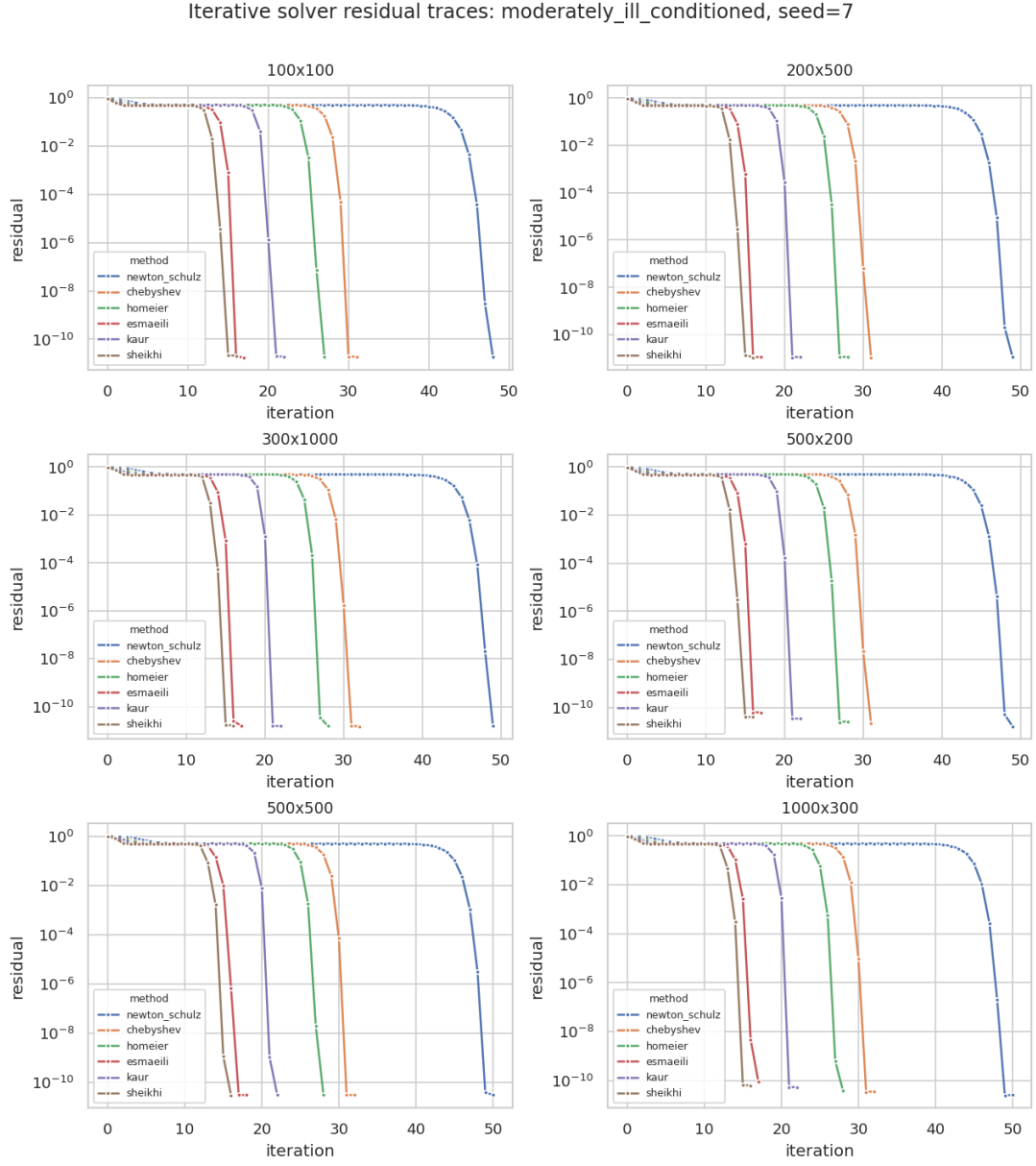


Figure B.7: Residual traces for the moderately ill-conditioned spectrum under $\varepsilon_{\text{conv}} = 10^{-7}$.

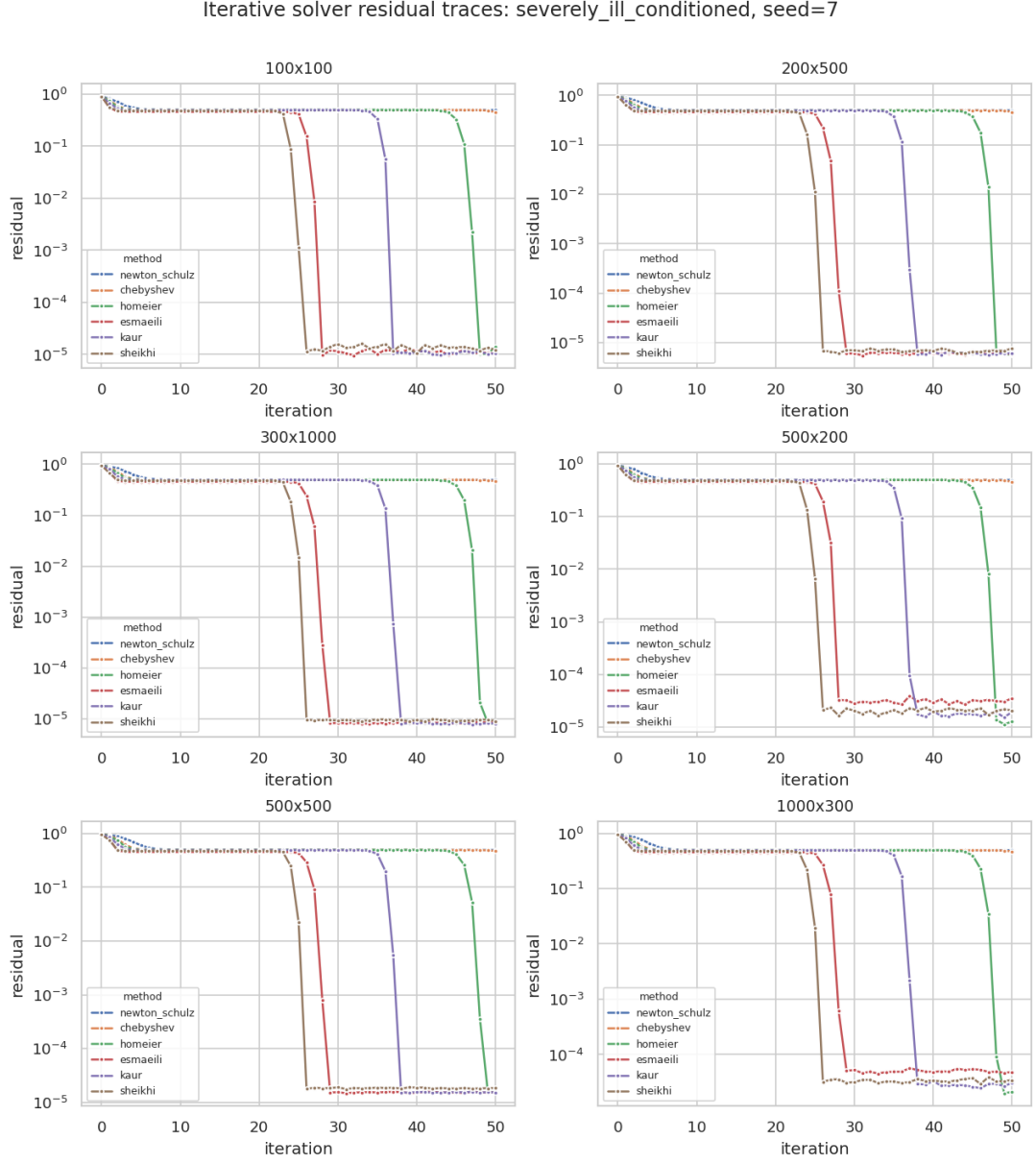
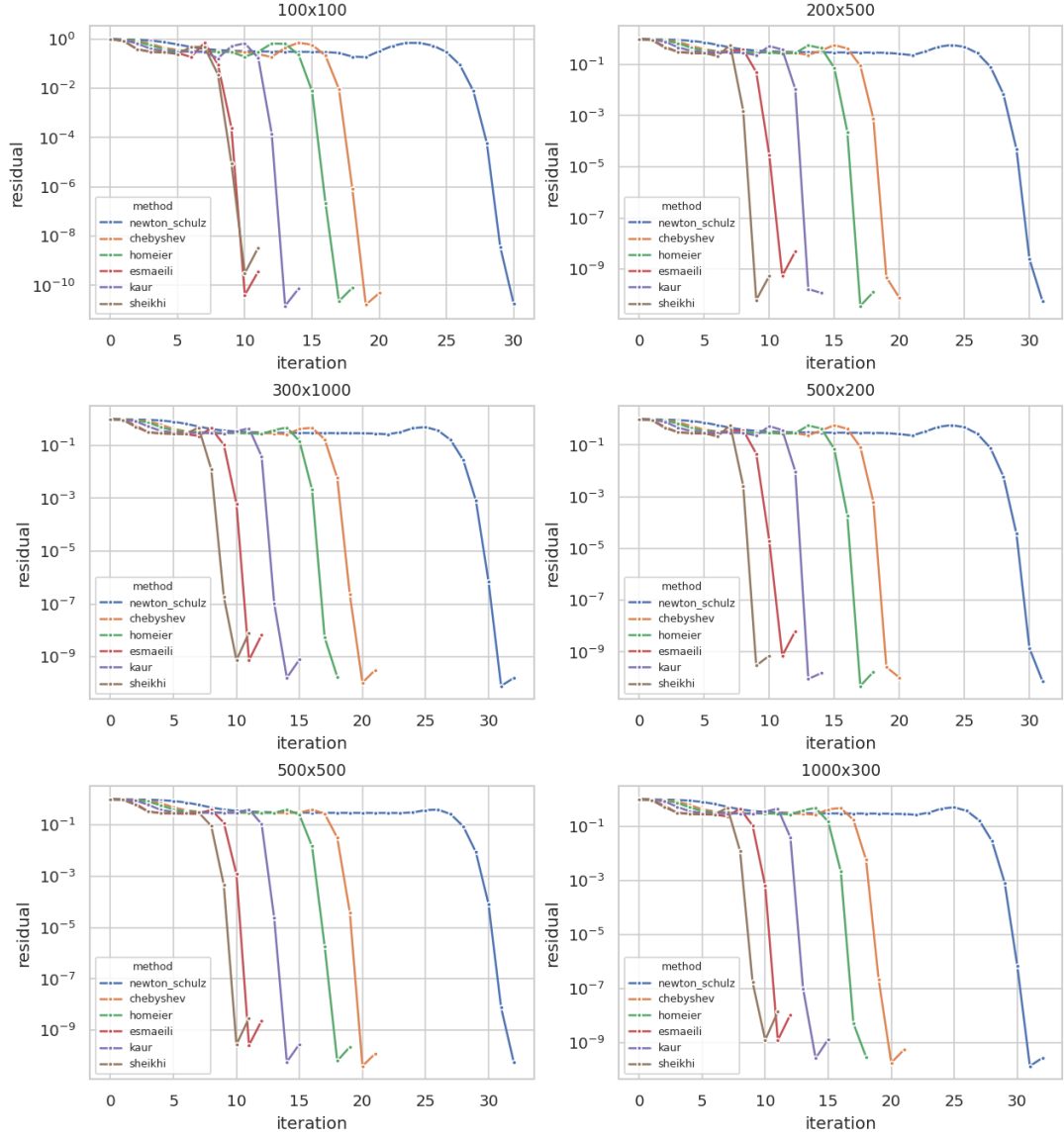
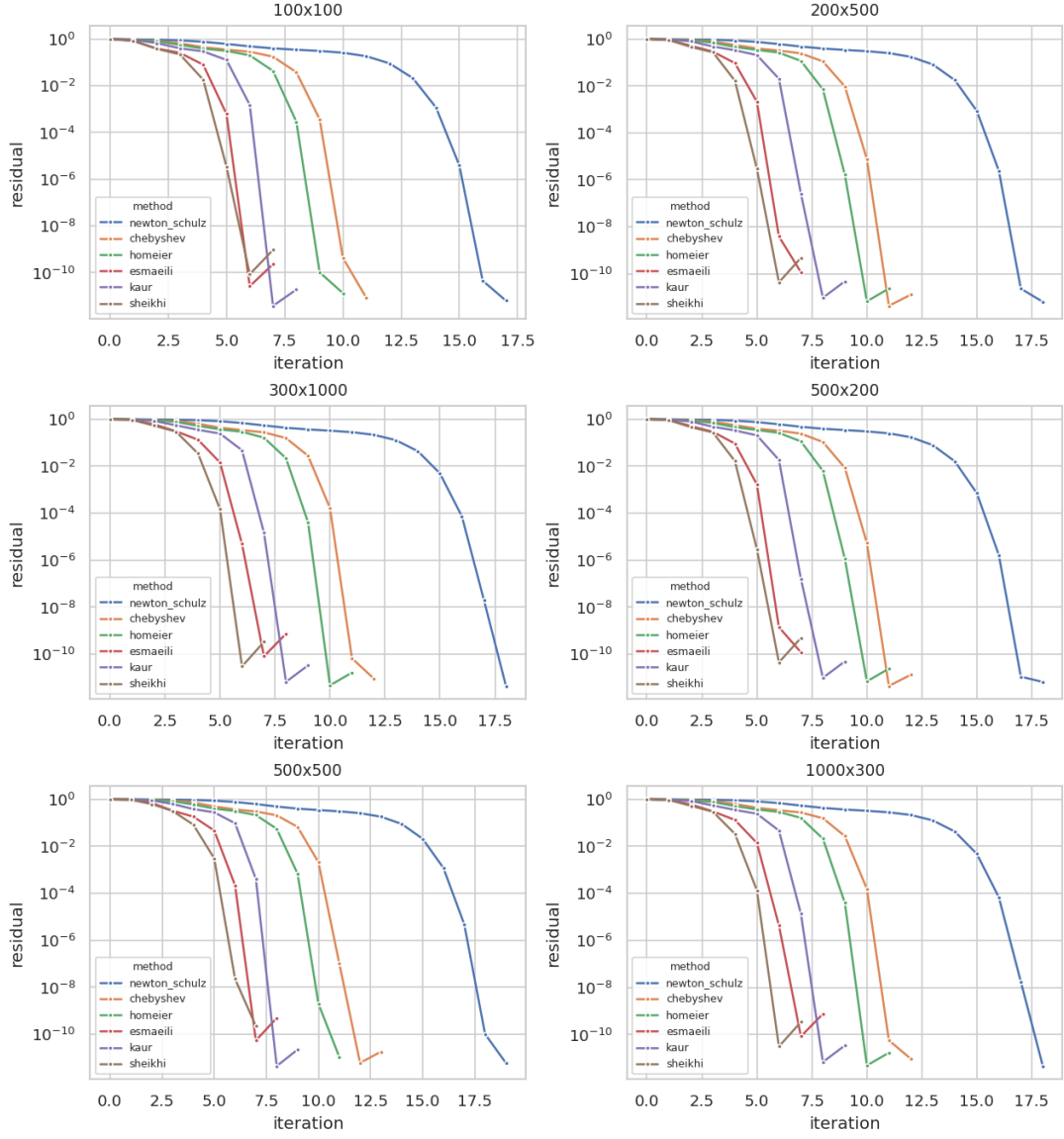


Figure B.8: Residual traces for the severely ill-conditioned spectrum under $\epsilon_{\text{conv}} = 10^{-7}$.

Iterative solver residual traces: rank_deficient, seed=7

**Figure B.9:** Residual traces for the rank-deficient spectrum under $\epsilon_{\text{conv}} = 10^{-7}$.

Iterative solver residual traces: clustered_small, seed=7

**Figure B.10:** Residual traces for clustered small singular values under $\varepsilon_{\text{conv}} = 10^{-7}$.

Bibliography

- [1] Adi Ben-Israel and Thomas NE Greville. *Generalized Inverses: Theory and Applications*. Springer, 2003.
- [2] Lloyd N. Trefethen and David Bau, III. *Numerical Linear Algebra*. Society for Industrial and Applied Mathematics, 1997.
- [3] Nicholas J. Higham. *Accuracy and Stability of Numerical Algorithms*. Society for Industrial and Applied Mathematics, second edition, 2002.
- [4] Alston S Householder. Unitary triangularization of a nonsymmetric matrix. *Journal of the ACM (JACM)*, 5(4):339–342, 1958.
- [5] Wallace Givens. Computation of plain unitary rotations transforming a general matrix to triangular form. *Journal of the Society for Industrial and Applied Mathematics*, 6(1):26–50, 1958.
- [6] John GF Francis. The QR transformation: a unitary analogue to the LR transformation—part 1. *The Computer Journal*, 4(3):265–271, 1961.
- [7] Eliakim H Moore. On the reciprocal of the general algebraic matrix. *Bulletin of the American Mathematical Society*, 26:294–295, 1920.
- [8] Arne Bjerhammar. *Application of Calculus of Matrices to Method of Least Squares: With Special Reference to Geodetic Calculations*. Royal Institute of Technology, 1951.
- [9] Roger Penrose. A generalized inverse for matrices. *Mathematical Proceedings of the Cambridge Philosophical Society*, 51(3):406–413, 1955.
- [10] Gene Golub and William Kahan. Calculating the singular values and pseudo-inverse of a matrix. *Journal of the Society for Industrial and Applied Mathematics, Series B: Numerical Analysis*, 2(2):205–224, 1965.
- [11] G. H. Golub and C. Reinsch. Singular value decomposition and least squares solutions. *Numerische Mathematik*, 14(5):403–420, 1970.

-
- [12] Ming Gu and Stanley C Eisenstat. A divide-and-conquer algorithm for the bidiagonal svd. *SIAM Journal on Matrix Analysis and Applications*, 16(1):79–92, 1995.
- [13] Magnus R Hestenes. Inversion of matrices by biorthogonalization and related results. *Journal of the Society for Industrial and Applied Mathematics*, 6(1):51–90, 1958.
- [14] P. P. M. de Rijk. A one-sided jacobi algorithm for computing the singular value decomposition on a vector computer. *SIAM Journal on Scientific and Statistical Computing*, 10(2):359–371, 1989.
- [15] M. Altman. An optimum cubically convergent iterative method of inverting a linear bounded operator in hilbert space. *Pacific Journal of Mathematics*, 10(4):1107–1113, 1960.
- [16] Adi Ben-Israel. An iterative method for computing the generalized inverse of an arbitrary matrix. *Mathematics of Computation*, 19(91):452–455, 1965.
- [17] Adi Ben-Israel. A note on an iterative method for generalized inversion of matrices. *Mathematics of Computation*, 20(95):439–440, 1966.
- [18] Hou-Biao Li, Ting-Zhu Huang, Yong Zhang, Xing-Ping Liu, and Tong-Xiang Gu. Chebyshev-type methods and preconditioning techniques. *Applied Mathematics and Computation*, 218(2):260–270, 2011.
- [19] Hamid Esmaeili, Raziye Erfanifar, and Mahdis Rashidi. A fourth-order iterative method for computing the moore-penrose inverse. *Journal of Hyperstructures*, 6(1):52–67, 2017.
- [20] Manpreet Kaur, Munish Kansal, and Sanjeev Kumar. An efficient hyperpower iterative method for computing weighted moore-penrose inverse. *AIMS Mathematics*, 5(3):1680–1692, 2020.
- [21] Salman Sheikhi and Hamid Esmaeili. A new iterative method to find the moore-penrose inverse. *Applicationes Mathematicae*, 52(2):205–217, 2025.
- [22] Guang-Bin Huang, Qin-Yu Zhu, and Chee-Kheong Siew. Extreme learning machine: theory and applications. *Neurocomputing*, 70(1-3):489–501, 2006.
- [23] Irina Perfilieva, Nicolas Madrid, Manuel Ojeda-Aciego, Piotr Artiemjew, and Agnieszka Niemczynowicz. A critical analysis of the theoretical framework of the extreme learning machine. *Neurocomputing*, 621:129298, 2025.
- [24] Vijay Manikandan Janakiraman, XuanLong Nguyen, and Dennis Assanis. Stochastic gradient based extreme learning machines for stable online learning of advanced combustion engines. *Neurocomputing*, 177:304–316, 2016.

- [25] Rossella Cancelliere, Mario Gai, Patrick Gallinari, and Luca Rubini. OCREP: an optimally conditioned regularization for pseudoinversion-based neural training. *Neural Networks*, 71:76–87, 2015.
- [26] Mark Rudelson and Roman Vershynin. Smallest singular value of a random rectangular matrix. *Communications on Pure and Applied Mathematics: A Journal Issued by the Courant Institute of Mathematical Sciences*, 62(12):1707–1739, 2009.
- [27] Diederik P Kingma. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- [28] Han Xiao, Kashif Rasul, and Roland Vollgraf. Fashion-MNIST: a novel image dataset for benchmarking machine learning algorithms. *arXiv preprint arXiv:1708.07747*, 2017.
- [29] Ron Cole and Mark Fanty. ISOLET. UCI Machine Learning Repository, 1991. DOI: <https://doi.org/10.24432/C51G69>.